#ASLI ENGINEERING

What is System Design

SWIPE

BY

ARPIT BHAYANI

What is System Design?

Set of requirements

- ↳ decide architecture
- ↳ decide components
- ↳ decide modules

} how they interact with each other

↓

To solve the problem*

"Product Development" ← almost

Why is it so popular?

Every single "tech" product is a "system" this has been "designed"

companies are building product and need you all to design it

Why understanding system design is important?

- this is what people do at work
- everything is practical
- you can see it from Day 1 of joining the company
- once you grow in your career,

  you will spend 80% of time doing this.

System design is hence relevant for literally everyone!!

ARPIT BHAYANI

Side effects of System Design

- it will make everything uninteresting (else)
- you solve real problems, not something made up
- break down problem statement
- re-wires your brain to think in a structured way
  - ↳ considering all possible cases to deliver a great user experience!

What will we do when we design a system?

- break down problem statement into solvable sub-problems
- decide on key components and responsibilities
- decide on boundaries of each component
- touch upon key challenges in scaling it
- make our architecture faulttolerant and available

ARPIT BHAYANI

#ASLI ENGINEERING

How to approach system design?

SWIPE

BY

ARPIT BHAYANI

How to approach System Design?

System Design is extremely practical

and there is a structured way to tackle the situations.

Take baby steps, no matter what!

understand the problem statement

Without having a thorough understanding of the problem at hand

we would easily digress

break it down into components (essential)

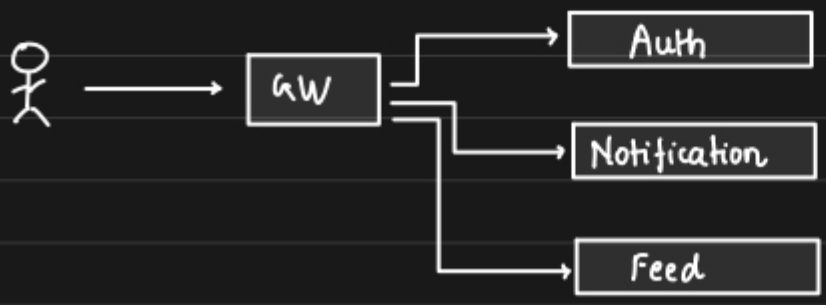
Note: do not create components for the sake of it } to start with

Note: create component that you know are must }

eg: Design Facebook

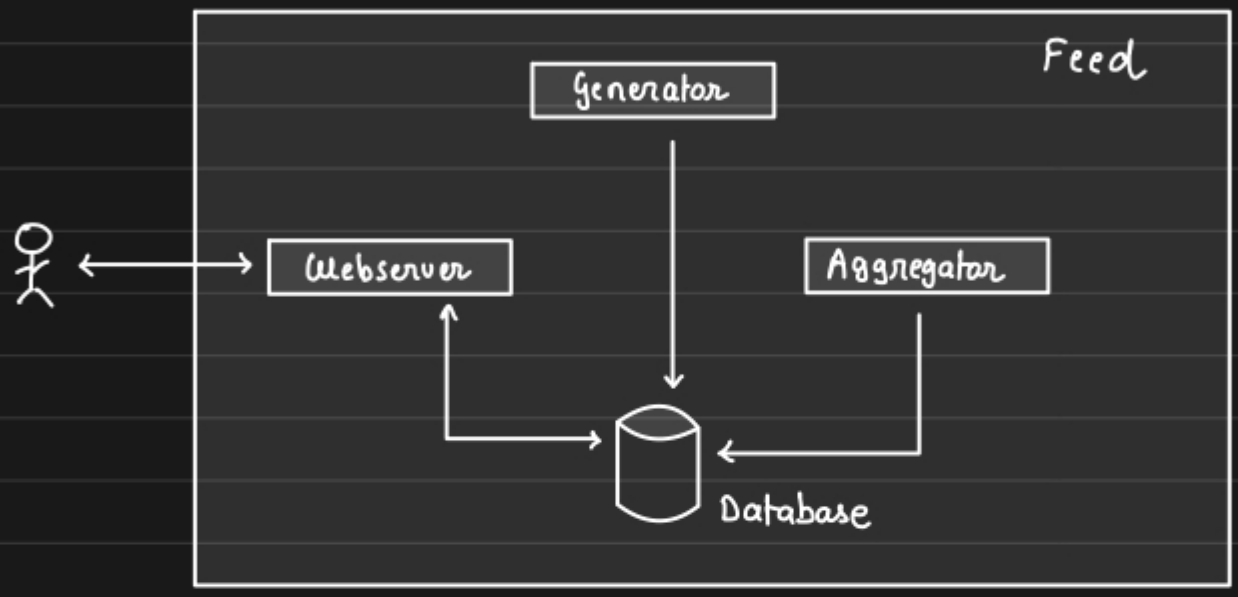
Auth, Notification, Feed, Gamification } components / features

when statement is too big



Dissect each component (if required)

eg: Feed might have - generator, aggregator, webserver



For each sub-component look into

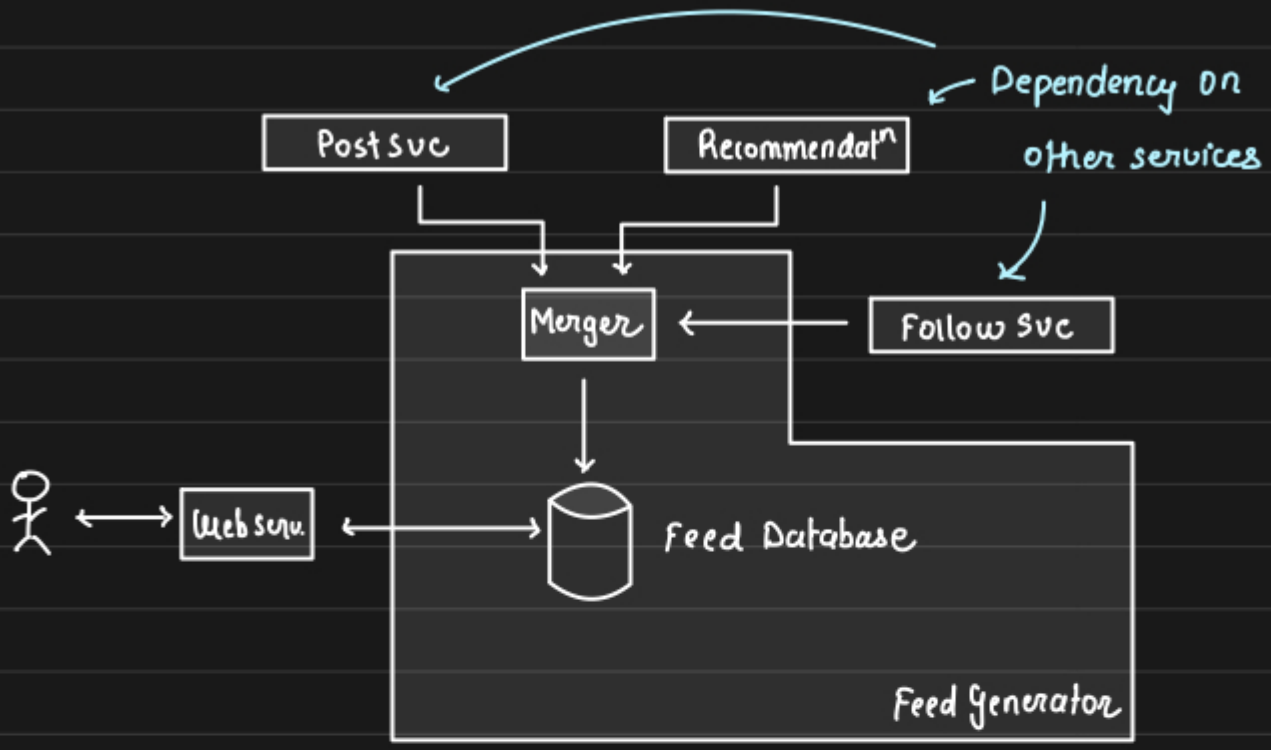
1. Database and Caching
2. Scaling & Fault Tolerance
3. Async processing (Delegation)
4. Communication

} Repeat for each sub component one by one

Add more sub-components if needed

- understand the scope
- decide how other componets will talk to this new one
- decide on 4 above factors for this new component
- repeat

ARPIT BHAYANI


Dependency on other services
Post Svc
Recommendat^n
Merger
Follow Svc
Web Serv.
Feed Database
Feed Generator

#ASLI ENGINEERING

Knowing you have built a good system

SWIPE

BY

ARPIT BHAYANI

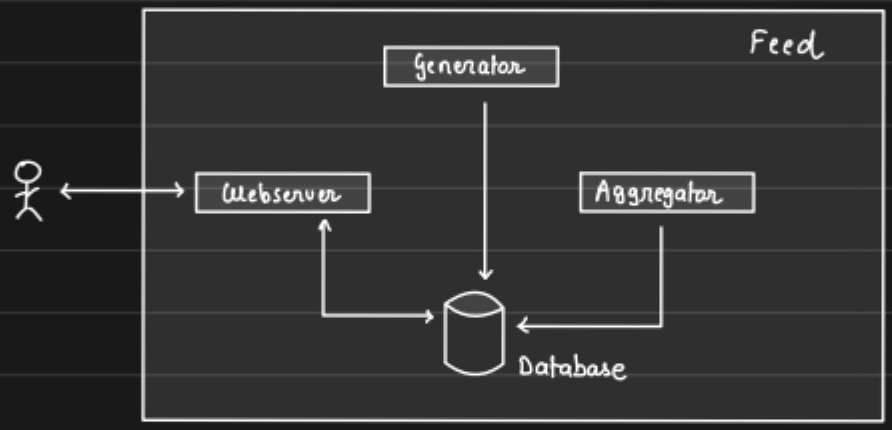
How do you know that you have built a good system?

Every system is "infinitely" buildable

and hence knowing when to stop the evolution is important

Here are some pointers that will help you

1. You broke your system into components



2. Every component has a clear set of responsibilities → exclusive

Feed Webserver → serves feed over HTTP (posts, friends, recommend^n)

Feed Generator → pulls data from multiple services and puts them in DB

↳ candidate feed items

Feed Aggregator → combines candidate items fetch be generator,
filters out redundant, ranks and
creates a final consumable feed

ARPIT BHAYANI

3 For each component, you've slight technical details figured out

1. Database and Caching
2. Scaling & Fault Tolerance
3. Async processing (Delegation)
4. Communication

4. Each component (in isolation) is

Scalable → horizontally scalable

fault tolerant → plan for recovery (mostly data*) in case of a failure
↳ to a stable state

available → component functions even when some component "fails"

This is precisely how cue would tackle every single system.
structured and detailed

ARPIT BHAYANI

#ASLI ENGINEERING

Relational Databases

SWIPE

BY

ARPIT BHAYANI

Relational Databases

Databases are most critical component of any system. They make or break a system.

Data is stored & represented in rows and columns

History of relational databases → Computers, Internet, Blockchain

Everything "revolutionary" starts with Financial Applications!

Computers first did "accounting" → ledgers → Rows & Columns

Databases were developed to support accounting

Hence, key properties were

1. Data consistency
2. Data durability
3. Data integrity
4. Constraints
5. Everything in one place

Because of these reasons, relational databases provides "Transactions"!

A C I D

Atomicity Consistency Isolation Durability

ARPIT BHAYANI

Atomicity All statements within a transaction takes effect or none

eg: start transaction

publish a post and increase total posts count:

```
insert into posts values (....);
update stats set total_posts = total_posts + 1
    where user_id = 100;
commit
```

Consistency

data will never go incorrect, no matter what

constraints, cascades, triggers

eg: Foreign key checks do not allow you to delete parent if child exists ← * can be tuned

You have the necessary tools to ensure that your data never goes inconsistent

total_posts = total entries in posts table for user!

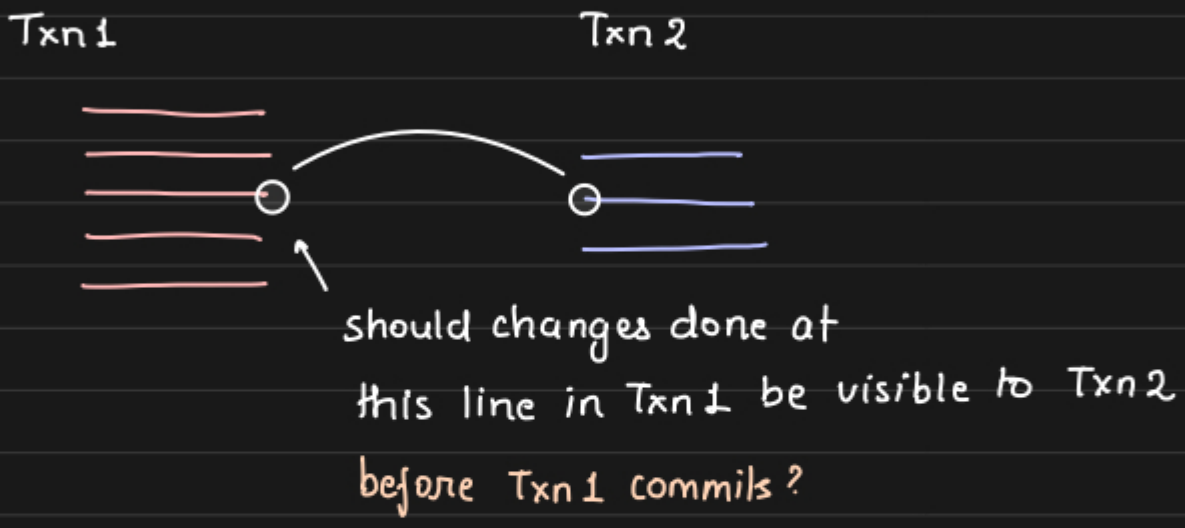
Durability

when transaction commits, the changes outlives outage.

ARPIT BHAYANI

Isolation

When multiple transactions are executing parallely, the isolation level determines how much changes of one transaction are visible to other



Remember

You pick relational databases for relations and acid.

Exercise

1. Setup a sql database (MySQL or PostgreSQL)
2. Create a schema for a social network
   users, posts, profile, photos, following etc. → define relationship
3. Insert data in (users & profile) in one transaction

#ASLI ENGINEERING

Database Isolation Levels

SWIPE

BY

ARPIT BHAYANI

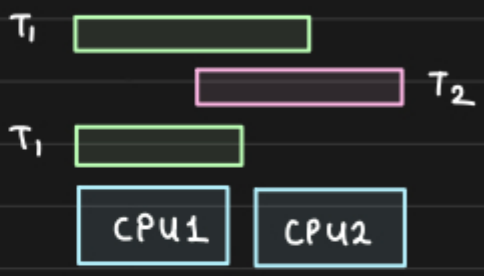
Isolation levels

Relational databases provides ACID guarantees

and I in ACID is 'Isolation' and Isolation levels helps us tune them

Isolation levels dictate how much one transaction knows about the other

We look at each one of them and understand with examples



Repeatable Reads

Consistent Reads within same transaction

even if other transaction committed 1st transaction

would not see the changes (if value already read)

Read Committed

Reads within the same transaction always reads fresh value

con: multiple reads within same transaction are inconsistent

Read Uncommitted

Reads even uncommitted values from other txns

"dirty read"

ARPIT BHAYANI

Serializable

depends on engine

Every read is a locking read

and while one txn read, others will have to wait.

* Storage engines can alter the implementation

so read documentation before you alter

ARPIT BHAYANI

#ASLI ENGINEERING

Database Scaling Techniques

SWIPE

BY

ARPIT BHAYANI

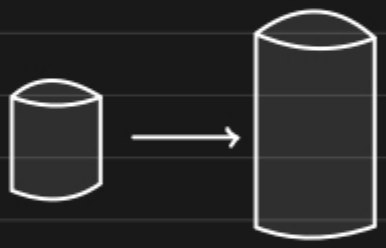
Scaling Databases

Databases are the most important component of any system out there. It makes or breaks any system...

Hence, it is critical to understand how to scale them...

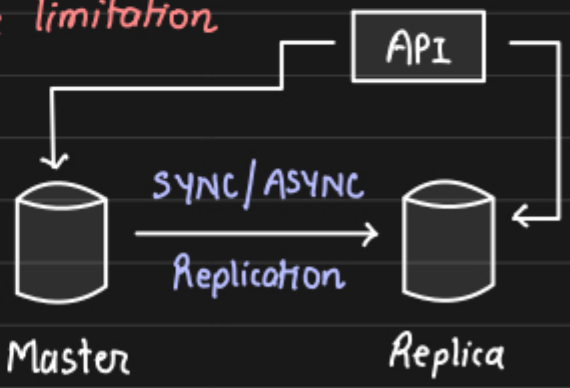
* These techniques are applicable to most databases out there. (read your DB documentation.)
  ↓
  relational and non-relational

Vertical Scaling



- add more CPU, RAM, Disk to the database
- requires downtime during reboot
- gives you ability to handle "scale", more load
- vertical scaling has a physical hardware limitation

Horizontal Scaling : Read Replicas

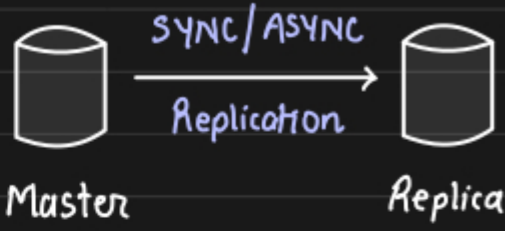


- when read : write = 90:10
- you move reads to other database so that "master" is free to do writes
- API servers should know which DB to connect to get things done

ARPIT BHAYANI

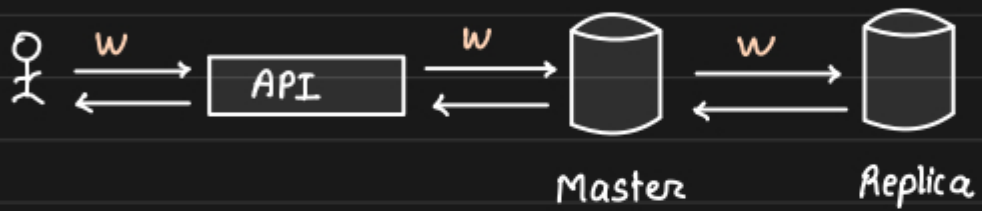
Replication

Changes on one database (Master) needs to be sent to Replica to maintain consistency



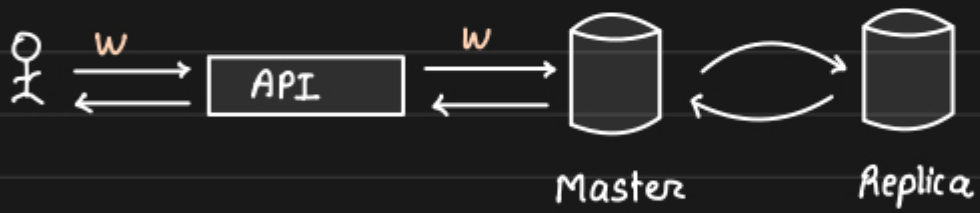
Two modes of replication

1. Synchronous replication



- Strong consistency
- zero replication lag
- slower writes

2. Asynchronous replication



- eventual consistency
- some replication lag
- faster writes

Horizontal Scaling : Sharding

Because one node cannot handle the data/load we split it into multiple exclusive subsets

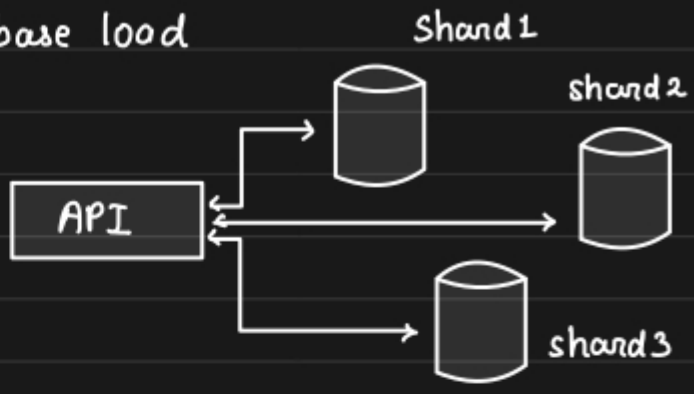
Writes on a particular row/document will go to one particular shard.

ARPIT BHAYANI

This way, we scale our overall database load

Note: shards are independent

no replication b/w them



API server needs to know whom to connect to, to get things done.

Note: some databases has a proxy that takes care of routing

Each shard can have its own replica (if needed)

Exercise

1. configure one MySQL as replica of another
2. put some data and see the replication happening
3. write a small API service that has two connection obj
   - ↳ one master and one replica
4. depending on the request, make call to either master or replica
5. implement sharding by spinning Two DB
   - ↳ one handling keys (a to m)
   - ↳ second handling keys (n to z)
6. write API service that routes request to one of them depending on key

ARPIT BHAYANI

#ASLI ENGINEERING

Sharding and Partitioning

SWIPE

BY

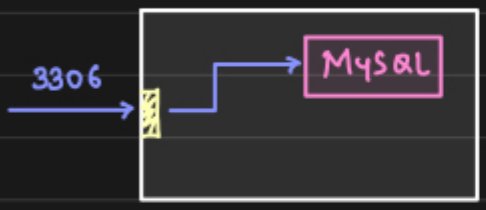
ARPIT BHAYANI

Sharding and Partitioning

Sharding : method of distributing data across multiple machines

Partitioning: splitting a subset of data within the same instance

How a database is scaled ?



Virtual Server
Ec2 machine

A database server is just a database process (mysqld, mongod) running on an Ec2 machine.

And we represent this as

You put your database in production, serving real traffic

You are getting more users, that your DB is unable to manage

you scale up your DB ... give it more CPU, RAM and DISK

Bulkier server + Read Replica

ARPIT BHAYANI

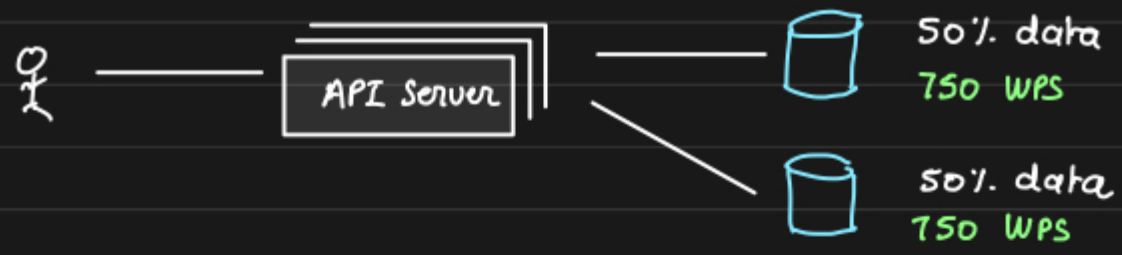
Your product went viral and your bulky database is unable to handle the load, so you scale up again

But, after a certain stage you know you would not be able to scale "up" your DB because **Vertical scaling has limit**

So, you will have to resort to **Horizontal Scaling**

Say, one DB server was handling 1000 WPS and we cannot scale up beyond that but we are getting 1500 WPS, we scale horizontally and split the data

By adding one more database server, we reduced the load to 750 WPS on each node and thus handled higher throughput

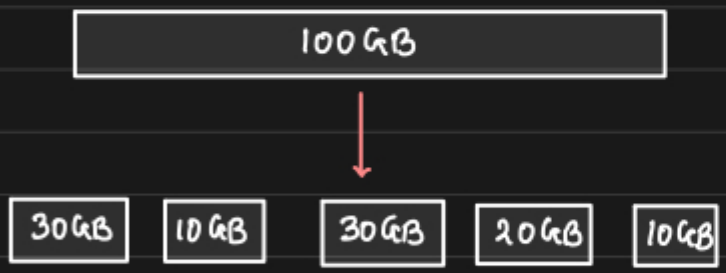


Each database server is thus a shard and we say that the data is partitioned

Overall, a database is sharded while the data is partitioned

↗ Oversimplification, most people use the terms interchangably

↑ split across

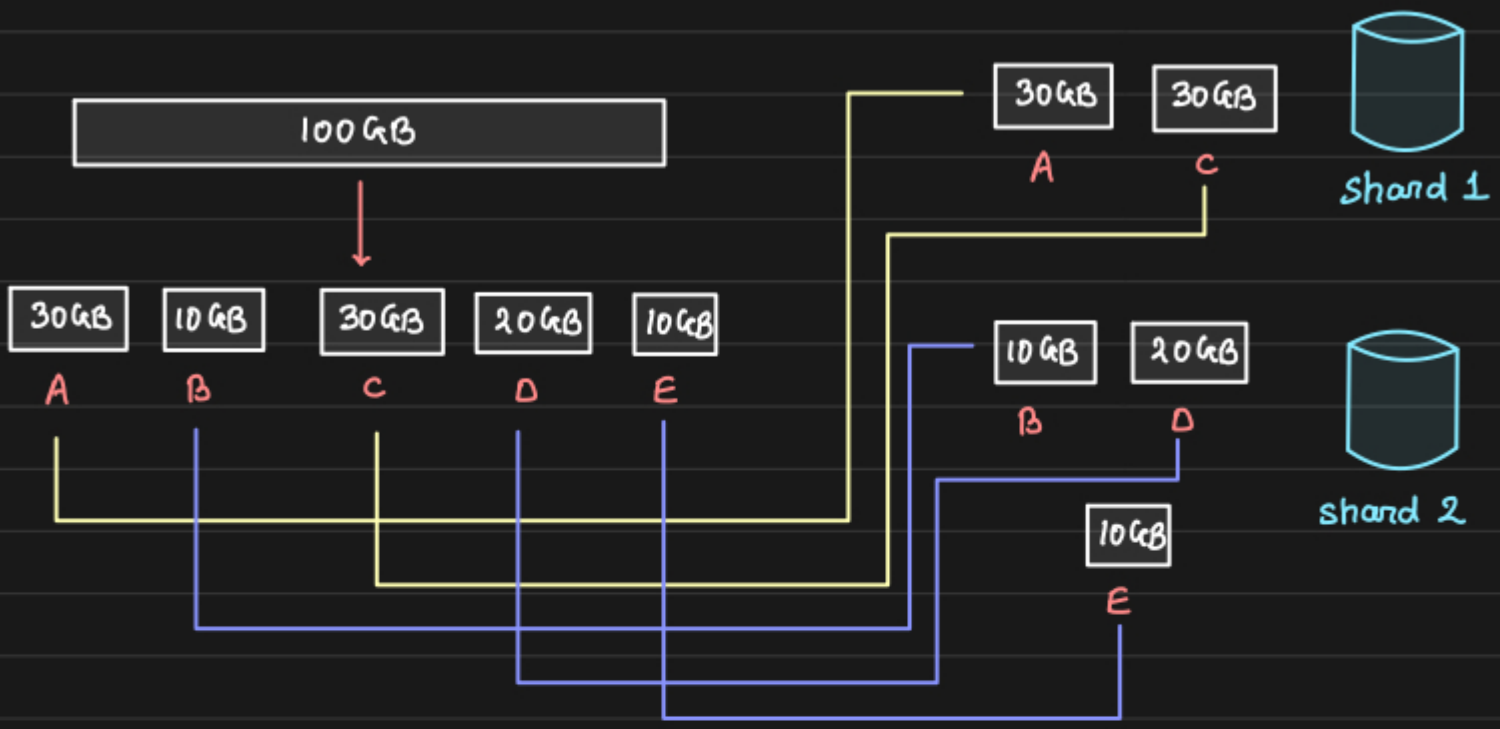


You partitioned the 100 GB of total data into 5 mutually exclusive partitions.

Each of these partitions can either live on one database server or a couple of them can share one server.

And this depends on the # shards you have

ARPIT BHAYANI



5 partitions of our 100 GB dataset is distributed across 2 shards

How to partition the data?

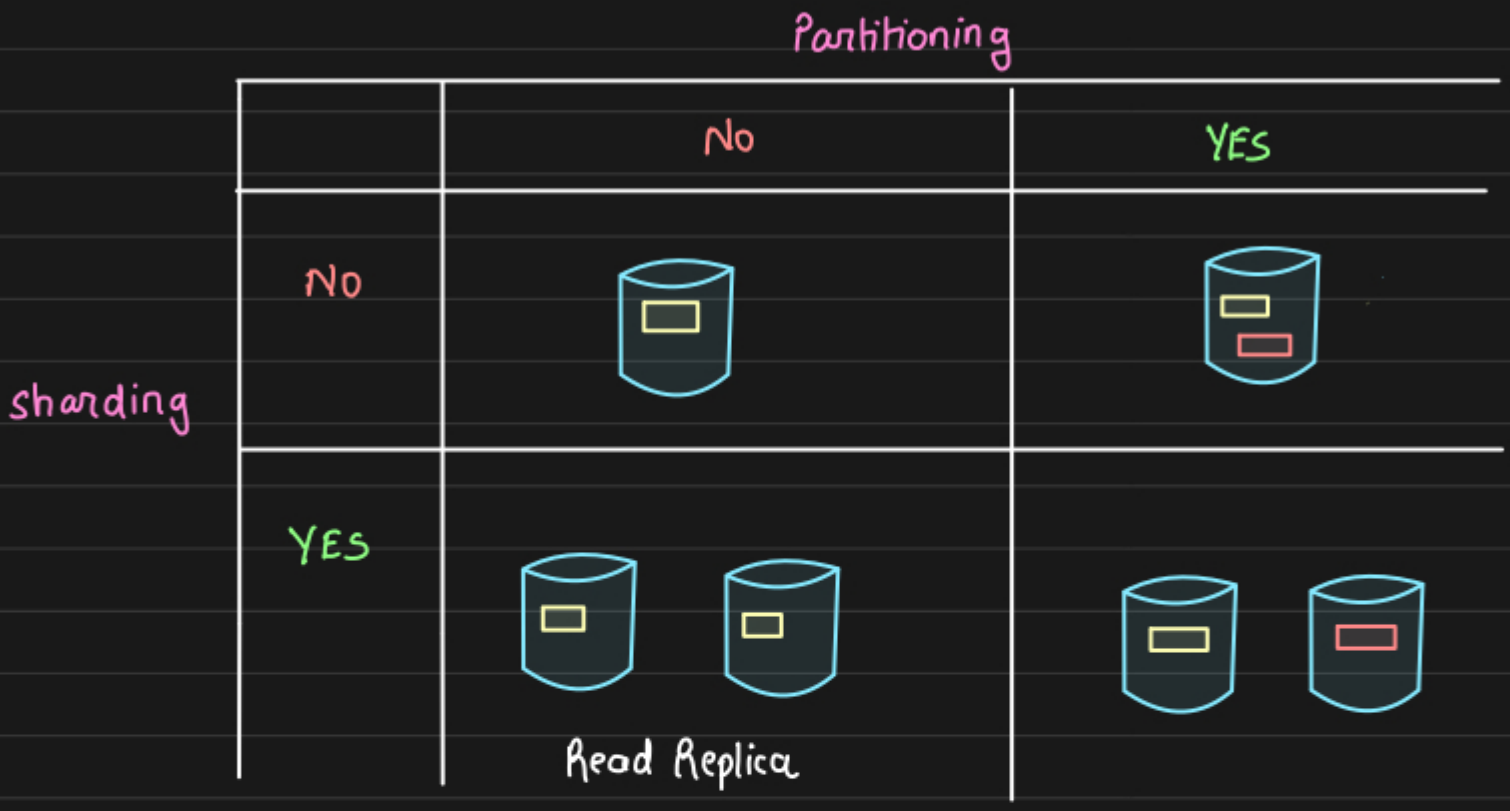
There are two categories of partitioning

1. Horizontal Partitioning
2. Vertical Partitioning

When we "split" the 100 GB data, we could have used either of the ways but deciding which one to pick depends on load, usecase, and access pattern.

ARPIT BHAYANI

Sharding and Partitioning



Advantages of sharding

- Handle large Reads and Writes
- Increase overall storage capacity
- Higher availability

Disadvantages of sharding

- operationally complex
- cross-shard queries expensive

ARPIT BHAYANI

#ASLI ENGINEERING

Non Relational Databases

SWIPE

BY

ARPIT BHAYANI

Non-Relational Databases

It is a very broad generalization of databases that are non-relational (MySQL, Postgresql, etc)

But this does not mean all non-relational databases are similar

What makes Non-relational databases interesting?

Most non-relational databases = shard out-of-the box !!

↑ Horizontal Scalability

We talk about 3 most important types of NoSQL databases

Document DBs MongoDB, Elasticsearch

- Mostly JSON based
- Supports complex queries → almost like relational (sql) databases
- Partial updates to documents possible
- closest to relational database

```
{
  "user_id": ——,
  "total_posts": 270
}
```

can do total_posts += 1 without re-writing the entire document

- in-app notification service, catalog service

ARPIT BHAYANI

Key Value Stores (Redis, DynamoDB, Aerospike)

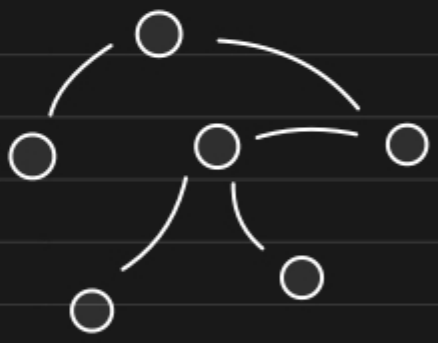
- Extremely simple databases
- Limited functionalities (GET, PUT, DEL)
- Meant for key-based access pattern
- Does not support complex queries (aggregations)
- Can be heavily sharded and partitioned
- Use case: profile data, order data, auth data, messages, etc

GET (K)
PUT (K, V)
DEL (K)

most of the usecases ↑

* You can use relational databases and document DBs as KV stores

Graph Databases (Neo4j, Neptune, DGraph)



* What if our graph data structure had a database
- it stores data that are represented as nodes, edges, and relations

eg: A —Follows→ B Arpit —BOUGHT→ iPAD

- Great for running complex graph algorithms
- Powerful to model Social Networks, Recommendations & Fraud Detection

ARPIT BHAYANI

Exercise

On your local machine, spin up MongoDB, Redis and Neo4j and play around with them

#ASLI ENGINEERING

Picking the right database

SWIPE

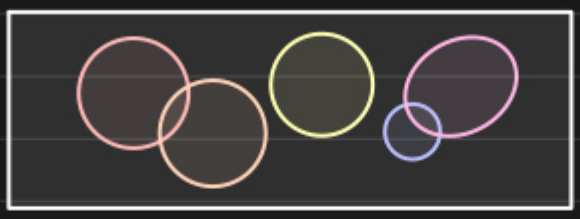
BY

ARPIT BHAYANI

Picking the right database

It is not a fight, so no need to pick a side

a database is designed to solve a particular problem really well.



→ Each kind of database picks a segment with a slight overlap

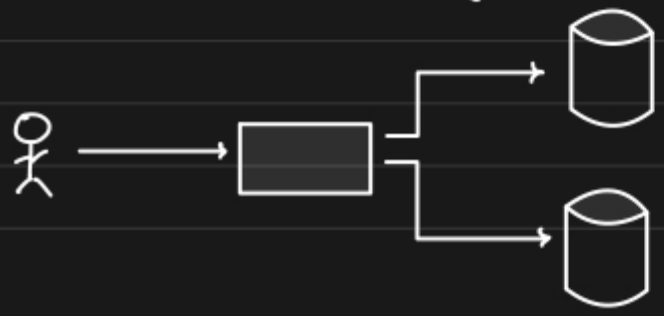
Common Misconception: Picking Non-relational DB because relational Databases do not scale

Why non-relational DBs scale

- There are no relations and constraints
- Data is modelled to be sharded
  ↳ split across multiple nodes

if we relax the above ↑ on relational DB, we can scale it too!!

- do not use Foreign Key check
- do not use cross shard transaction
- do manual sharding



ARPIT BHAYANI

Does this mean, no DB is different ?

No!! every single database has some peculiar properties and guarantees

and if you need those, you pick that DB

How does this help in designing system ?

while designing any system,

do not jump to a particular DB right away

1. understand what data you are storing
2. understand how much of data you will be storing
3. understand how you will be accessing the data
4. what kind of queries you will be firing
5. any special feature you expect

eg: Expiration

How to pick the right DB? [Not exhaustive, but you'll get the idea]

if data can fit on a single node

You need strong consistency? & data correctness is critical

↳ go for relational database

ARPIT BHAYANI

You need complex queries, aggregations

↳ go for relational database

Your access is KV based but need it to be really fast

↳ go for Redis

You need advanced data structures & algorithms

↳ go for Redis

if data cannot fit on one node

You have expertise in SQL & can do manual sharding

↳ drop constraints & go for relational DB

You have simple KV based access

↳ go for KV store like DDB, MongoDB etc

if you require sophisticated graph algorithms

↳ go for graph DB like Neo4j

if you have nothing specific, but want to future proof

↳ go for document DB like MongoDB

ARPIT BHAYANI

#ASLI ENGINEERING

Understand caching

SWIPE

BY

ARPIT BHAYANI

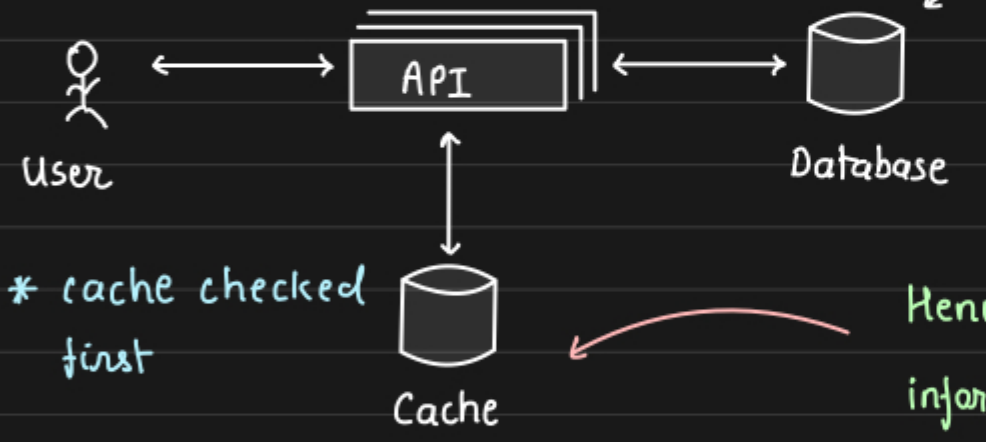
Caching

Caches are anything that helps you avoid an expensive network I/O, disk I/O, or computation

1. API call to get profile information
2. Reading a specific line from a file
3. doing multiple table joins

* Performance Improvement

Store frequently accessed data in a tempory storage location



Fetching from DB is expensive
(eg: expensive db query, expensive disk I/O)

* cache checked first

Hence, we "cache" the information is some other place (so that we do not have to go to main database)

Caches are faster and expensive

* hence we do not cache all the data. (just a subset of it which is most likely to be accessed)

Caches that we typically use are

Redis, Memcached

ARPIT BHAYANI

Note: Caches are not restricked to RAM based storage

Any storage, that is 'nearer' and helps you avoid something expensive is a cache for you!

In their simplest form,

Caches are just glorified hash tables

Some examples

1. Google News: Most recent news articles are more likely to be accessed, hence served from cache
2. Auth Tokens: Authentication are cached in "cache" to avoid load on database
   [tokens are checked on every request]
3. Live Stream: Last 10 min of Live Stream is cached on CDN as it will be accessed the most

Exercise
1. Setup Redis locally
2. put and get some data
3. measure time taken
4. compare it with database

#ASLI ENGINEERING

Populating and scaling caches

SWIPE

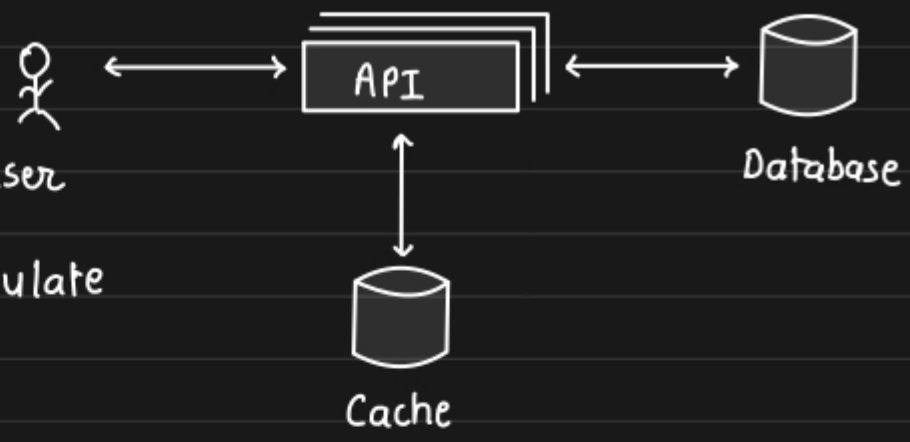
BY

ARPIT BHAYANI

Populating the cache

Cache between the API server and the database.

There are two ways to populate the cache



Lazy population (most popular)

Read first go to the cache

if data exist, return data

else

go to database / do heavy operation } lazy population

persist in the cache

return data

* Whenever we set something in cache, we set an expiry

Example: Caching Blogs (multiple joins)

Fetching a blog from DB is expensive

Hence, when someone accesses it we fetch from DB and cache it on Redis

Subsequent requests are served from cache

ARPIT BHAYANI

Eager population

1. Writes go to both database and cache in the same request call

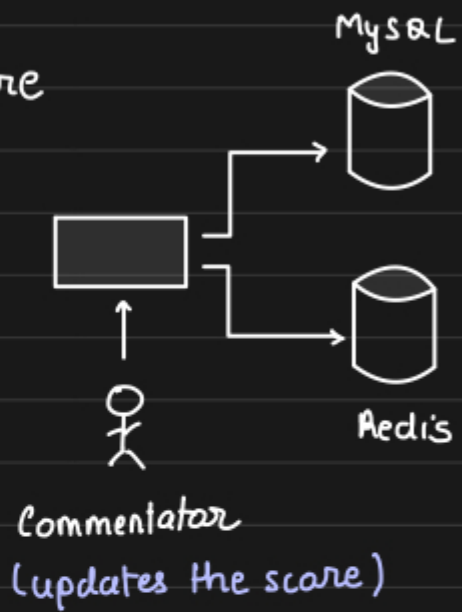
   example: live cricket score

   Thousands of people are watching cricket score

   You will be serving it from cache

   so, why not update cache & db at once

   and save the cache miss.



2. Proactively push data to cache

   because you anticipate the need

   Example: when a celebrity tweets | post something

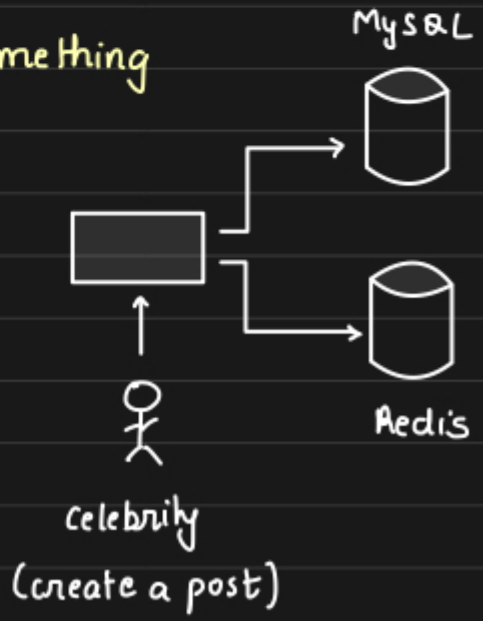
   When account with 100,000 followers

   post something, proactively push it

   to the cache

   We will anyway need it and we

   save a cache miss.



ARPIT BHAYANI

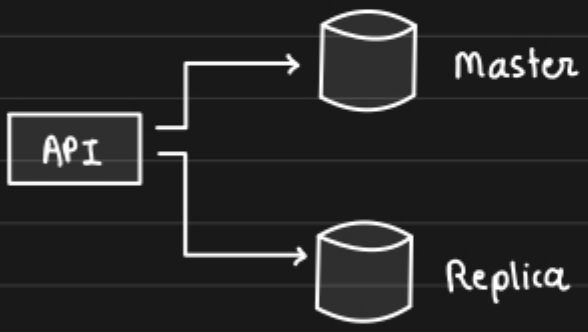
Scaling Cache

Cache is just like a database, hence scaling technique for a cache like Redis is similar to a regular database

Vertical scaling

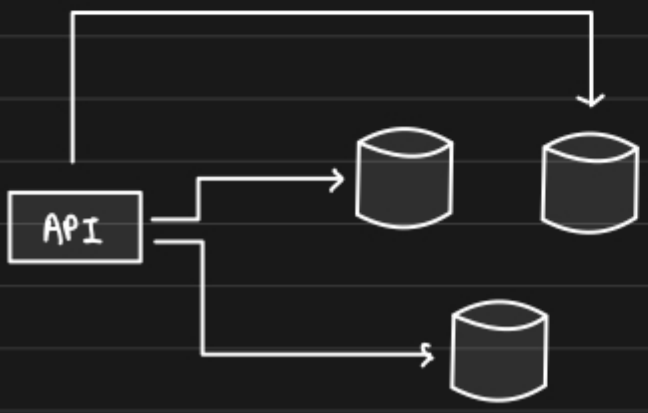
Make your cache bigger to handle more data / load

Horizontal Scaling - Replica (scaling reads)



Same data replicated across multiple nodes so that reads could scale

Horizontal Scaling - Sharding (scaling writes)



Data partitioned across multiple shards, so that writes could scale

* each shard can have a replica

* shards are mutually exclusive

#ASLI ENGINEERING

Caching at different levels

SWIPE

BY

ARPIT BHAYANI

Caching at different levels

Most common cache we saw was Redis

but that is not the only type of cache out there

or not the only place that can be used as a cache

Literally every piece/component in your infrastructure can cache something for you. But should you?

↓

depends on the guarantees ← stale data and invalidation

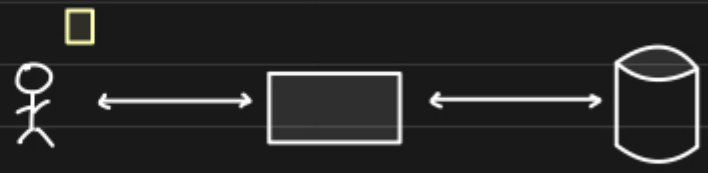
also, too much caching is bad!

let's take a look at different places where we can cache

Client Side Caching

Storing frequently accessed data on client side

eg: Browser, Mobile Devices, etc.



- cache near constant data
  (eg: images, js files, user information, etc)
- it should be okay serving ↑ cached info (stale)
- invalidation by time (expiry)

Massive performance boost, as we need not make any request to backend

Live Streaming
Serving Images, Videos, Audio, Bundles, etc

Content Delivery Networks (CDN)

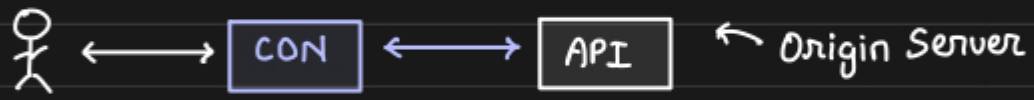
used for caching

CDNs are a set of servers distributed across the world.

Request from a user, goes to the nearest CDN server and hence user gets very quick response

US folks getting images from US servers is faster than fetching it from India.

* CDN does lazy cache population



User's request comes to CDN → closest server

CDN server checks if it has the data

if yes, return the data

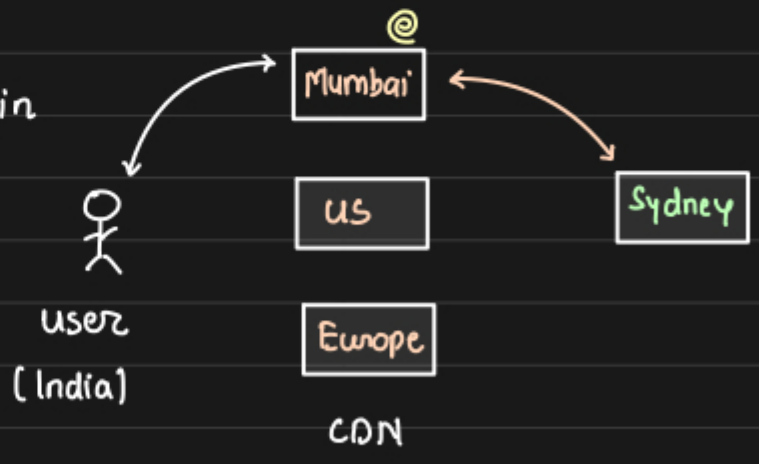
else,

CDN makes the same request to origin

gets the response

caches the response

return the data



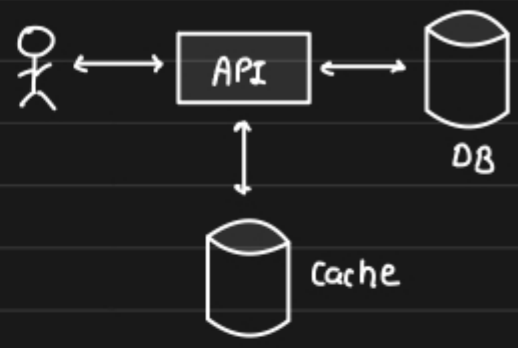
Like any other cache, when you put data on CDN you set an expiry to it (post which CDN deletes data)

ARPIT BHAYANI

Remote cache (Redis)

Expensive & store data in main memory

Remote cache is centralized cache that we most commonly use (Redis). Multiple API servers use it to store frequently accessed data.



* Every key stored should have an expiration (memory leak)
* Size of cache is relatively very small as compared to a database

Database caching

Instead of computing total posts by users everytime we store `total_posts` as column and update it once a while. (saves an expensive DB computation)

```
Select count(*) from posts
    where user_id = 123
```

} expensive query

users

id	name	....	total_posts
123	Arpit		77

Everytime a post is published, we also update the `users` table and do

total_posts = total_posts + 1

Note: There are other places like load Balancer where we can cache

Note: We can cache some data at every single component in the system but should we do it?

Not necessarily (it is very use-case specific and subject to tolerance level of staleness of the served data)

* Just because you can, does not mean you should

Exercise

1. create an account on Akomai / Cloudflare
2. Configure a simple CDN and understand how it is used
3. cache a simple "image" on CDN and access it from CDN using CDN URL

ARPIT BHAYANI

#ASLI ENGINEERING

Message Queues

SWIPE

BY

ARPIT BHAYANI

Asynchronous Processing

user sent the request and you immediately handle it is synchronous

1. loading insta feed is synchronous
2. login on website is synchronous
3. payments are synchronous

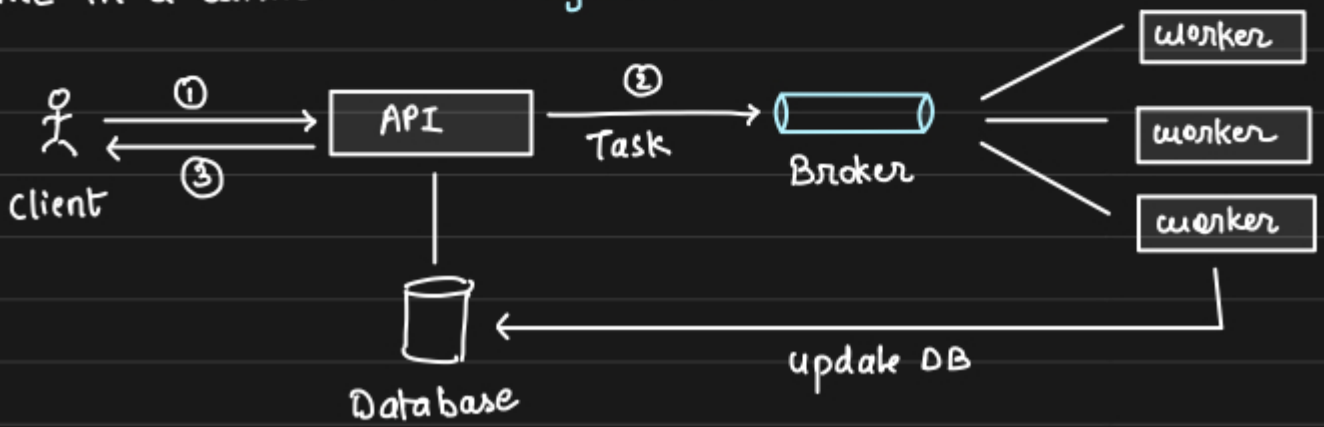
Most interactions on web are Synchronous

some things that should not be synchronous

eg: spinning up a virtual machine

spinning up VM takes minutes, and user will not wait on the same page, waiting for the response

Instead, he/she would love to move around and keep checking status once in a while This is asynchronous



ARPIT BHAYANI

Message Queues

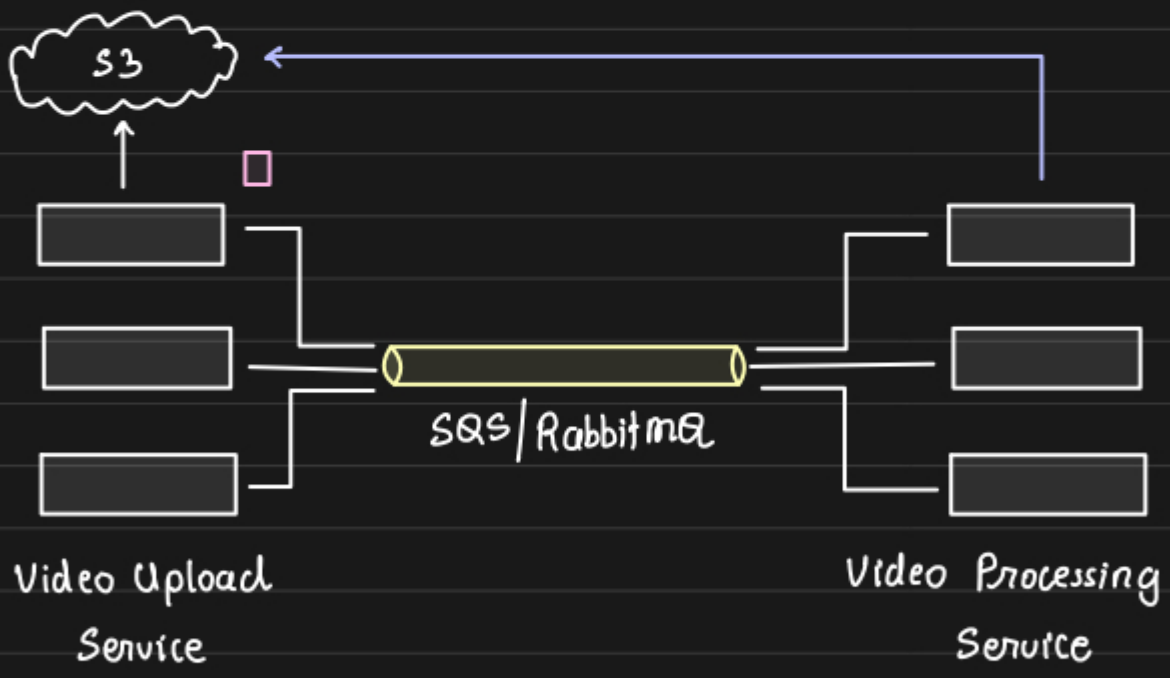
Brokers help two services/applications communicate through messages.

We use message brokers when we want to do something asynchronously.

1. Long-running task
2. Trigger dependent tasks across machines

Example: Video processing

Once the video is uploaded we need to convert it to 360p, 480p, 720p



Message Queues are also called Message Brokers

ARPIT BHAYANI

Features of message brokers

1. Brokers help us connect different sub-systems

2. Brokers act as a buffer for the messages

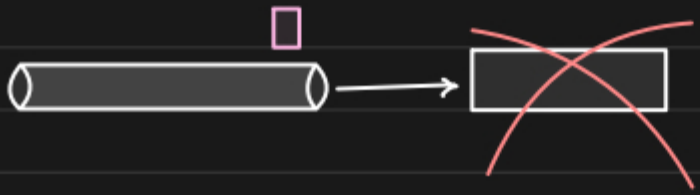
   i.e. consumers can consume at their own pace

   eg: Notification system

   ↳ no synchronous load on connected systems

3. Brokers can retain messages for 'n' days ← depends on broker you use
4. Brokers can re-queue the message if not deleted

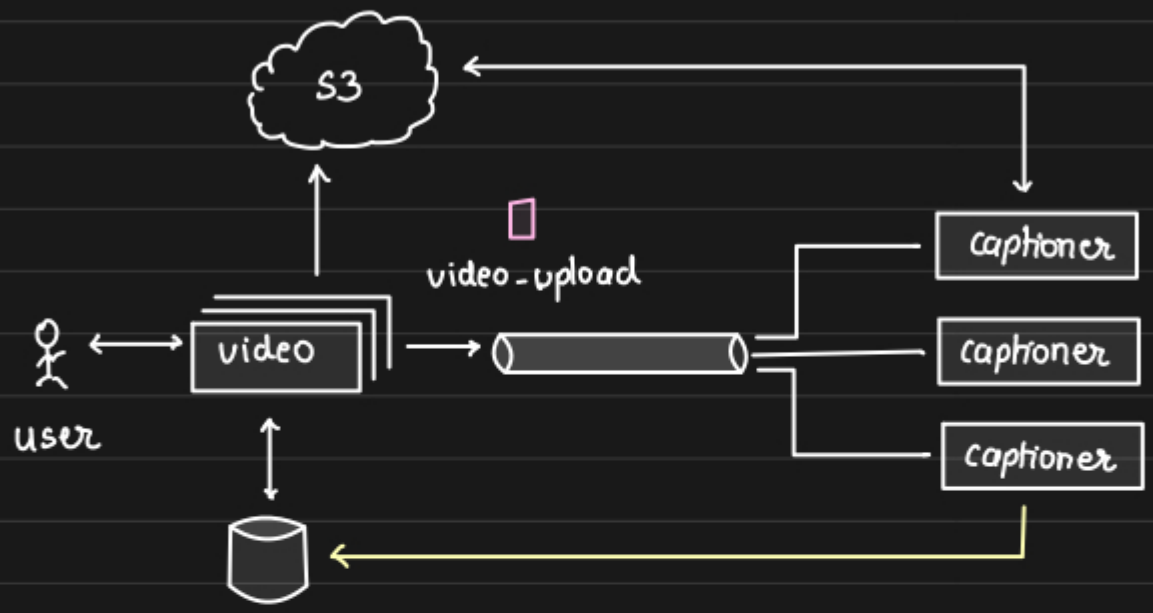
   eg. consumer read the message but before it could delete it, it crashed



ARPIT BHAYANI

Typical flow while using a message queue

Example: Auto-subtitle (auto-captioning)



1. user uploads video to S3 through video service
2. video service puts a message (after upload completes) to broker
3. and returns response to the user

   user sees upload complete

4. message is asynchronously read by 'captioner' service
5. captioner downloads the video
6. captioner generates captions and updates in the DB
7. user now sees caption button enabled

ARPIT BHAYANI

Exercise

1. setup RabbitMQ locally
2. write some code to push and read messages
3. go through the documentation to understand its features

ARPIT BHAYANI

#ASLI ENGINEERING

Message Streams and Kafka

SWIPE

BY

ARPIT BHAYANI

Message Streams

Message streams are similar to message queues ... with a few differences

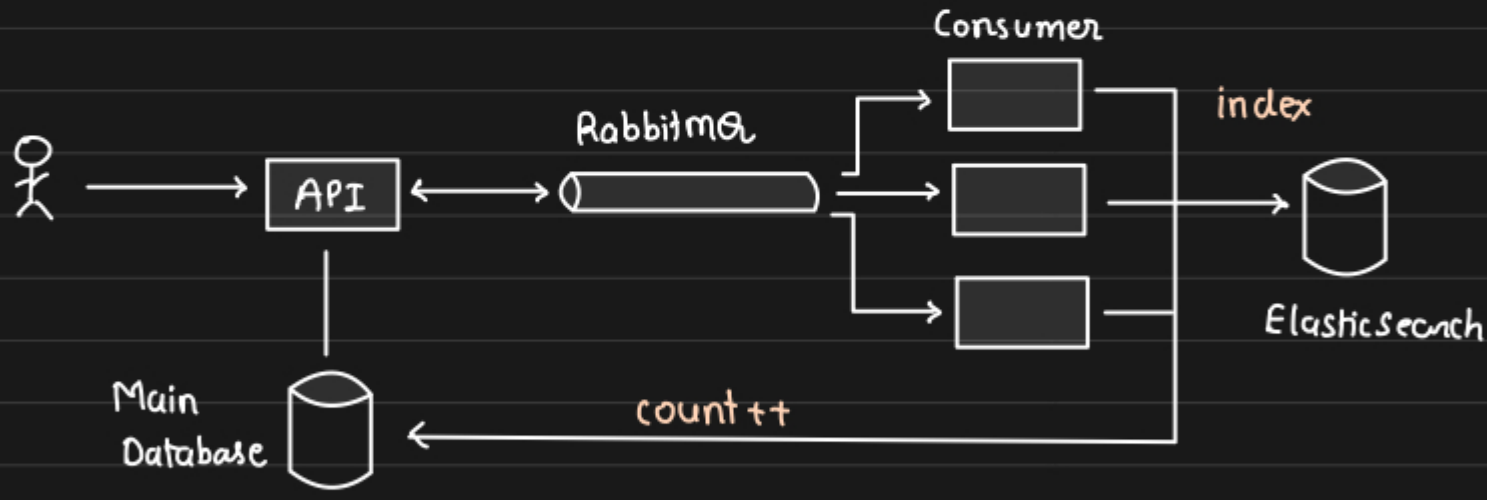
To understand, let's take an example

Say, we are building Medium.com

and upon every blog published, we

- need to index it in search engine (Elastic Search)
- need to do count ++ for user's total blogs

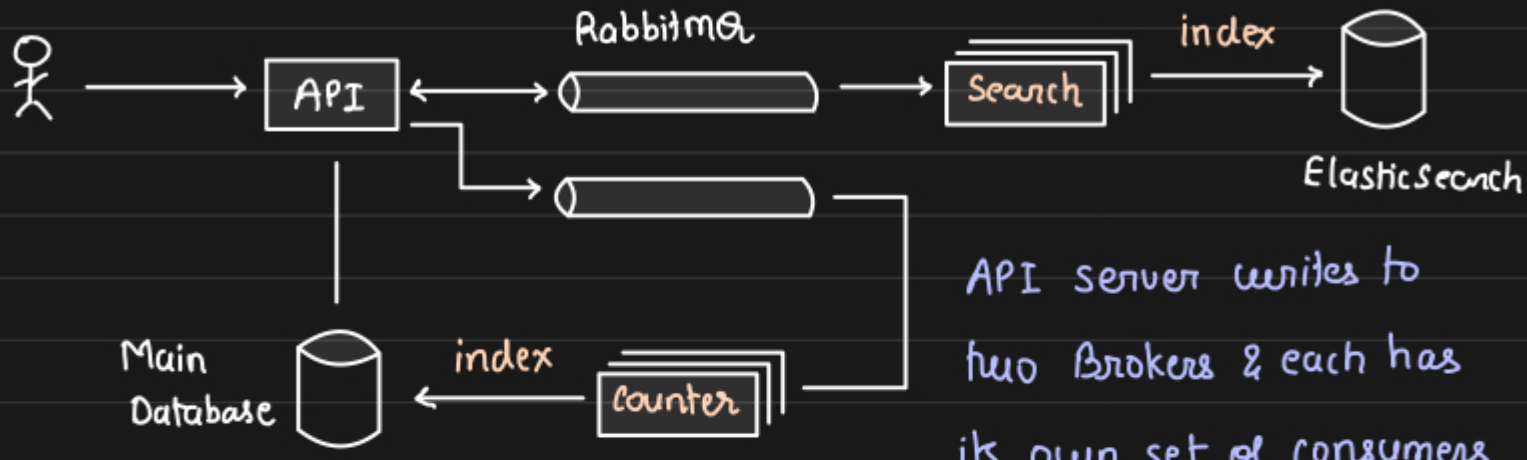
Approach 1: We use one message broker and add logic in consumer



Consumers are doing two things: count++ and index

Issue: what if write to one succeeded but other one failed. count ++ ✓ but not there in search or there in search but count does not match

Approach 2: Two brokers and two set of consumers



API server writes to two Brokers & each has its own set of consumers

This still does not solve the problem!

When API server writes to two RabbitMQ, one of them fails. we end up in the same spot.

Hence, we want "write to one" and "read by many" semantic

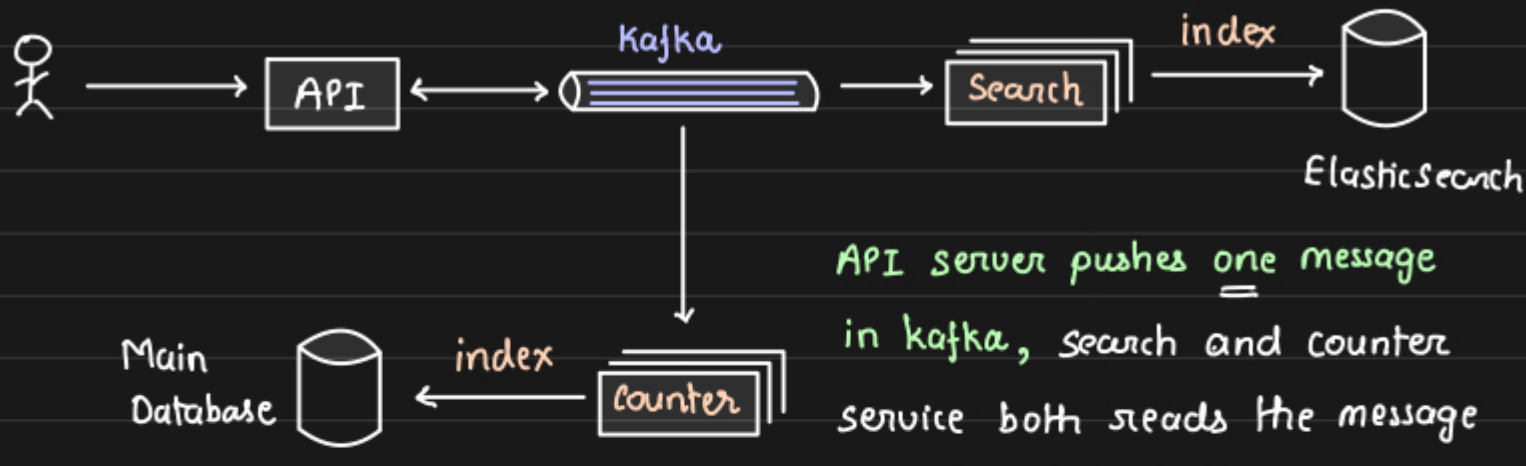
This is where Message Streams come into the picture

↳ Kafka, Kinesis

Message Streams

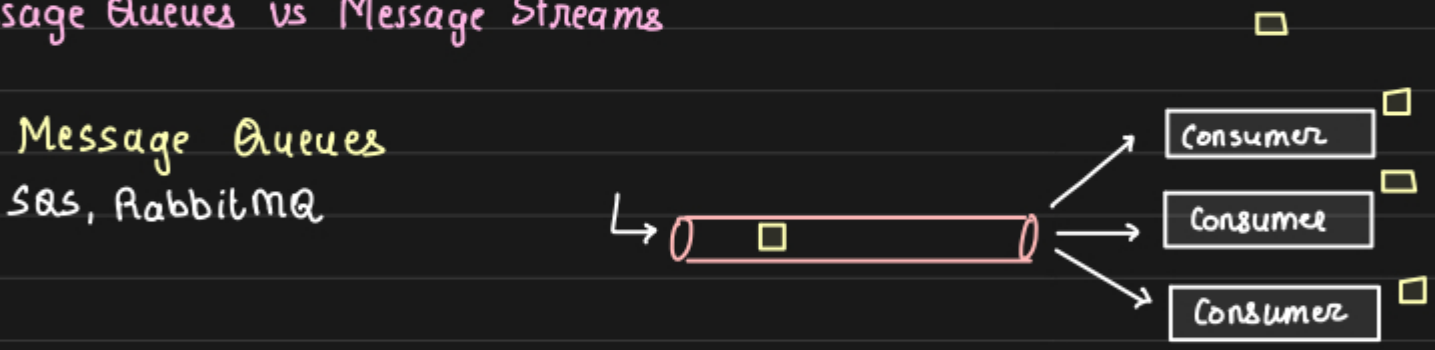
Message streams are similar to message brokers with one change: multiple types of consumers read the same message

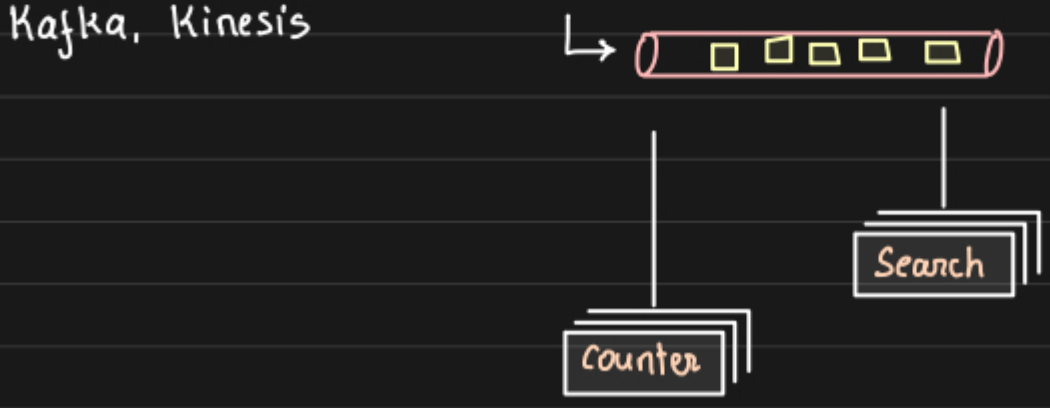
Approach 3 : using streams and multiple types of consumers



API server pushes one message in kafka, search and counter service both reads the message and does their work

Message Queues vs Message Streams





ARPIT BHAYANI

Kafka essentials

Kafka is a message stream that holds the messages

Internally kafka has topic

Every topic has 'n' partitions

Message is sent to a topic

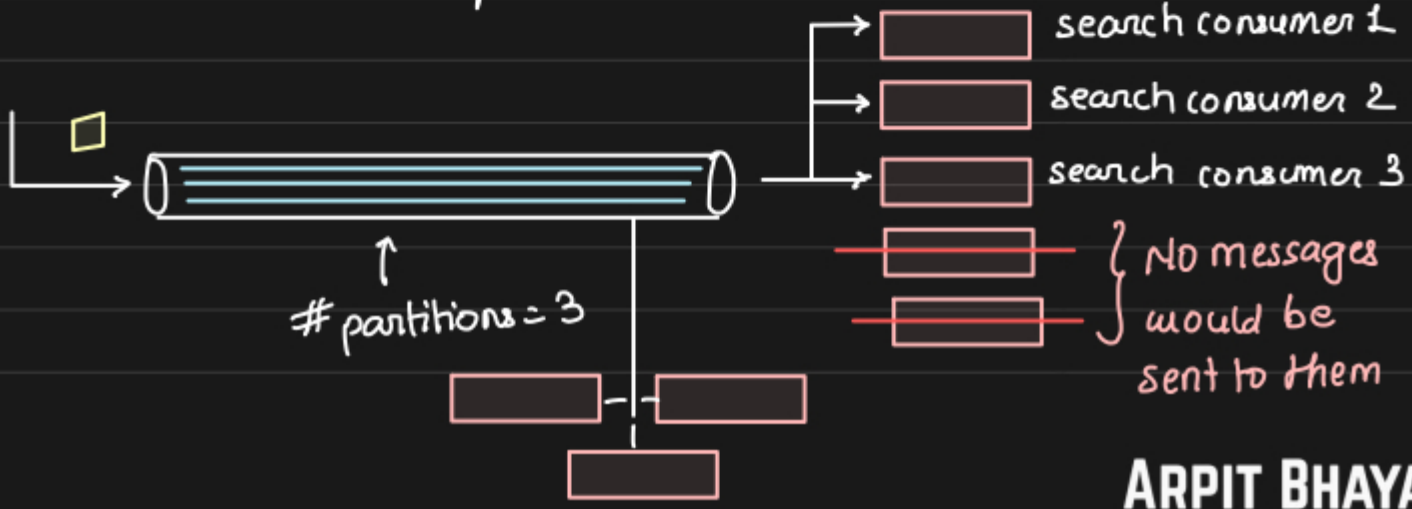
and depending on the configured hash key
it is put into a partition

Within partition, messages are ordered

↳ no ordering guarantee across partitions

Limitation of kafka

consumers = # partitions



ARPIT BHAYANI

Exercise

1. setup Kafka locally
2. write some code to push and read messages
3. go through the documentation to understand its features

ARPIT BHAYANI

#ASLI ENGINEERING

Realtime PubSub

SWIPE

BY

ARPIT BHAYANI

Realtime PubSub

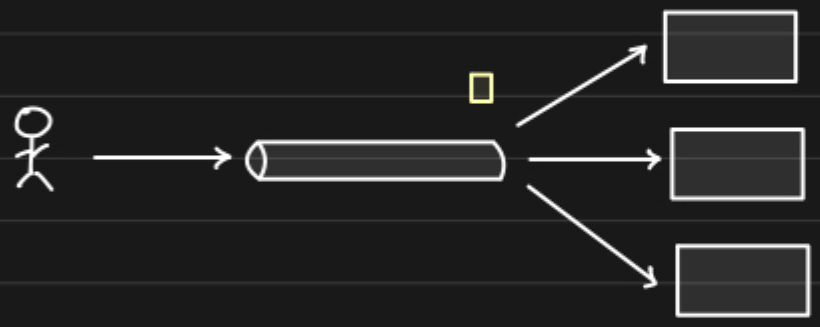
Both message brokers and message streams require consumers to "pull" the messages out.

Advantage: consumers can pull at their own pace
consumers do not get overwhelmed

Disadvantage: consumption lag when high ingestion

What if we want low latency & zero lag? → realtime pubsub

Realtime PubSub makes things reactive instead of continuous polling.



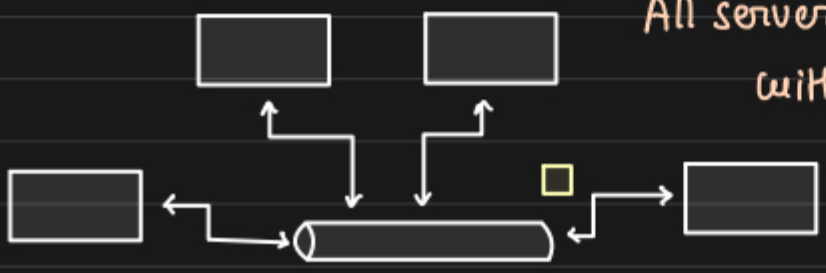
Instead of consumers pulling the message, the message is pushed to them

eg: Redis PubSub

This way we get really fast delivery time
But it can overwhelm the consumers

↳ what if consumers receive messages faster than they could process?

Practical usecase: Message Broadcast, Configuration Push



All server receives updates without polling for data

ARPIT BHAYANI

Exercise

1. setup Redis locally
2. go through Redis PubSub documentation
3. test realtime broadcast
4. test if it persists the message

   (check if new subscriber gets old messages)

#ASLI ENGINEERING

Load Balancers

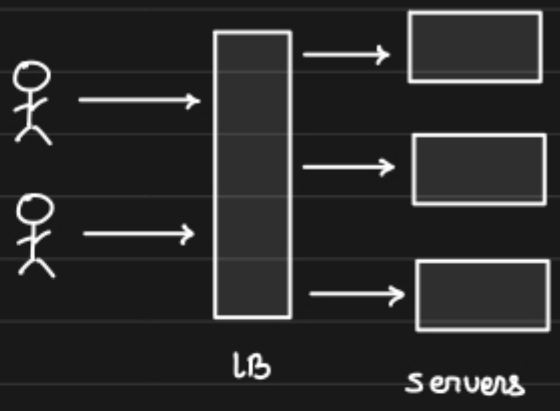
SWIPE

BY

ARPIT BHAYANI

Load Balancers

One of the most important component in Distributed System that makes it easy to scale the load horizontally



* load Balancer is the only point of contact

Every load balancer has either

1. static IP
2. static DNS name

} allowing clients to talk to it (users / servers)

Load balancer hides the #servers that are "behind" it allowing us to add as many servers as possible → Horizontal Scalability without client knowing about it

Request Response Flow

1. client already has ip/domain of load balancer

   eg: auth.example.com

2. client makes API call and it comes to load Balancer

   eg: GET auth.example.com/login

3. load balancer picks one server and makes the same req
4. load balancer gets the response from the server
5. load balancer responds back to the client

ARPIT BHAYANI

Job of the load Balancer is to "balance" the load

↓

it depends on how it picks the server to forward the request → Configurable

Load Balancing Algorithms

Round Robin

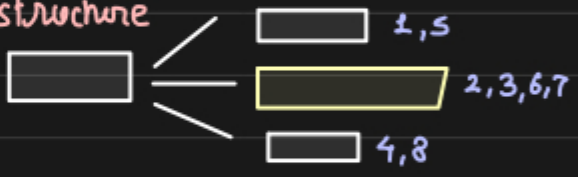
Uniform infrastructure

Distribute the load iteratively

Weighted Round Robin

NON-Uniform infrastructure

Distribute the load iteratively but as per weights



least Connections

Pick the server having the least connections from the load balancer

When response times has a big variance * Analytics

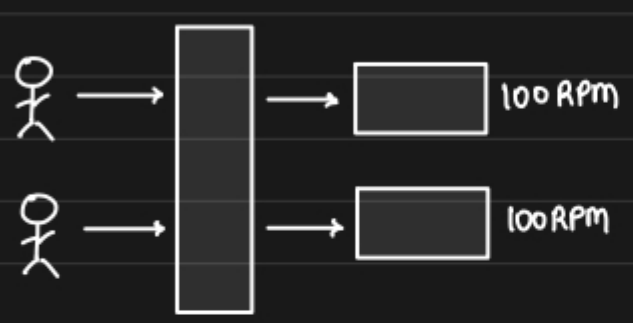
Hash based routing → random enough

Hash of some attribute (ip, user id, url) determines which server to pick

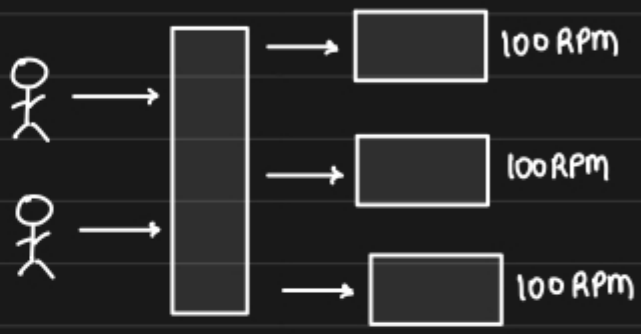
ARPIT BHAYANI

Key advantages of load Balancers

Scalability : With more servers behind load balancer we can now handle more requests

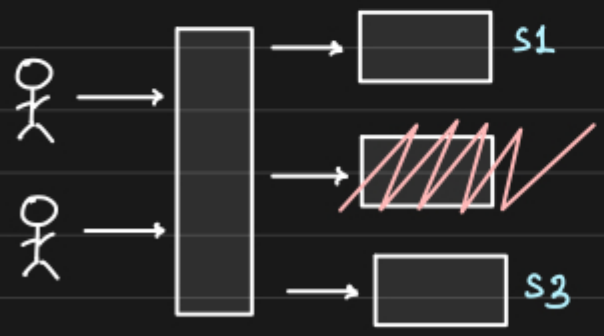


We can handle 200 RPM



We can handle 300 RPM

Availability : Even if one of the servers crash, it does not take down our entire system. Load Balancer will forward request to other Healthy servers. Improving the availability



With S2 down, load balancer will forward any new request to S1 and S3.

Exercise

Go through AWS load Balancer's documentation

ARPIT BHAYANI

#ASLI ENGINEERING

Circuit Breakers

SWIPE

BY

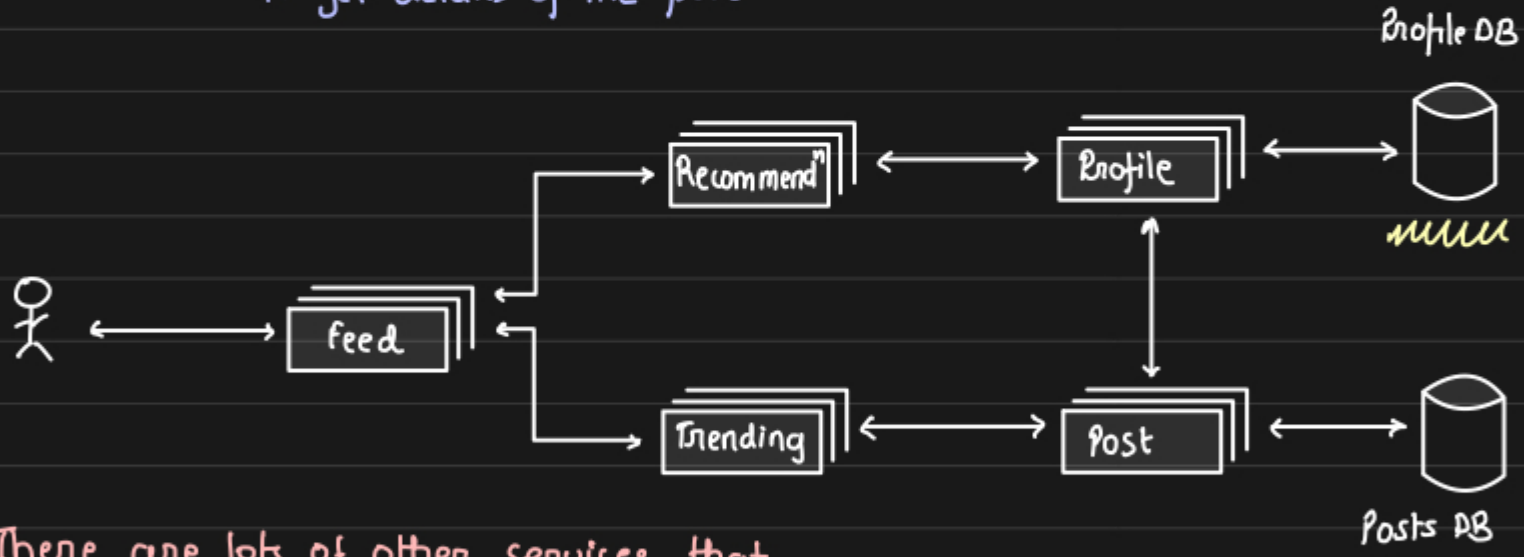
ARPIT BHAYANI

Circuit Breakers

Circuit Breakers prevent CASCADING FAILURES !!

Say, you are building social network that serves feed for a user

1. User's request comes to Feed Service
2. Feed service pulls some info from Recommendation some from Trending
3. Recommendation and Trending both relies on Profile service
   ↳ to get profile detail of user who made the post
4. Recommendation and Trending depends on Post service
   ↳ to get details of the post



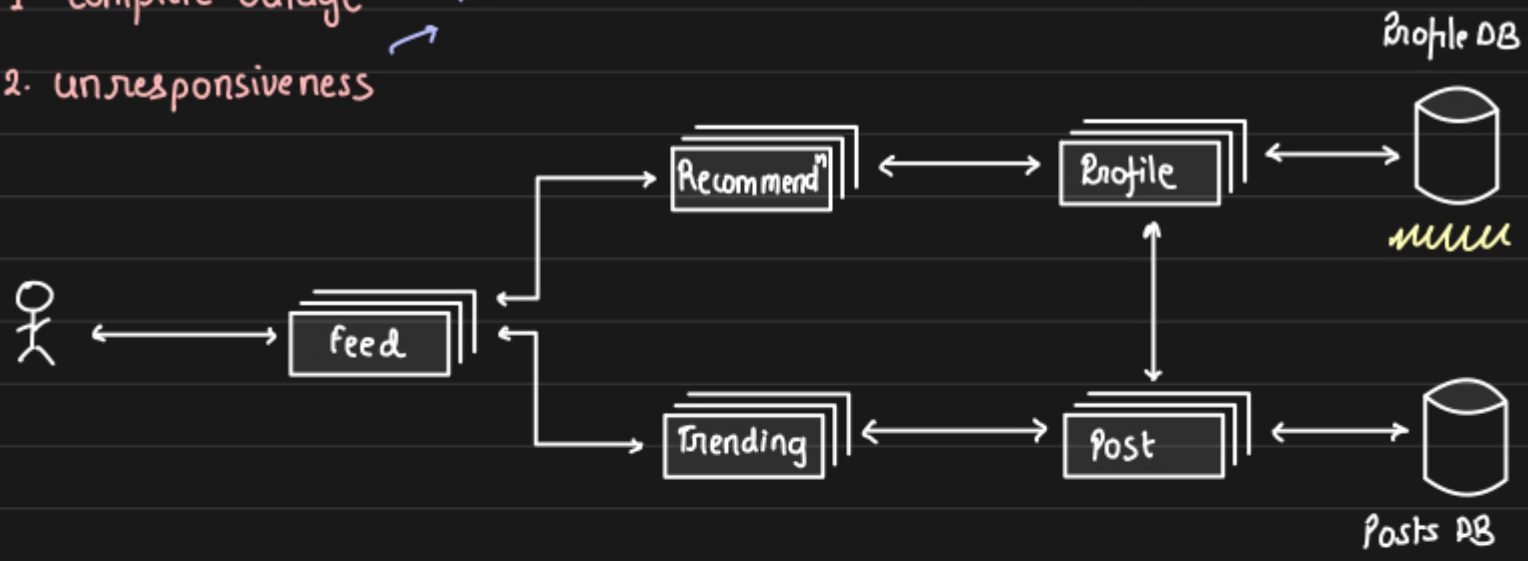
There are lots of other services that depend on profile service.

if profile service DB is overwhelmed !! it slows down and transitively all services dependent on it are affected

"Timeouts" [higher response times]

ARPIT BHAYANI

1. Complete outage → Poor user experience
2. Unresponsiveness



Idea: What if we make call to a service, only if it is healthy?

↑ This is Circuit Breaker

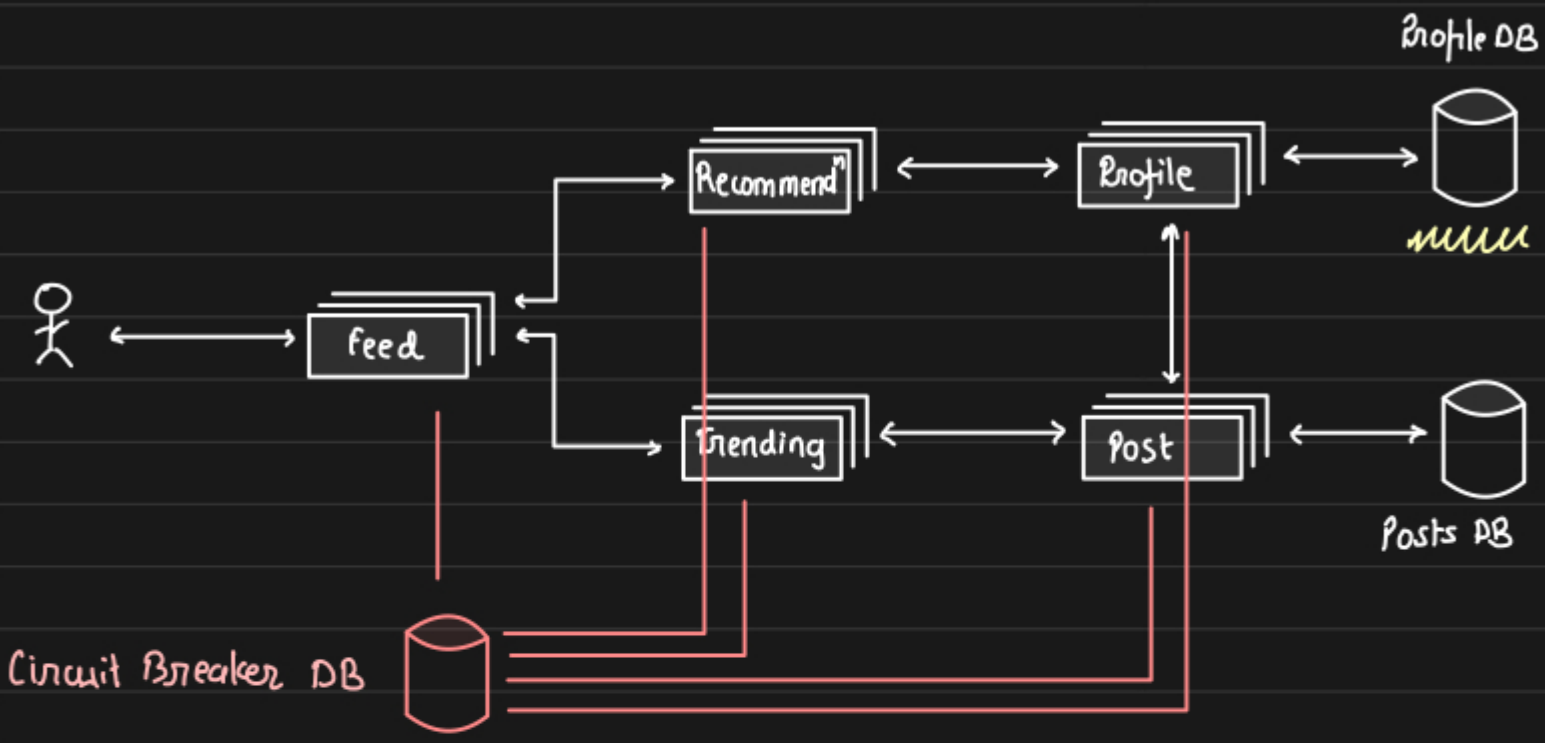
We break the circuit down when we see the failure cascade!

Circuit Breakers prevent entire product from collapsing by preventing cascading failures.

How is it implemented?

- A common database holds the settings for each breaker
- Services, before making calls to others, checks the config
  - ↳ cache the config to avoid checking the DB

ARPIT BHAYANI



In case of the outage,

the circuit is tripped and DB updated

Services will periodically check and stop sending req. to affected services

Exercise

- Implement a simple circuit breaker (CB) * See if you can use Redis Pubsub here?
- write simple service that checks this setting everytime before calling

  eg: post service serves post

  profile service serves profile

  post service calls profile for info

  if circuit is tripped do not make call to profile

ARPIT BHAYANI

#ASLI ENGINEERING

Data Redundancy and Recovery

SWIPE

BY

ARPIT BHAYANI

Data Redundancy & Recovery

API servers are "stateless" but databases are "stateful".

↳ API servers going down is fine because a new one will be spin up almost instantly

API server gets request and it does not matter which one handles it.

↖ Databases going down is catastrophic; almost always an outage!!

Worst: Disk crash leading to loss of data!

A good system always takes care of such catastrophic situation ↑

* The only way to protect ourselves against loss of data is to create multiple copies of it → Data Redundancy

Redundancy can be implemented at row/document level, table level or DB level

Redundant data can be stored on different table, different DB or different region

Backup and Restore

- Daily backup of data (incremental)
- Weekly complete backup
- storing one copy across region ↷ Disaster recovery
- when something goes wrong, just restore the last backup
- almost always the easiest thing to do

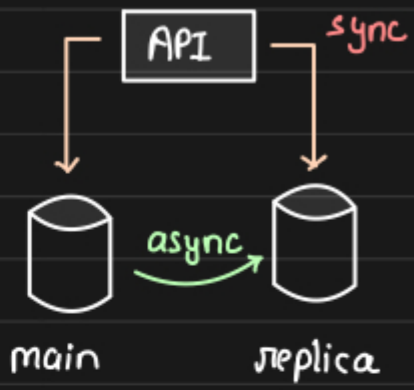
ARPIT BHAYANI

Continuous Redundancy

Setup replica of the database

and writes go to both DB (sync / async)

1. API server writing to both Databases
2. API writes to one and is copied to other asynchronously



* if the main database goes down, replica can take its place (almost instantly)

Note: this replica may just be a stand-by and not serve any production traffic

Exercise

1. setup replication b/w two mysql servers
2. see how you can backup a Mysql db
3. see how you can restore the database

ARPIT BHAYANI

#ASLI ENGINEERING

Leader Election for Auto-Recovery

SWIPE

BY

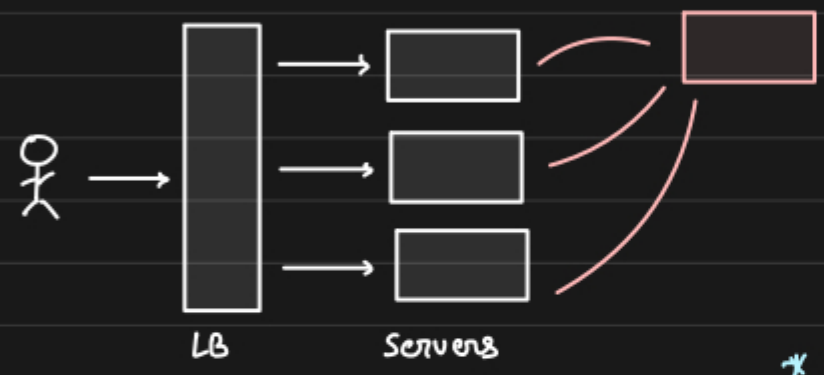
ARPIT BHAYANI

Leader Election for Auto Recovery

Say, we have a bunch of servers, serving HTTP requests

When one of them goes down, we have to spin up a new one

Doing this is responsibility of another module, say orchestrator



Orchestrator

Keeps an eye on the servers.

When one goes down,

Orchestrator spins one and add

* No human intervention
* Minimal time - outage !

But what if orchestrator is down? Who monitors it?

↳ we need orchestrator for orchestrator
↳ who monitors ↑
↳ another orchestrator ??

} This goes out of hand quickly !

Hence, we need an automated way to recover the system.

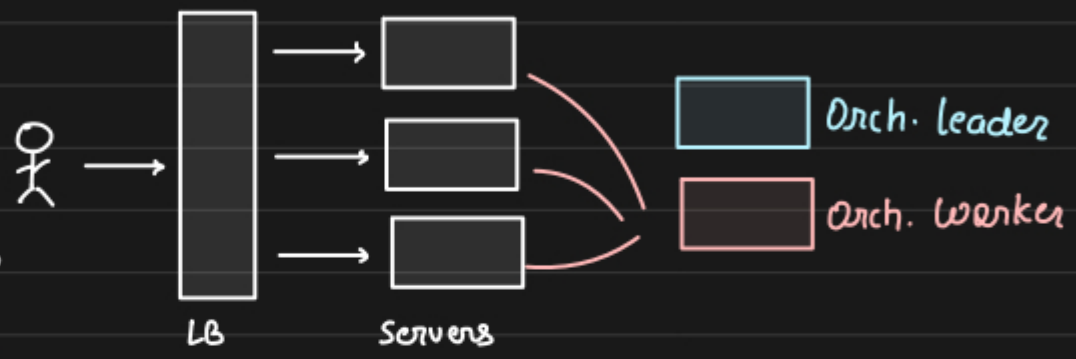
↳ when one orchestrator is down,
somehow other orchestrator comes back up
and takes responsibilities

ARPIT BHAYANI

Leader Follower Setup

We run orchestrator in leader-follower mode. multiple nodes running the orchestrator code One leader while others are workers/follower

leader keeps an eye on workers. if worker dies, leader spins up new one



Workers ping the servers and checks if they are healthy.

if orch worker finds a backend server is unhealthy, it spins new one

if orch leader finds that orch worker is unhealthy, it spins new one

if orch worker finds orch leader is dead,
they trigger a leader election and system is auto-recovered.

How workers will choose a leader depends on the leader Election algo.

Note: This is a generic concept and we can apply it to any system that needs an abilily to auto-recover.

Exercise Simulate leader failure & run leader election.
workers = threads
(no need of multiple machines)

ARPIT BHAYANI

#ASLI ENGINEERING

Client Server Model and Communication Protocols

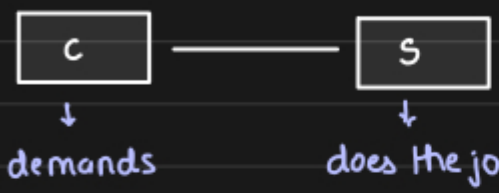
SWIPE

BY

ARPIT BHAYANI

Client Server Model

Most common way for two machines to talk to each other



eg: give me my profile info

eg: delete the post I made

eg: spin up one server

The communication happens over the common network connecting the two.

Two protocols to exchange data TCP and UDP

↑ we almost 99.99% times use this

Some important properties of TCP

1. TCP connection requires 3-way handshake ⟺ for setup
2. TCP connection requires 2-way handshake ⟺ for teardown
3. TCP connection does not break immediately after data is exchanged
   - ↳ breaks because of network interruption
   - ↳ breaks because server/client initiated it

Hence connection remains open ... almost 'forever'

Protocol over TCP

TCP does not dictate what data can be sent over it

Common format agreed upon by client & server

is called a protocol : HTTP

=

ARPIT BHAYANI

HTTP is just a format that client and server understands.

* You can define your own format, and make

1. your client send data in it
2. your server parses & processes it

Properties of HTTP 1.1

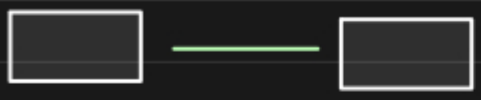
There are many versions of it - HTTP 1.1 / HTTP 2 / HTTP 3

HTTP 1.1 is the most commonly used one

1. For client and server to talk over HTTP 1.1,
   they need to establish TCP connection
2. Connection is typically terminated once response is sent to client
3. almost new connection for every request / response
   ↳ little expensive
4. Hence people pass "Connection: Keep-alive" header
   (which tells client and server to not close the connection
   * depends if server follows it or not

ARPIT BHAYANI

WebSocket



Websockets are meant to do bi-directional comm^n

Key feature: Server can proactively send data to client, without client asking for it

Because there is no need for setting up TCP, every single time we get really low latency in communication

Websockets

HTTP 1.1

Any where you need "realtime", "low latency" communication for your end user over the internet think about websockets

eg: Chat, Realtime likes on live stream, Stock market ticks

Exercise

- build a chat application using Socket IO

ARPIT BHAYANI

#ASLI ENGINEERING

Blob Storage and S3

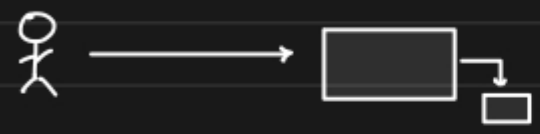
SWIPE

BY

ARPIT BHAYANI

Blob Storages and S3

Earlier when people uploaded any files, they uploaded it to "server" and were stored on the hard disk attached to it



getting a file was simple, make an API call, the handler reads the file & return

* This is precisely how /static folders/routed worked

when user uploads the file → accept it on HTTP POST

'a.txt' → create absolute path using folder mapped to '/static'

/home/arpit/www/static/a.txt

→ store the file at ↑ location

when user requests '/static/a.txt' → get the path from URL

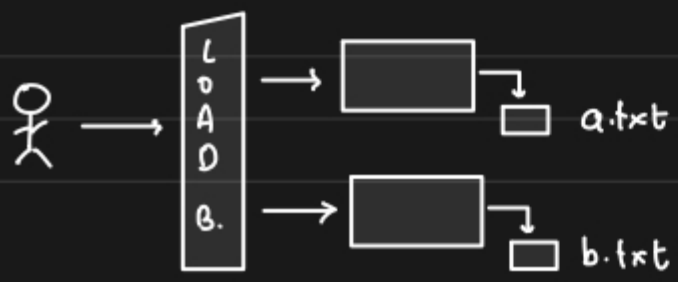
→ create absolute path

/home/arpi/www/static/a.txt

→ read the file and return

Early days of the internet

This worked well for quite some time, but won't work with multiple servers because each server will have its own disk.

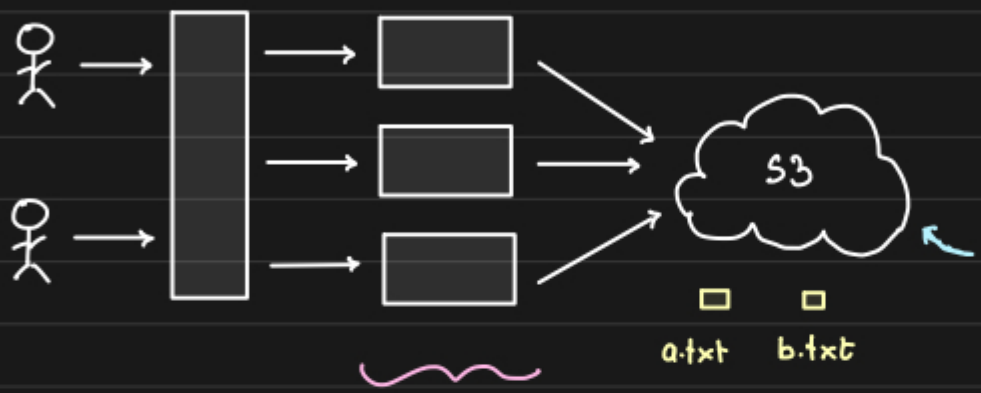


ARPIT BHAYANI

Hence, we need an infinitely scalable network attached storage / file system

↑

This is S3 / Blob Storage



Any "file" that needs to be accessible by any server, is stored at a places accessible by all

Our API servers are now stateless

On S3 you have

any unique name

buckets : (namespace) eg: insta-images, my-bucket

keys : path of the file within bucket

s3://insta-images/user123/72896.png

bucket key

You can seamlessly, create the file, replace the file, delete the file.

Not a full fledged file system.

Read entire file or segment of it

ARPIT BHAYANI

Advantages:
- cheap, durable storage
- can store literally any file → images, video, audio, text, DB backup, DB CSV exports, anything
- scalable and available
- integration with a lot of AWS and Big Data services

Disadvantages
- reads on S3 are slow
  so if you want quick reads, you should not use S3
  ↳ SSD and HDD attached to instances are better
- not a full fledged file system

You should use S3 when you want to store 'blob' that is centrally accessible.
- Database backups
- Static website hosting
- Big Data storage
- logs archival
- infrequently accessed data dumping ground

Exercise
1. go through S3 documentation an explore the API
2. read about ACL on S3
3. if possible, play around with its API

#ASLI ENGINEERING

Bloom Filters

SWIPE

BY

ARPIT BHAYANI

Bloom Filters

Bloom Filters are approximate data structures that (also probabilistic data structure)

says with 100% certainity that an element does not belong to a set.

For example,

Instagram wants to recommend reels

But it does not want to recommend something you saw already

Naive way: keep track of everything that a user saw in a set

u1 → < p1, p4, p7, p1024, p1056 >

u1 : user

p1, p4, p7, p1024, p1056 : post he/she saw

People watch 100s of reels every single day

Over time the set of all post watch by a user will be huge!!!

So what?

To check the existence of a key in set ← Super expensive and time consuming

we have to load the entire set

in memory and then check

So, can we do better?

Key Insight: Once something is "watched", you cannot take it back

↳ once a post/reel is watched by you,
we do not take it out of the set !!

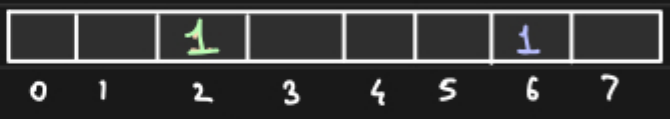
This means, storing actual data is not worth it.

This is the concept over which the Bloom Filters are setup.

Bloom Filters

Filter ≈ bit array

(eg: we take 8-bit array)



Words I want to put in

apple → f(apple) % 8 → 2

ball → f(ball) % 8 → 6

cat → f(cat) % 8 → 2

dog → f(dog) % 8 → 5 filter[5] = 0 → dog not present !!

elephant → f(elephant) % 8 → 6 filter[6] = 1 → elephant may be presen

Thus we see, when Bloom Filter says No → we can be sure

But when it says yes → we still need to check.

ARPIT BHAYANI

Space Efficiency

because it does not store keys

Bloom Filters take significantly less space to hold the information and is very efficient in checking existence (just an array lookup)

False Positivity Rate

As number of keys we put in Bloom Filter increases

the false positivity rate increases

↳ says key is present but in reality it is not

Hence, when # keys increases,

1. we have to re-create bloom filter with larger size and populate the keys again
2. estimate the max keys and provision a large one to start with

Practical Bloom Filter

We do not have to re-implement Bloom Filter

There are libraries in every single language

Redis has it as one of its core feature

(now a days, mostly people go for this)

Practical Application

Use it whenever

- you insert but not remove data
- you need a NO with 100% certainty
- having false positive is okay

eg: medium recommendation
eg: feed generation
eg: web crawler
eg: Tinder feed

Exercise

1. setup Redis locally
2. read Redis's documentation on Bloom Filter
3. write small code to play around with it (understand how to use it)
4. try to get one False positive result

ARPIT BHAYANI

#ASLI ENGINEERING

Consistent Hashing

SWIPE

BY

ARPIT BHAYANI

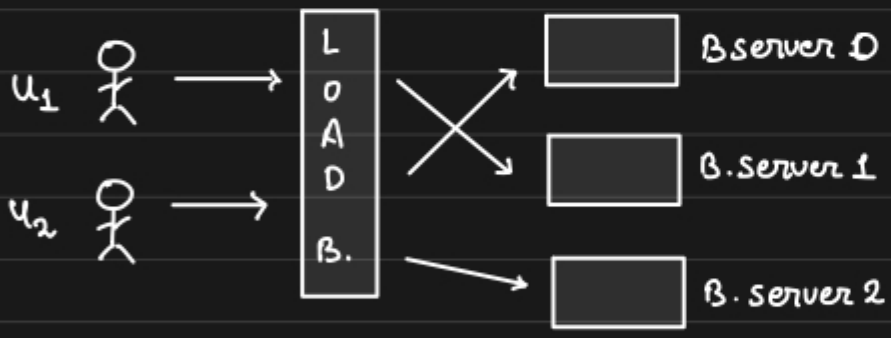
Consistent Hashing

One of the most amazing and popular algorithm out there.
and the only problem it solves is Data Ownership

We first understand Consistent Hashing and
then look into practical implementation

Hash Based Ownership

Say, we have a load balancer
and when a request comes in, it uses hash of access token
to decide which backend server to forward it to



fn(r.id) % 3
↳ access token in the request

SHA-256 or SHA-128 are popular choices.

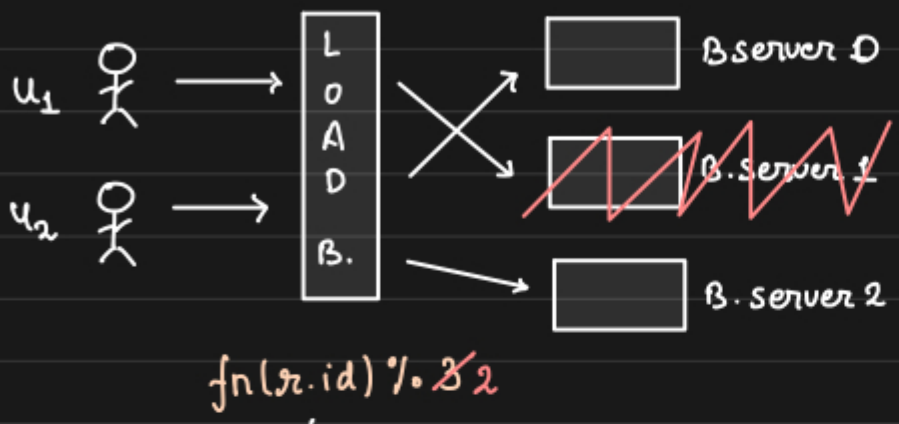
SH256 (u_1) = x

Server = x % 3 = i

Note: Hashing logic is NOT a 'service', but just a simple code running in the load Balancer's code
↳ more of a function

ARPIT BHAYANI

if one of the servers is taken down, then the routing function changes



now the requests will be evenly distributed between remaining two servers.

So, no hiccups !!

fn(r.id) % ~~3~~ 2

↳ access token in the request

load Balancers are stateless

Because the API servers are stateless,

which means every server is equally capable of handling requests

it does not matter if

request that was handled by server 1

now starts going to server 0.

This is precisely why we see 'Hash Based Routing'

as one of the most common ways of routing for

Stateless backends

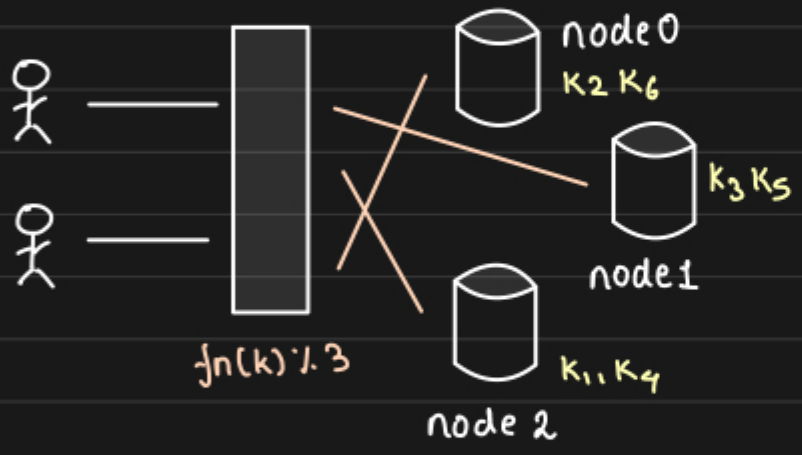
eg. Load Balancer + API servers

ARPIT BHAYANI

Hash Based 'routing' (ownership) for distributed storage

Instead of stateless API request, say we are having a stateful distributed storage

1. nodes store the data
2. proxy forwards the request to a node
3. end user / client talks to proxy



Picking which node "owns" the data depends on the Hash Based ownership

Say we want to store 6 keys K_1, K_2, K_3, K_4, K_5 and K_6

$K_1 \rightarrow fn(k_1)\ \%.\ 3 = 2$
$K_2 \rightarrow fn(k_2)\ \%.\ 3 = 0$
$K_3 \rightarrow fn(k_3)\ \%.\ 3 = 1$
$K_4 \rightarrow fn(k_4)\ \%.\ 3 = 2$
$K_5 \rightarrow fn(k_5)\ \%.\ 3 = 1$
$K_6 \rightarrow fn(k_6)\ \%.\ 3 = 0$

okay to do it when workload is stateless like API load balancer

Challenge: if a storage node is removed or added

the proxy cannot just forward request to any arbitrary node because it won't have the data

ARPIT BHAYANI

Repartitioning

When the number of nodes change the proxy will change the routing function and it would now become $fn(k) \% 2$

Now, all the keys would need to be re-evaluated and moved to the correct node.

↳ involves a lot of data transfer

$K_1 \rightarrow fn(k_1) \% 3 = 2$ $K_4 \rightarrow fn(k_4) \% 3 = 2$

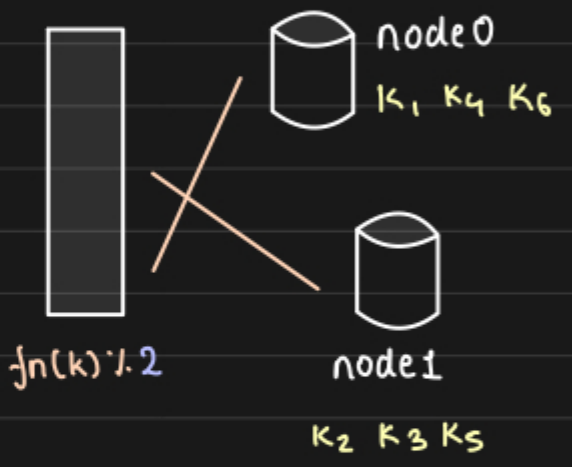
$K_2 \rightarrow fn(k_2) \% 3 = 0$ $K_5 \rightarrow fn(k_5) \% 3 = 1$

$K_3 \rightarrow fn(k_3) \% 3 = 1$ $K_6 \rightarrow fn(k_6) \% 3 = 0$

$K_1 \rightarrow fn(k_1) \% 2 = 0$ $K_4 \rightarrow fn(k_4) \% 2 = 0$

$K_2 \rightarrow fn(k_2) \% 2 = 1$ $K_5 \rightarrow fn(k_5) \% 2 = 1$

$K_3 \rightarrow fn(k_3) \% 2 = 1$ $K_6 \rightarrow fn(k_6) \% 2 = 0$



Can we minimize the data movement?

This is where consistent hashing comes in

What consistent hashing is?

an algorithm that helps in determing data ownership ← who owns this data

What consistent hashing is not?

it will not do data transfer for us

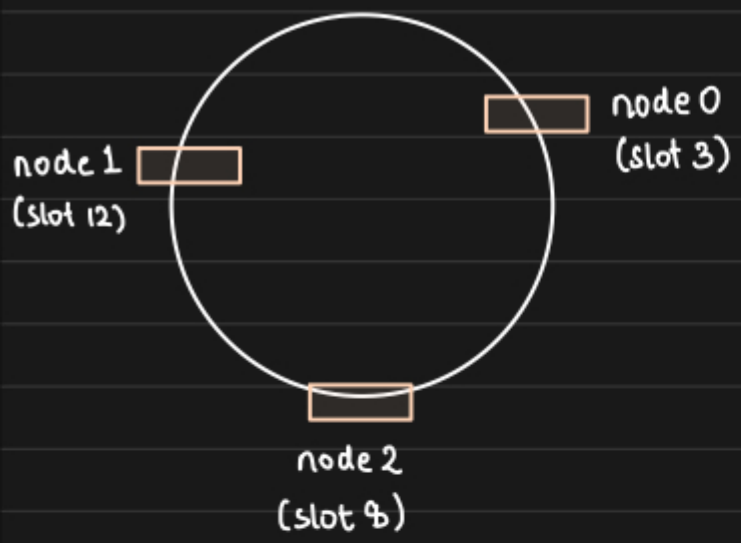
it is not a "service" in itself

ARPIT BHAYANI

[0 to 15]

Simple visualization

range of $[0, 2^{128}]$



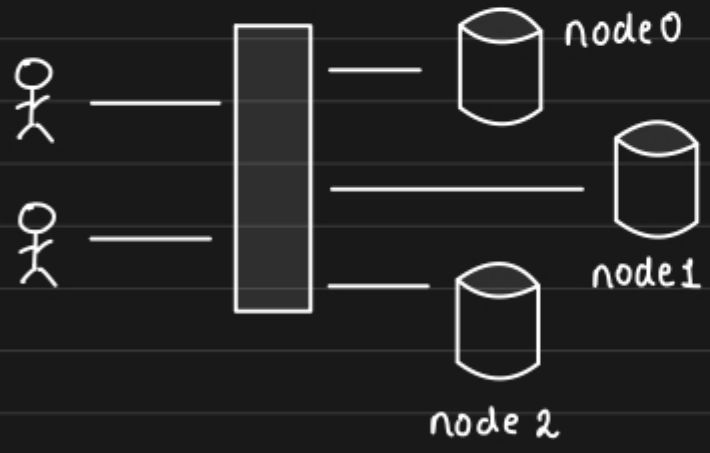
Hash function (SHA 128)

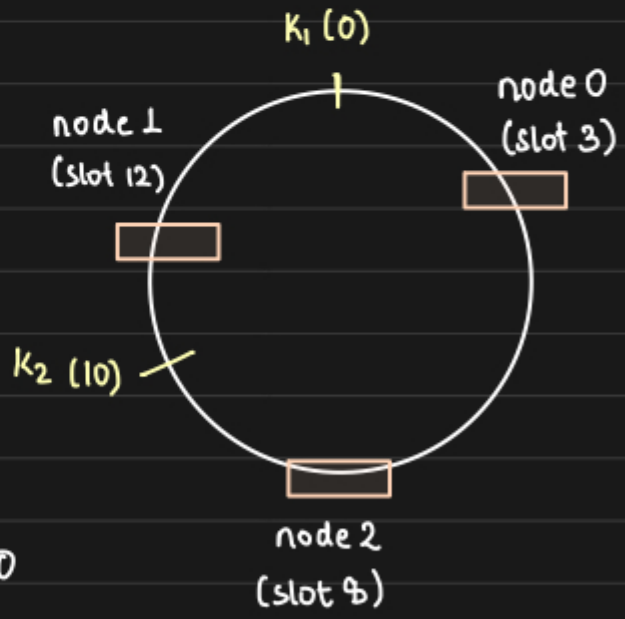
given hash functions are cyclic we can visualize it as a ring of integers

Every node occupies one slot in the ring, the slot is calculated by passing node's IP to hash fun^cn

The ring can be modelled as a simple array and be part of proxy.

Here's how the ownership is determined





$K_1 \rightarrow$ hash $\rightarrow$ 0 $\rightarrow$ node to right $\rightarrow$ Node 0

$K_2 \rightarrow$ hash $\rightarrow$ 10 $\rightarrow$ node to right $\rightarrow$ node 3

Consistent Hashing ring will only tell you the node that "should" own it.

ARPIT BHAYANI

Scaling up

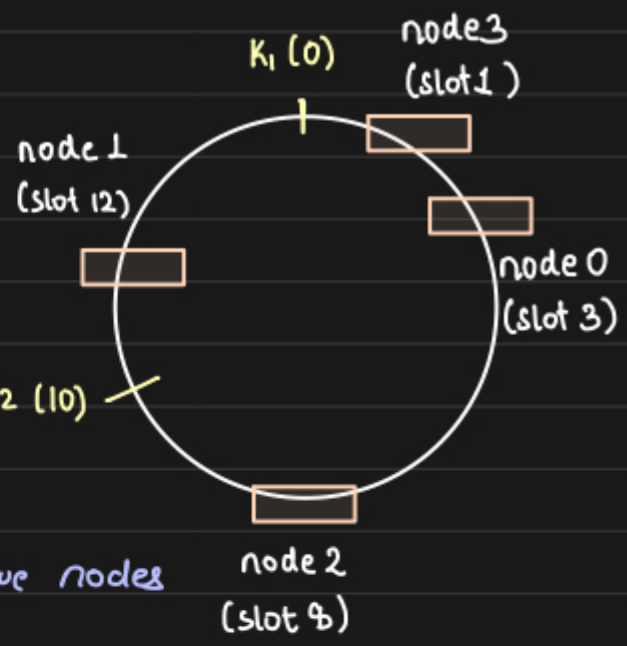
When we add a new node to the "ring"

Say node 3 hashes to slot 1

The keys that hashed between slot 12 and slot 1 will now be "owned" by node 3 instead of node 0

Other keys continue to remain at the respective nodes

↳ Minimal Data Movement



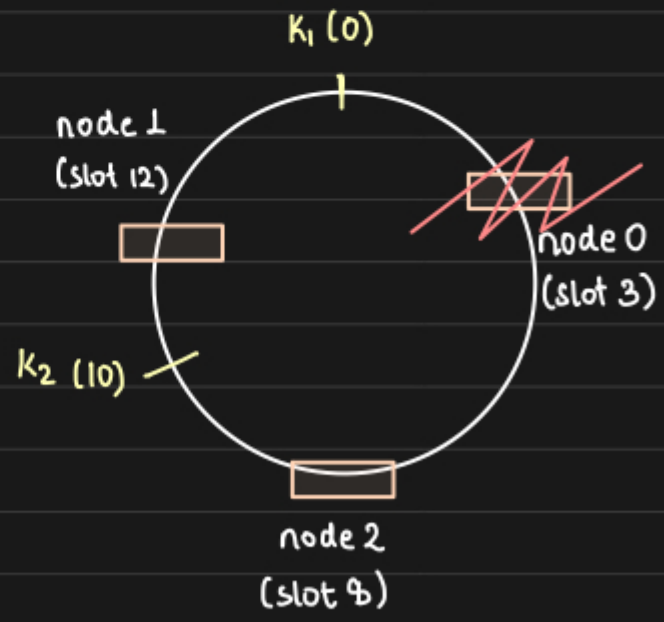
Operationally, you just have to

1. snapshot node 0 2. create node 3 3 delete unwanted keys

Scaling Down

Say we scale down and remove node 0. All the keys that were owned by Node 0 will now be owned by Node 2 (next in the ring)

Minimal Data Transfer



Operationally,

1. copy everything from Node 0 to Node 2
2. remove node 2 from the ring
3. delete node 2.

ARPIT BHAYANI

Exercise

1. understand consistent hashing
2. read the blog https://arpitbhayani.me/blogs/consistent-hashing
3. implement it in your favourite programming language

The implementation just contains

2 arrays + Binary Search

#ASLI ENGINEERING

Introduction to Big Data Processing

SWIPE

BY

ARPIT BHAYANI

Big Data Processing

When one machine is not enough to process the data

we divide and conquer → This essentially is Big Data Processing!

Companies use this to process massive amount of data and extract insights out of it, train ML models, move data across databases, and much more

- when too much data needs to be processed quickly, we use Big Data tools.

* all these fancy processing on commodity hardware

Counting word frequency

Given a 1 TB text dataset find frequency of each word

Approach 1: Simple

1. load data on one machine (disk)
2. read it character by character
3. when space is encountered, word_freq[word] += 1
   update an in-mem hash table
   do count++

This is a simple approach that runs in O(n) time approx.

But because only one machine is doing it,

it will take a long time so, can we parallelize it?

Yes... add threads....

Approach 2: Threads

You can easily parallelize the code

each thread can handle a chunk

of the file/dataset and can do `word_freq[word] += 1`

But, what if the dataset is not 1 TB but 100 TB ? (big data or smaller hardware)

↳ something that does not fit on a single machine, or

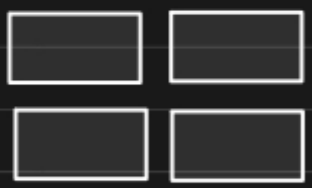
↳ even if it did, it is slow to compute

↳ threads are bounded by CPU cores

↳ limited computational capabilities of underlying hardware

Instead of one machine, can we distribute the workload

across multiple (often smaller) machines



↳ and leverage parallelism

Approach 3: Distributed Computing

More computers, more CPU, more processing

idea: split the file into 'partitions'

distribute the partitions across all servers

let each server compute word freq independently

send word freq to one server (coordinator)

merge the word freq

return the result

ARPIT BHAYANI

User submits the job to 'coordinator'

coordinator distributes the job across multiple machines

machines compute and send result to coordinator

Co-ordinator merges and returns

Challenges:

1. what about failures
2. what about completion
3. what about recovery
4. what about scaling & distribution

Although, we can always do it on our own,

but if there tools to manage this for us... adopt it.

Big Data Tools manages these complexities for us

↳ we just write the business logic

- distribute across machines
- knowing which machines are doing what
- retrying in case of failures
- reprocessing in case of crashes or corruption
- cleaning up the resources once job is complete

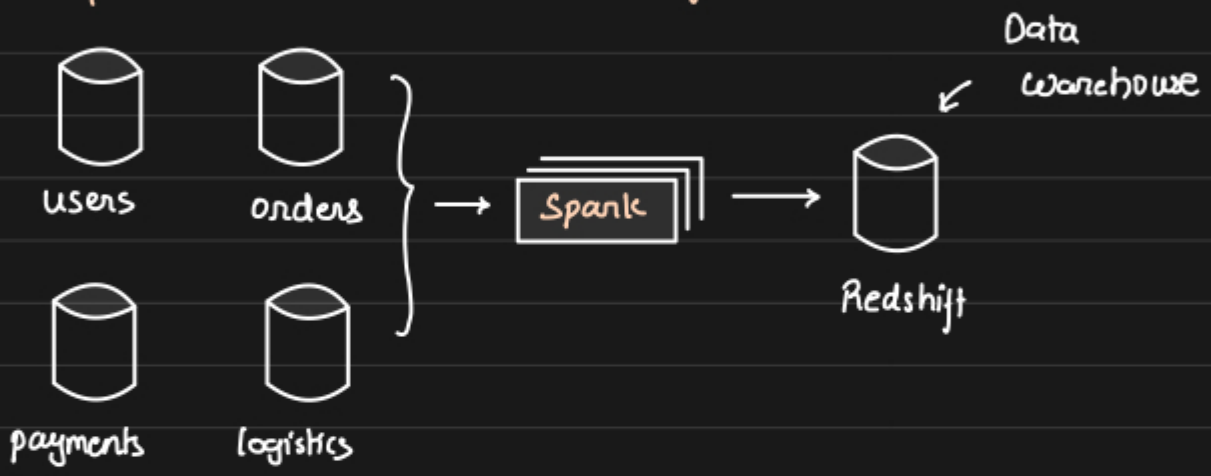
ARPIT BHAYANI

Spark and Flink

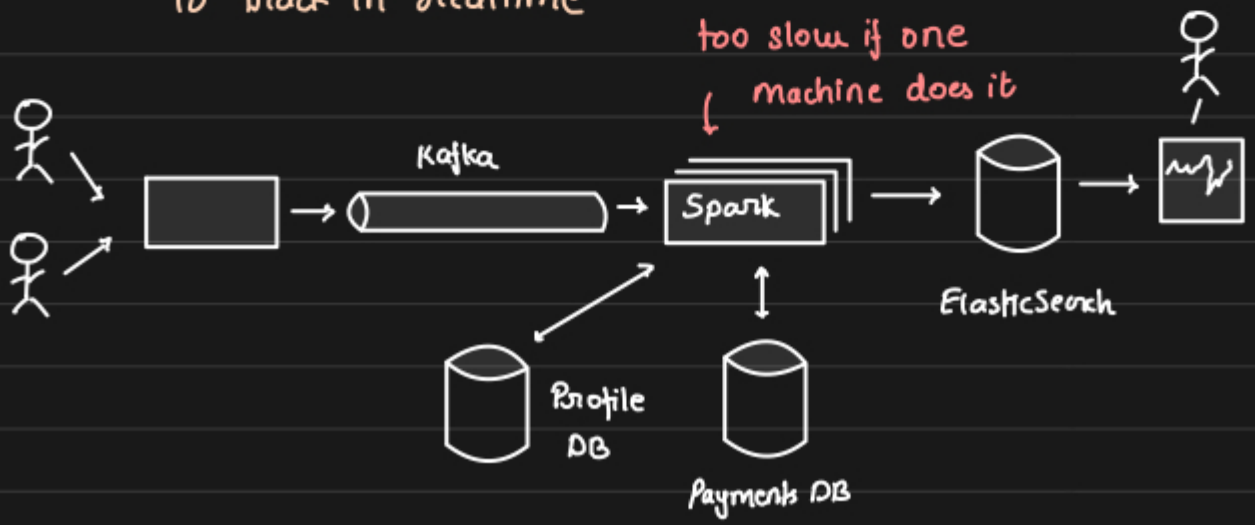
large scale data processing on commodity hardware

it has connectors to a lot of data bases and infra components

eg: combine user, order, payments and logistics DBs and put the result in Aws Redshift



eg: when any activity is happening, events are streamed to kafka. In realtime enrich the events and put them in analytic DB to track in realtime



ARPIT BHAYANI

There are plethora of tools available, each solving a niche problem } But the overall concept remains the same.

It is easy to get overwhelmed by the ecosystem... but

Exercise

1. Setup spark locally
2. take some sample datasets
3. write spark jobs to process the data

* Refer to any series on YouTube and get started

* Recommend Sales data } lots of stats and analytics

4. write a spark job that

- reads from kafka (each event has user id)
- for each event it goes to users db to get detail
- spark job should enrich and dump data on disk as JSON file

eg: kafka event BLOG_PUBLISHED (Topic)
payload <user_id, blog_id>

user detail in user db { name, email, bio }

Output: {
user_id, blog_id,
user: {
name, email, bio
}
}

ARPIT BHAYANI

#ASLI ENGINEERING

Designing e-commerce product listing

SWIPE

BY

ARPIT BHAYANI

Motive: Understand how a system is built and taking a structured approach is building one. Understand core infra components and how they fit

Designing e-commerce product listing

For a small shop (100 items) design a system through which the shop owner can

1. add a new product
2. update / delete existing product
3. list all the products on the website
4. customers should be able to quickly access the catalog

* Payment is out of scope for this problem statement

Storage

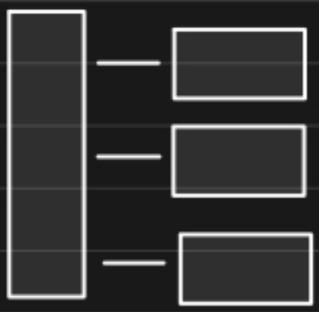
100 x 1KB ≈ 100 KB data

- not huge data, only 100 rows ↗ fits on single node
- there seems to be a structure

← 'products' table to hold all the catalog

we can go ahead with SQL DB

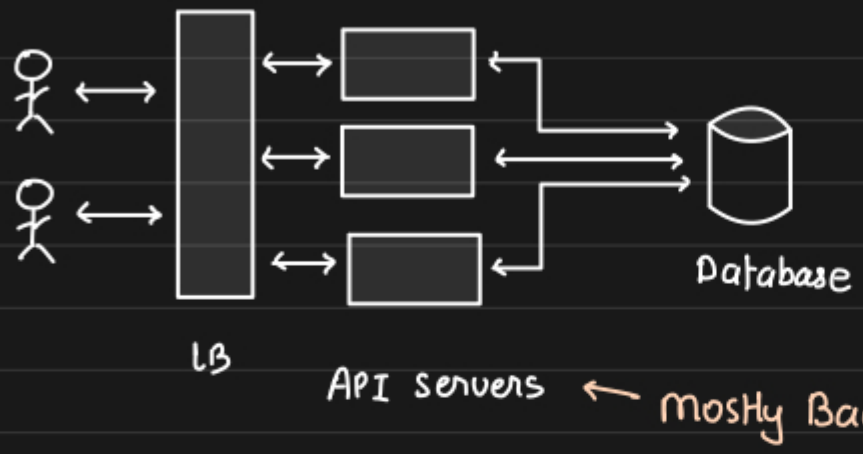
Serving the data



- simple REST based HTTP webserver is fine
- we will need many (handle large request)
- hence put a load balancer

Load Balancer will have DNS like api.mystore.com

Architecture until now



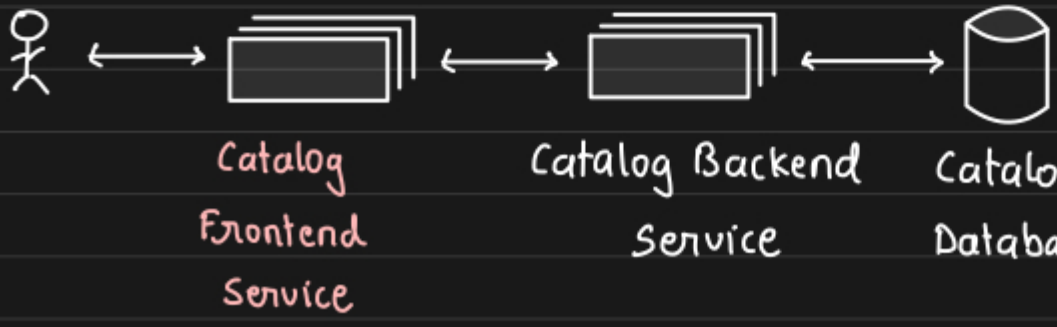
✓ storing the data
✓ serving the data

← Mostly Backend API servers

* Instead of drawing LB and multiple servers everytime we simply draw the above diagram as

We name our service Catalog Backend Service

End user will not directly interact with backend; needs a frontend

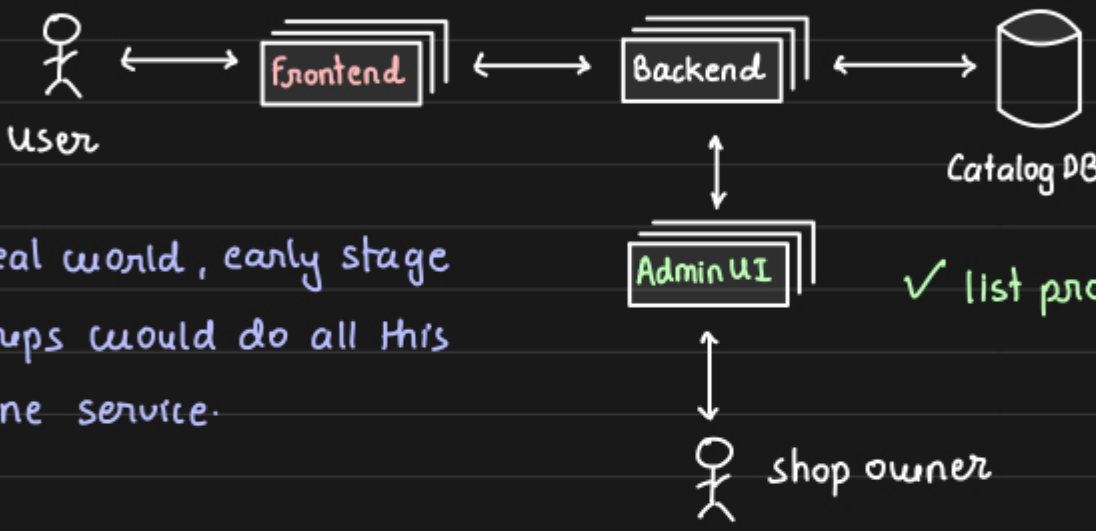


ARPIT BHAYANI

Shop owner's admin interface

Shop owner needs Admin console to manage the catalog

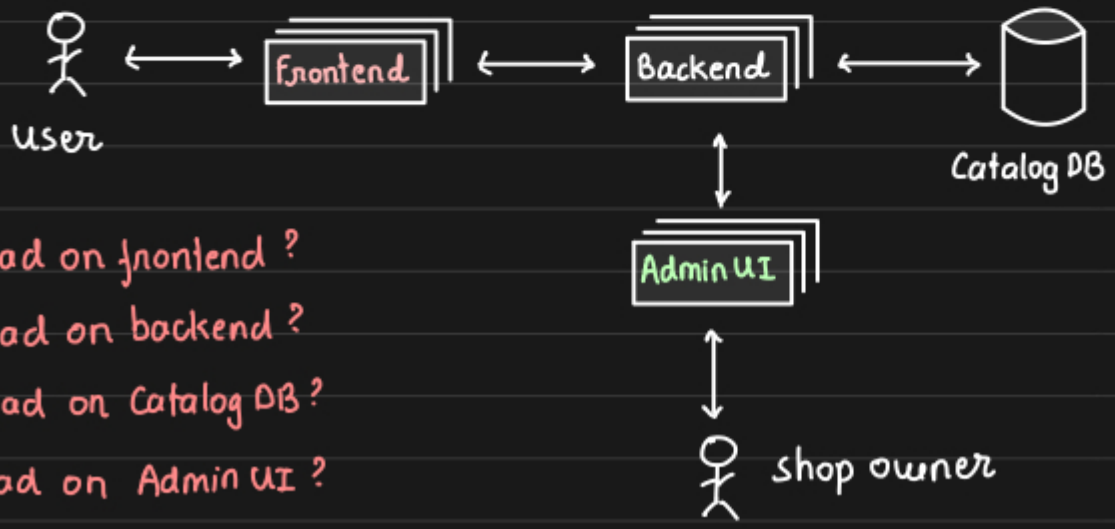
We can keep the Admin UI separate frontend service which interacts with the same Backend service to do admin things



* in real world, early stage startups would do all this in one service.

✓ list product on website

Let's understand the load on each component ↓



load on frontend ?
load on backend ?
load on Catalog DB ?
load on Admin UI ?

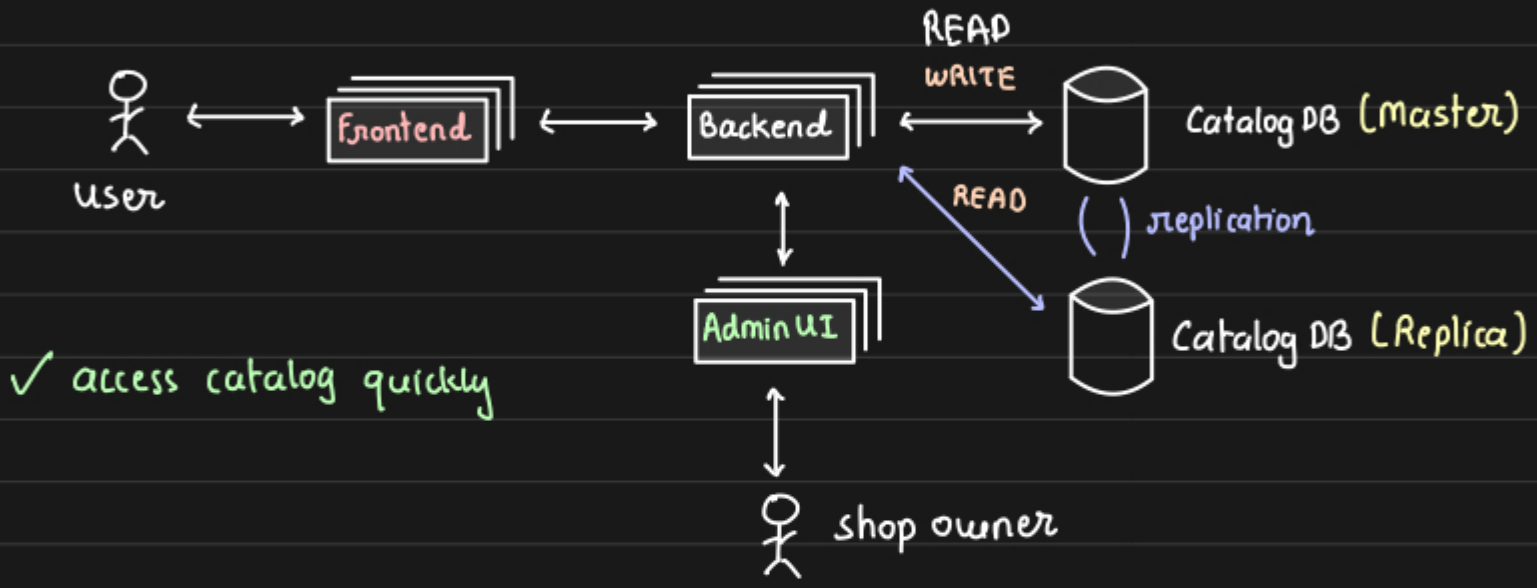
ARPIT BHAYANI

Scaling DB

When a lot of load comes in, the DB will be under heavy load.

Given the use case is to list the pages

the load is certainly Read load, hence we add Read Replica



✓ access catalog quickly

Exercise

1. Design DB schema for this system
2. Write a simple backend API service exposing APIs
3. Setup DB replication
4. move read APIs to read from Replica
5. add a cache
6. update catalog & invalidate the cache

#ASLI ENGINEERING

Designing Rate Limiter

SWIPE

BY

ARPIT BHAYANI

Motive: understanding that more boxes ≠ better system. Understanding tradeoffs making scaling decisions

Designing Rate Limiter

Systems break down under tremendous load.
and we need to ensure that doesn't happen.

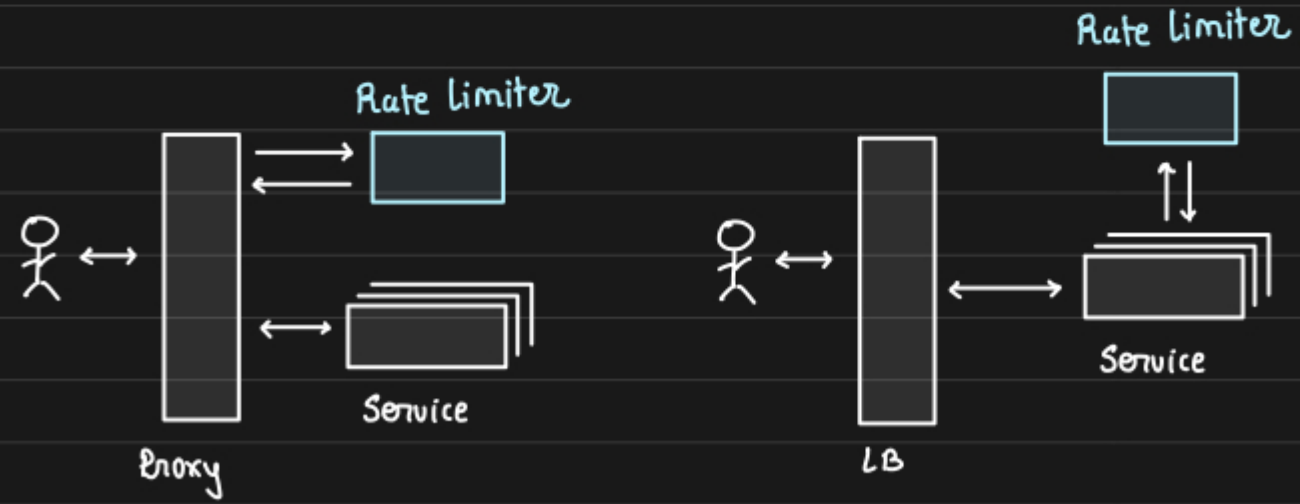
Design a rate limiter, that

- limits the number of requests in a given period
- allows developers to configure threshold at a granular level
- does not add a massive additional overhead

Rate limiter

- First line of defence
- Any incoming request is first consulted against rate limiter
- if we are under limits, go through
- otherwise, reject with error (429)

Where does it fit?



* Request does not even hit the service

ARPIT BHAYANI

Dissecting Rate limiter

Rate limiter needs to track # requests in a given period

Hence we need a database to hold the count, but which one?

1. for every incoming request, we would update the db
2. for every incoming request, we would read from db (aggregate)
3. we need ability to "clock" the time

Say, we rate limit per user

1. we store # request in current period
2. once the period is over we reset the counter

user_id → count

Resetting the counter ≈ expiring the key

↳ KV store + Expiration → Redis

↓

also gives us fast in-mem writes

+ periodic persistence

Checking and updating ← pick one

1. we expose service (HTTP endpoint) that talks to rate limiter database
2. we let the services directly talk to rate limiter DB

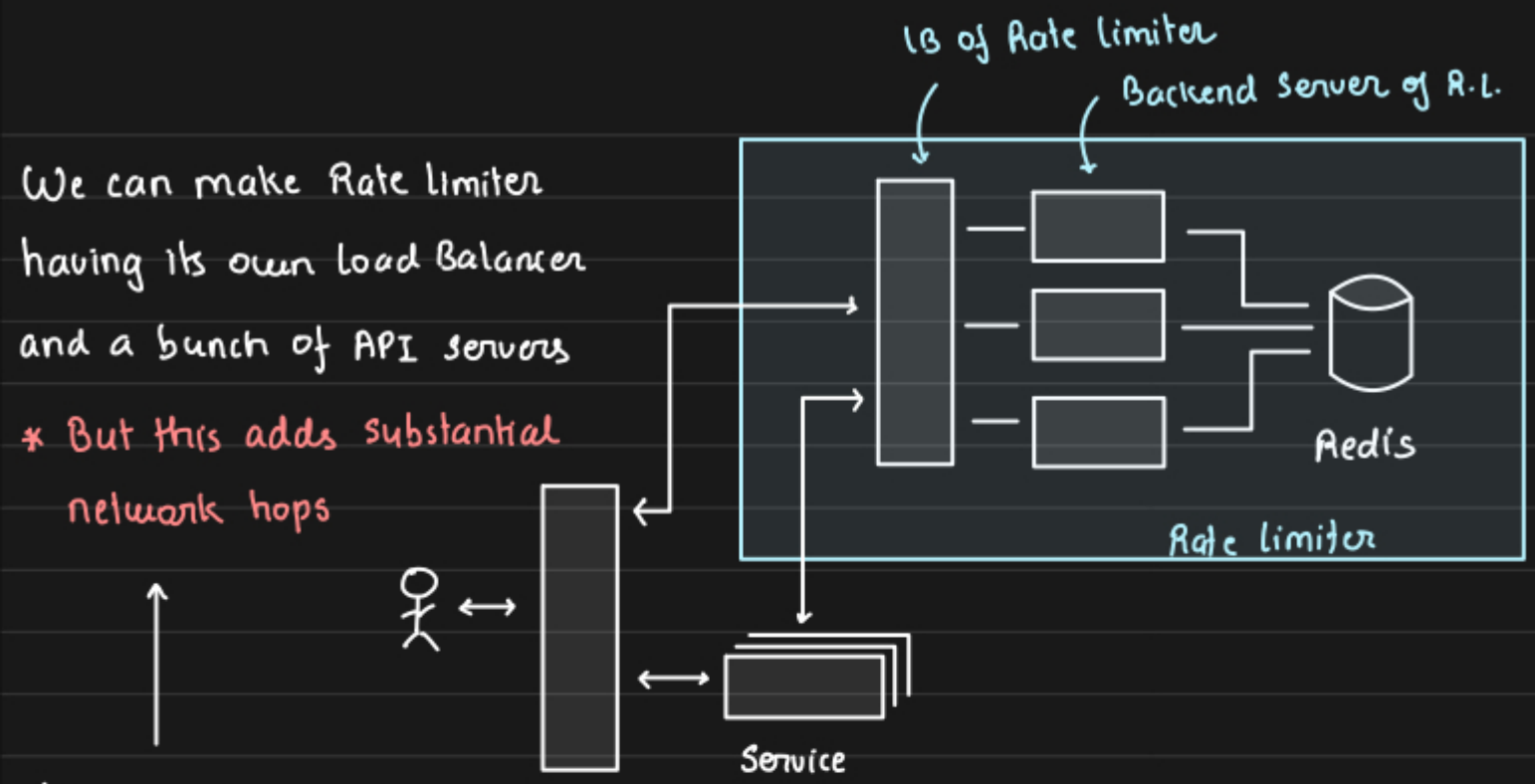
↳ saves one network hop

↳ but exposes critical internal details

ARPIT BHAYANI

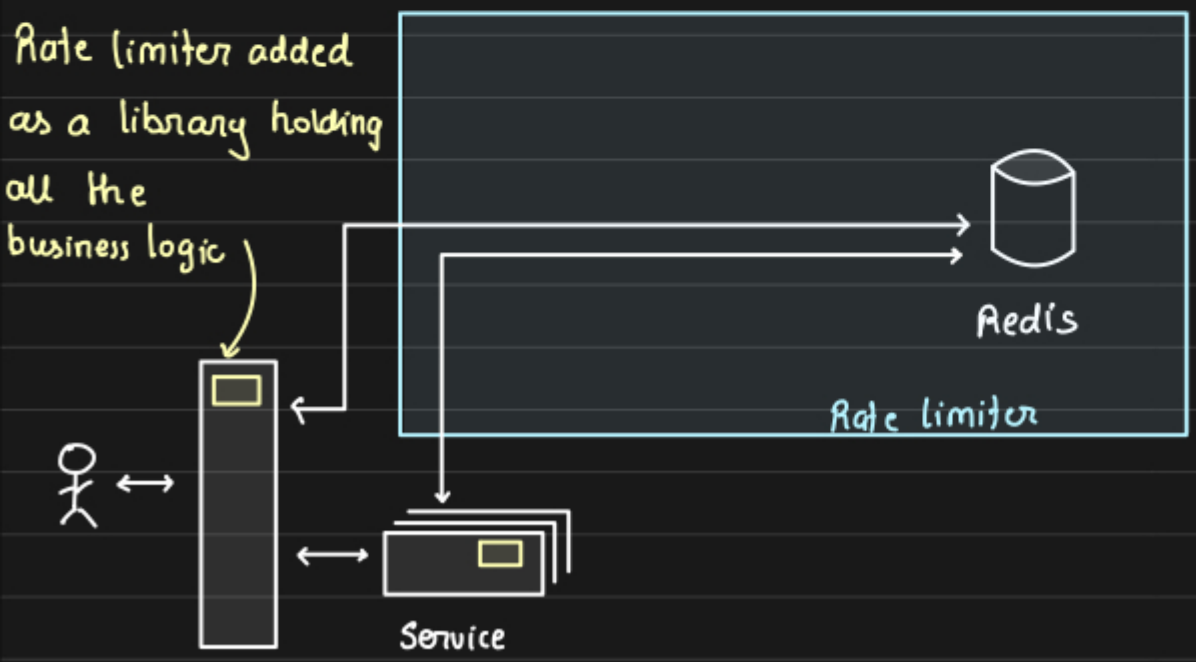
We can make Rate limiter having its own load Balancer and a bunch of API servers

* But this adds substantial network hops



Nothing wrong in this, just be aware of the trade off

To reduce the overhead of checking the Rate limiter, we let proxy/service directly manipulate the rate limiter database (saving 2 hops)



ARPIT BHAYANI

Scaling Rate limiter

Given the services are directly talking to the Rate Limiter Database

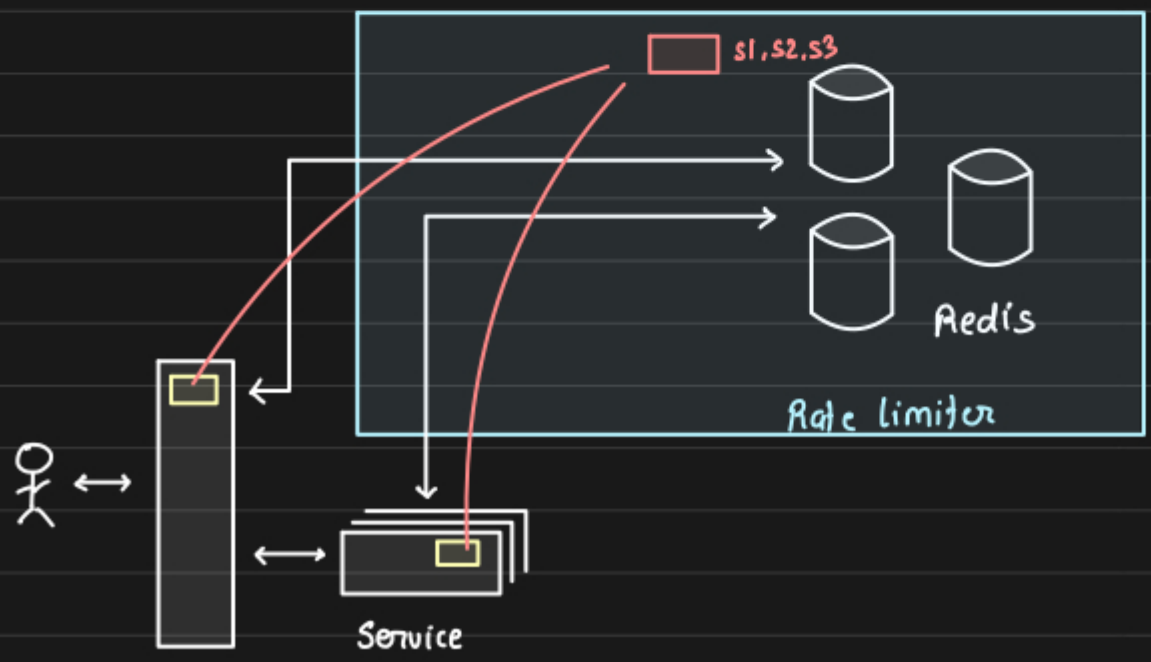
Scaling rate limiter = scaling the database

1. should we scale vertically? Yes
2. should we add read replicas? No! traffic is not read heavy
3. should we shard the database? Yes

↳ any function in backend service
- will extract user_id from token
- check rate limiter DB
- if good, the proceed
- otherwise, block

← Note: the access is key based
- user_id → count (4+4)
- ip → count (16+4)
- token → count (16+4)

Hence we can easily shard the database to handle more load



20B x 100M
= 2000 MB
≈ 2GB

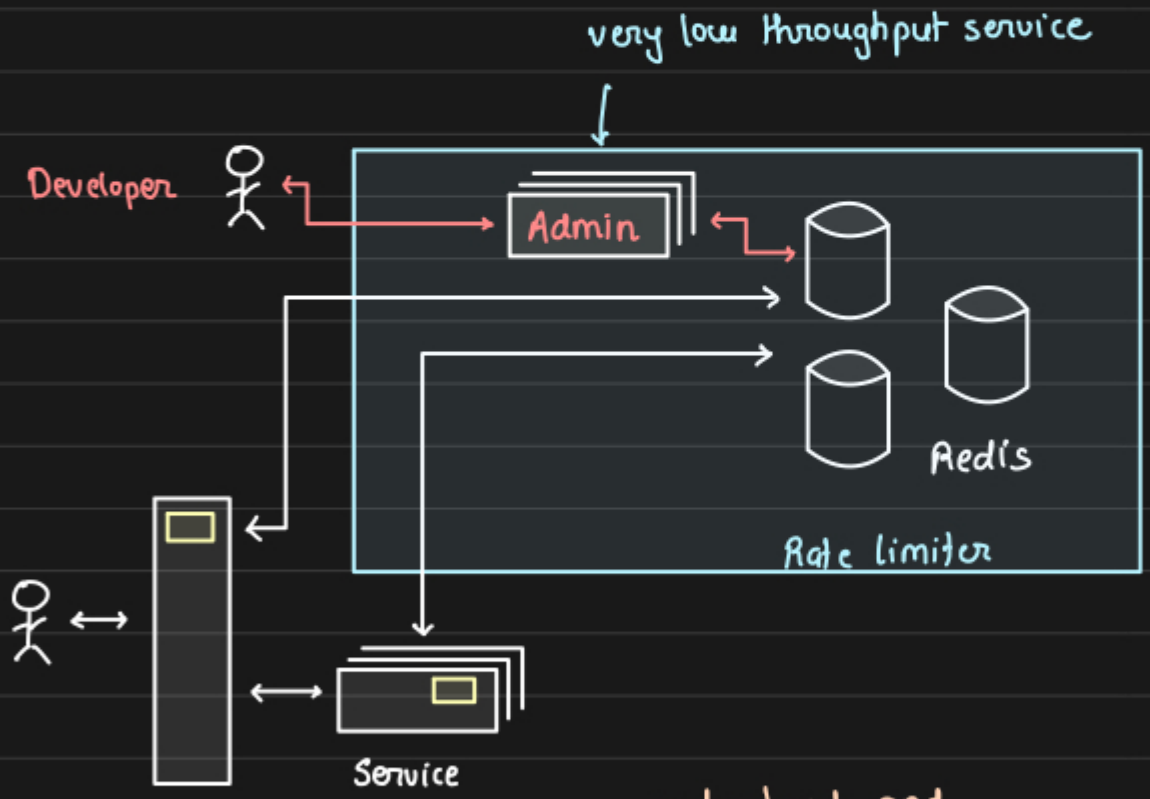
Compute is the concern not data

ARPIT BHAYANI

Admin Console

It is better to have a small Admin Console (Frontend / Backend) that is used by the developers to reset counters and debug when things go wrong.

The service will also provide observability on infra and rate limiting DB like # keys, # request blocked, cpu/mem load, etc



understand and then implement

Exercise

Read and implement : leaky bucket, Fixed window and Sliding window algorithm using Redis

#ASLI ENGINEERING

Designing a Notification Service

SWIPE

BY

ARPIT BHAYANI

Motive: Designing an extensible system and reading unsaid requirements

Designing a Notification Service

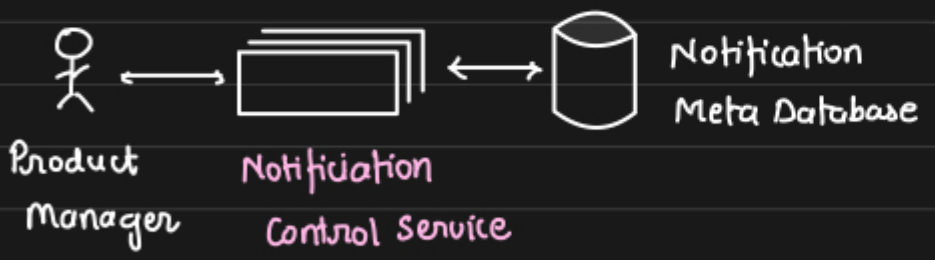
Design a notification service that sends notifications to users across channel
The system needs to be horizontally scalable and should support a very high fan-out.

Instead of directly jumping to send "millions" of notifications, lets start simple.

Notification Template

We need a UI and simple backend to create notification templates that will be configured by some internal team.

Number of notification templates will not be huge and will fit on a single machine. Hence, we start with relational DB



← Stores all templates

Hello {{ user.name }},
Order from XYZ and get {{ discount }} off !!

When any team| product wants to send notification

1. create a template using the Notification Control Service
2. specify the variables
3. note the ID of the defined template

ARPIT BHAYANI

Notification Channels

User can be notified via multiple channels

1. email
2. android push notⁿ
3. apple push notⁿ
4. SMS

} For each of these channels there are providers that expose APIs. We invoke them programmatically to send notification at the very instant

Some examples: SES, Mailgun, One Signal, Twilio, Msg91, etc.

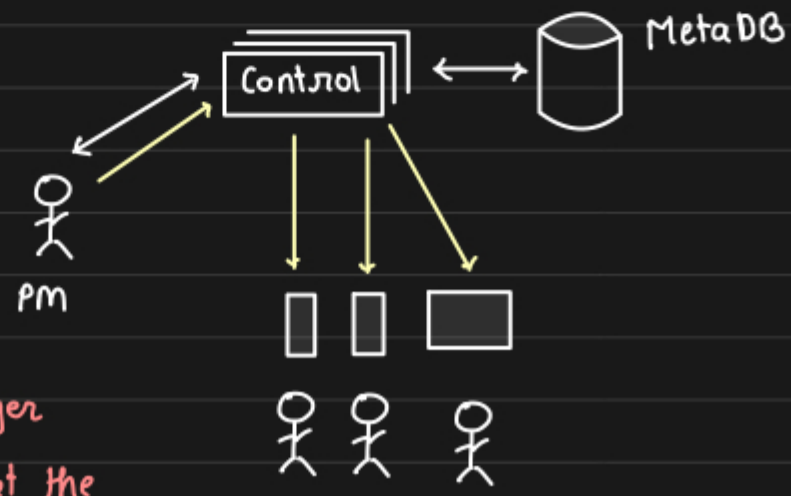
So, the servers that will send / trigger the actual notification will have to configure their SDKs / libraries.

Note: The API calls are expensive network calls with high latencies

↳ one machine will not be able to make millions of concurrent network calls

Simple Notification Flow (One user)

1. PM creates Notⁿ Template
2. PM triggers notⁿ, the control service notifies the user instantly



* What if PM wants to trigger thousands of notifications at the same time?

ARPIT BHAYANI

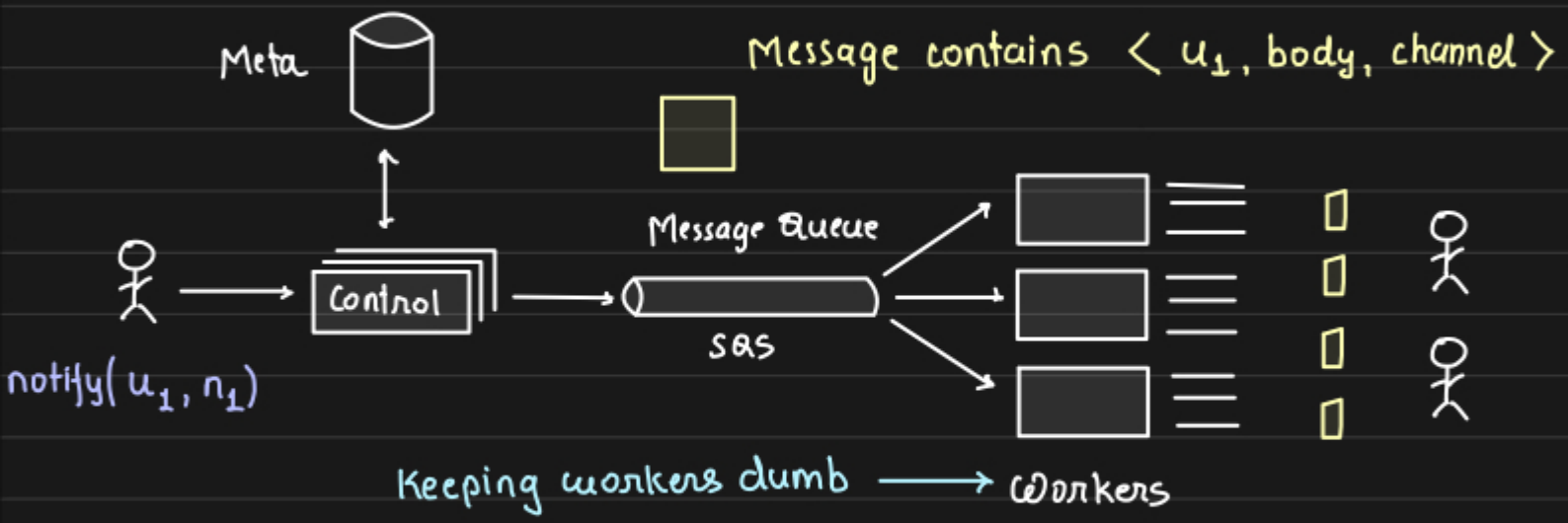
1. triggering one notification for every user is pain for PM
2. Control service becomes the bottleneck (network call takes time)

This is a classic place of making thing asynchronous + provider outage?

Asynchronous

Through control service

Sending notifications to thousands of users synchronously is a bad idea!



This architecture solves retries does not overwhelm Notification Control

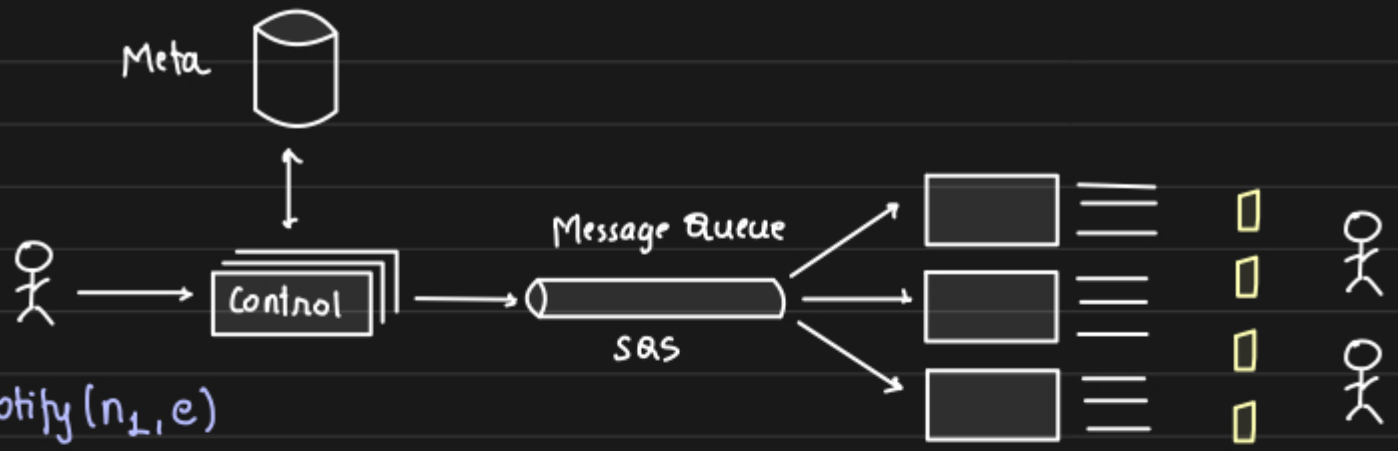
Bulk Notification

Above architecture works well when we have moderate traffic and notifications are triggered one on one.

a typical usecase: Notify 'everyone'

User experience: PM submits a job to notify everyone and we need to take care of everything else.

ARPIT BHAYANI



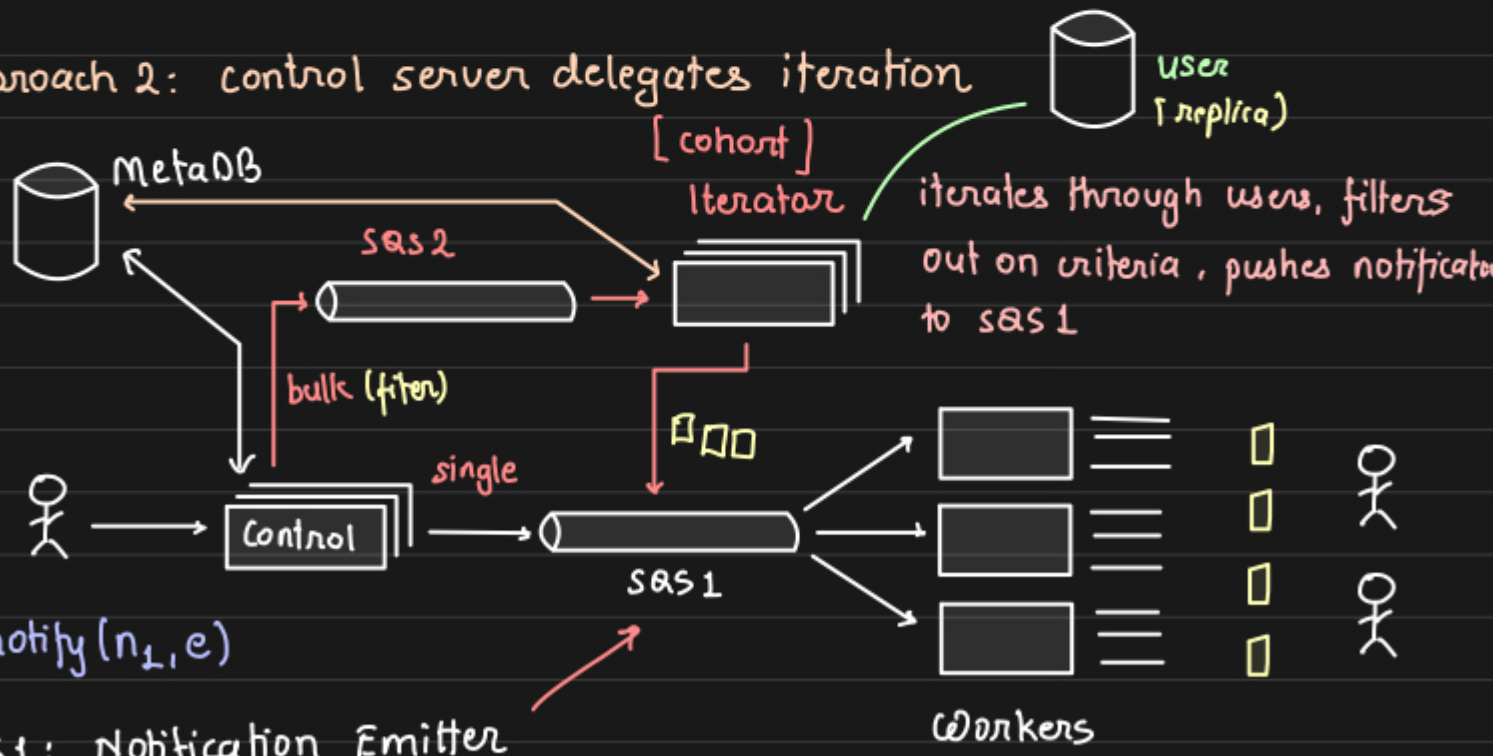
notify (n_1, e)

Approach 1: control server iterates

↳ iterating through million rows take time

↳ plus it eats up control service, whose responsibility is to accept "notification requests"

Approach 2: control server delegates iteration



notify (n_1, e)

SQS 1: Notification Emitter

SQ2: User iterator, and anything that requires some processing.

ARPIT BHAYANI

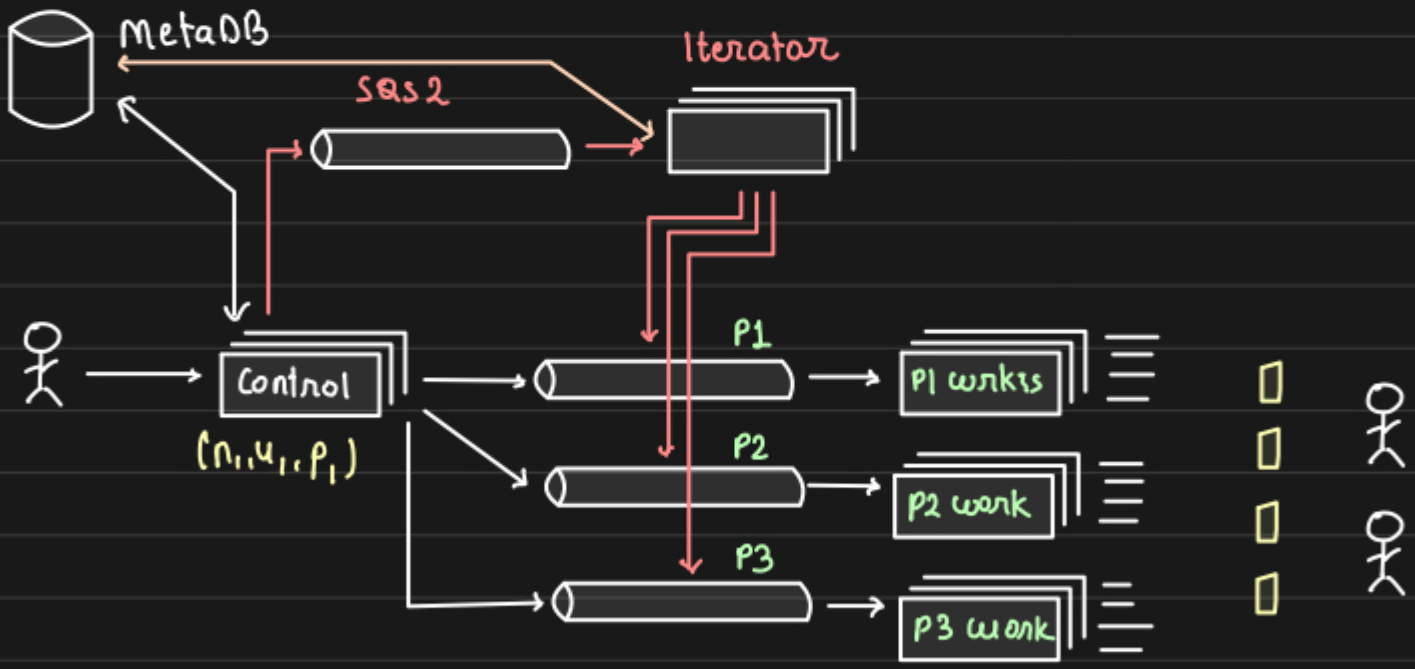
Important notifications

Some notifications are more important than others

eg: Appointment Reminder >>> Marketing push

In our current archechure, one marketing campaign will keep executors busy and all other notifications will starve in the Queue.

To solve this: instead of having one Notification Emitter Queue, have multiple - priority wise (P1, P2, P3) & each having its own set of workers



✓ Horizontal scalability - add more queues
add more workers

✓ load isolation and avoiding starvation

✗ Duplicate notifications

ARPIT BHAYANI

It is really irritating to receive marketing notifications and it is the worst if we receive multiple of the same campaign

To ensure we do not accidentaly send multiple notification from same campaign we keep track in a database (KV store is fine)

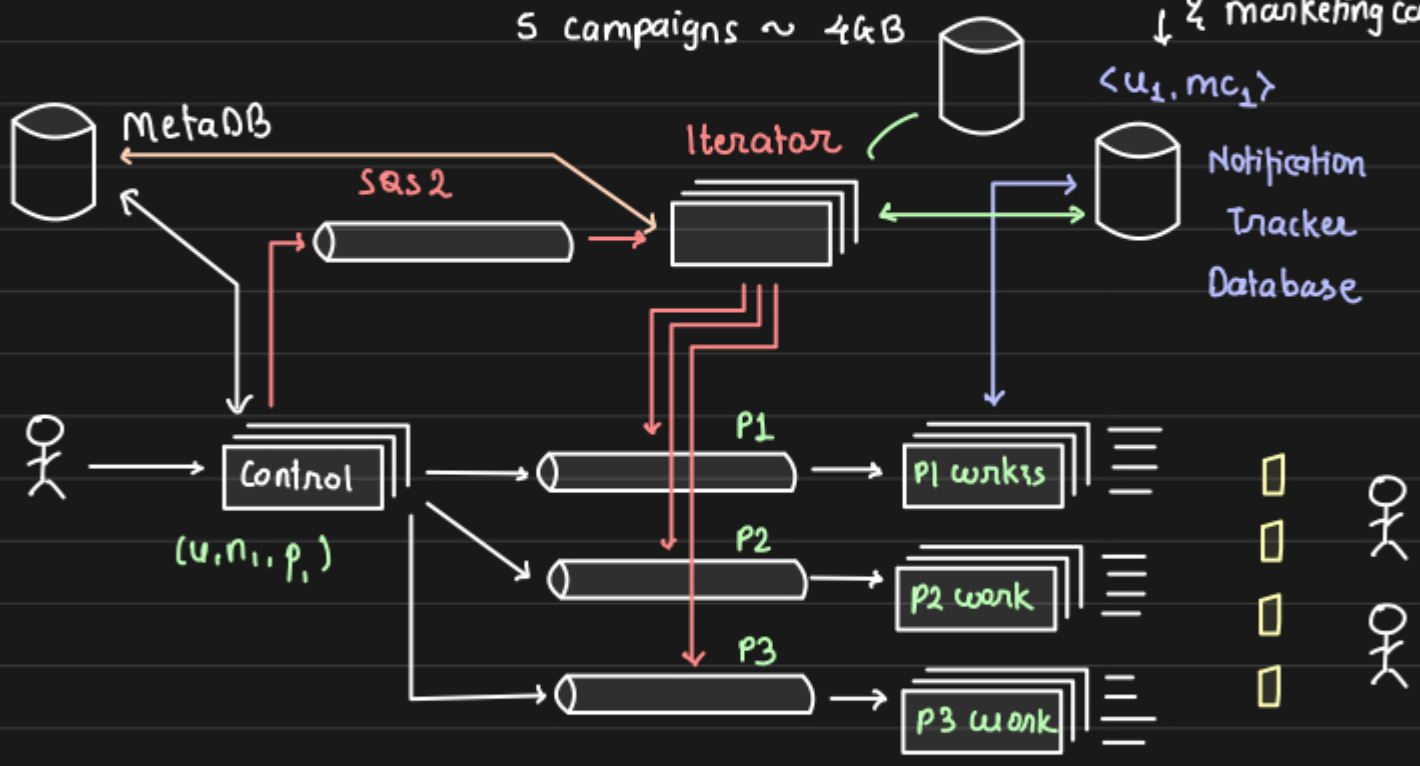
Delete or archive the old data to keep system performant

$100M \times (4B + 4B) = 800MB$

5 campaigns ~ 4GB

Keep track of user & marketing campaign

$\langle u_1, mc_1 \rangle$



Notification Tracker DB

- light weight & shard-able
- in-memory with periodic persistence
- * Bloom Filter support a plus
  - ↳ space efficient (mc_1 : BF<....>)

} Redis satisfies all the 3 requirements

ARPIT BHAYANI

#ASLI ENGINEERING

Designing realtime abuse masker

SWIPE

BY

ARPIT BHAYANI

Motive: Not everything is a service.

Designing realtime abuse masker

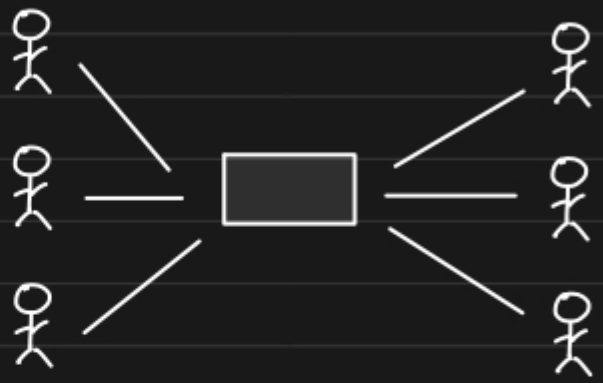
Say, one video live stream is powered by one server

All participants part of that live stream (max 100) are connected to the same server

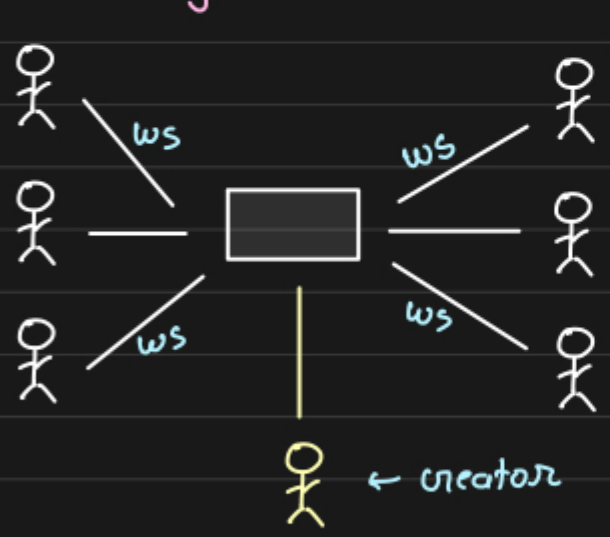
In this live stream people can send text messages (broadcast)

We want to ensure abuses are masked

S*** / f***



Understanding Architecture



- live stream of creator is captured over RTMP
- participants are connected to the server over WebSocket
- participant can send message to all

How socket.io works?

Socket IO has notion of rooms.

Simple Socket IO process is running on server.

We create "room" for this live stream

Participants join the room

When message is sent to room, Socket IO sends to all

ARPIT BHAYANI

Abuse Dictionary

Assume that we have a list of words (abuses) stored in a text file on Blob storage like S3 (at a path)

Hence, when the live stream server starts up

- downloads the abuse file
- loads it in memory ⟶ how and which data structure ?

Masking the abuse

Given all the messages (of one live stream) will go through the same server

we have to find some logic that efficiently detects and masks

Approach 1 : Tokenize and lookup

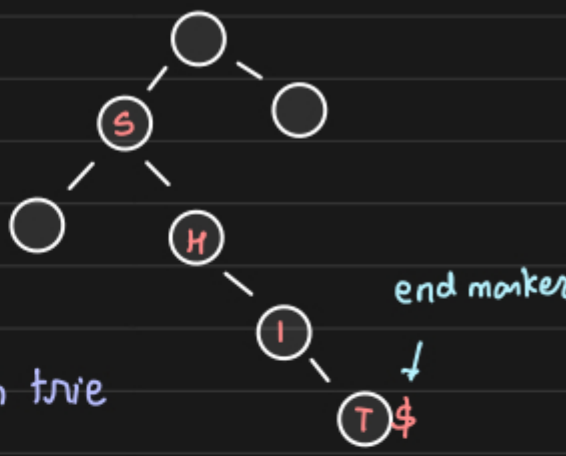
1. load all abuses in dictionary (hash table (set))
2. tokenize incoming text message
3. For each word, check if it is in abuse dict
4. if yes : mask and update the token
5. if no : copy the token

This is a simple approach but not efficient

1. tokenize (text to list of tokens requires extra space)
2. String lookup is expensive

ARPIT BHAYANI

Approach 2: Trie



We can build a trie out of abuses and in just O(n) traversal mask the abuses.

For each character in text we iterate through trie

every time we get '_', ',', ... etc we reset our trie iteration (to the top)

mondays are shit.

if the abuse is found and next char is EOS or non-alphabet, we mask and write to the new string. Else the word copied as is.

The final string is then sent / broadcasted to every participant

How this fits into the system?

We typically think of databases and all to hold and query but that is very slow for this

plus * no database exposes trie *

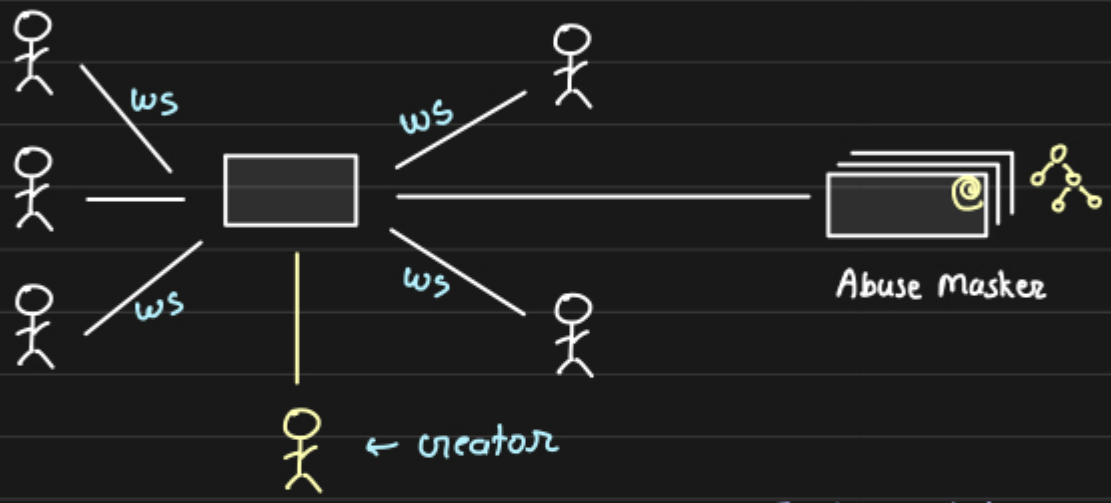
so, what do we do?

ARPIT BHAYANI

Really poor way to design this system

Have a "abuse masker" service that exposes an HTTP endpoint accepting the text and returns masked version of it

The web service will have the trie loaded in memory



Think realistically

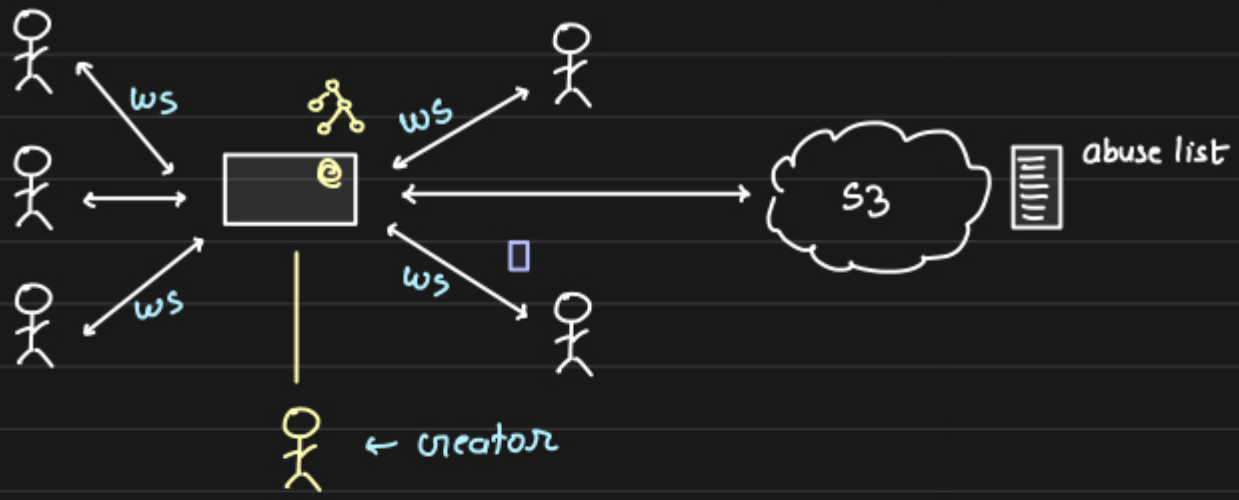
Note: Not everything needs to be a "service"

Disadvantages:

- making network call every text will be massively slow
  - ↳ Setting up TCP connection every time (3 way handshake)
  - ↳ terminating once we get response (2 way teardown)
- for even persistent connection we incur network I/O

Hence, creating a service to clean the text is a bad idea.

ARPIT BHAYANI

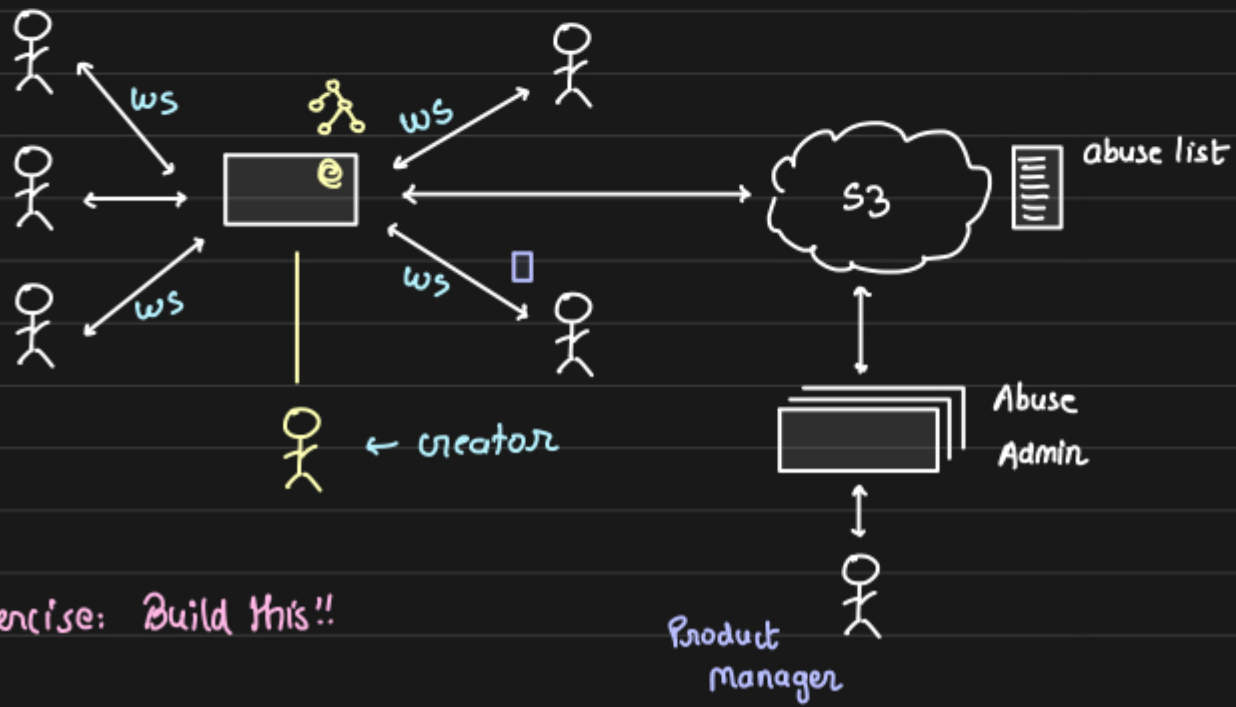


Configuring Abuse list

We need a way to update the list of abuses (internal team)

- we need a UI where abuses are listed
- we should update the abuses and push them to S3

We create Abuse Admin Service to take care of this



Exercise: Build this!!

ARPIT BHAYANI

#ASLI ENGINEERING

Designing Tinder Feed

SWIPE

BY

ARPIT BHAYANI

Motive: playing with location and keeping things efficient

Designing Tinder Feed

Design Feed for Tinder and feature to swipe left or right. For a great user Experience, a user should not be shown a profile that he/she already swiped.

Feed Criteria and Preference

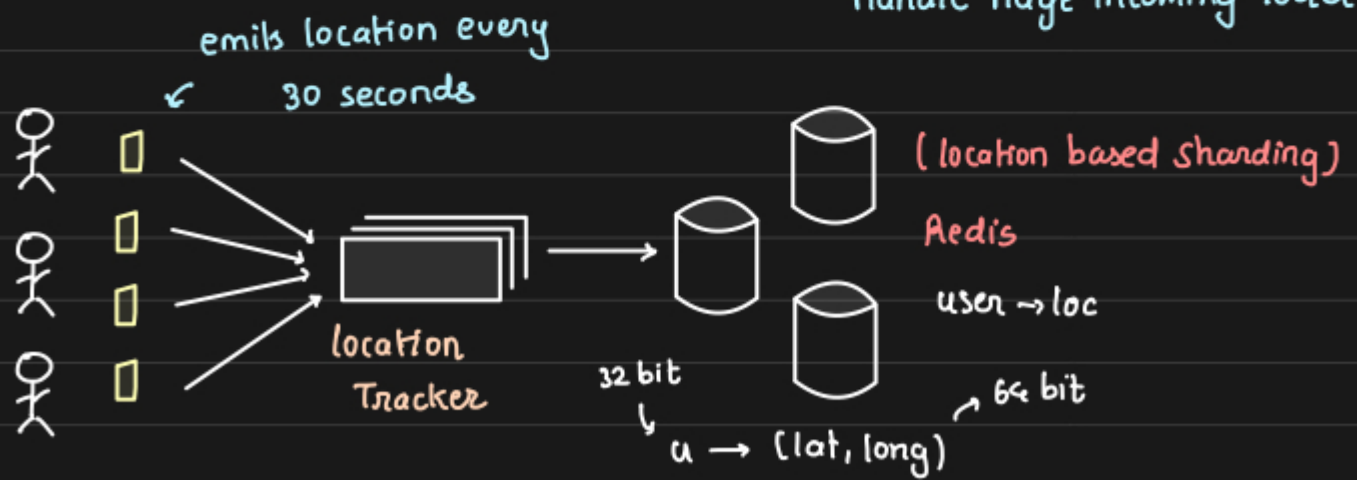
- Proximity → near the better
- Common Interest → shared interest

Capturing Proximity

User's device (app) continuously emits (lat, long) to our backend

We store this information in a database

1. that supports geo-spatial queries ← generating feed
2. can be horizontally scaled through sharding ← handle huge incoming load



12B x 50M users = 600MB

Amount of data is not the concern but the query to store and get is Hence we are sharding'

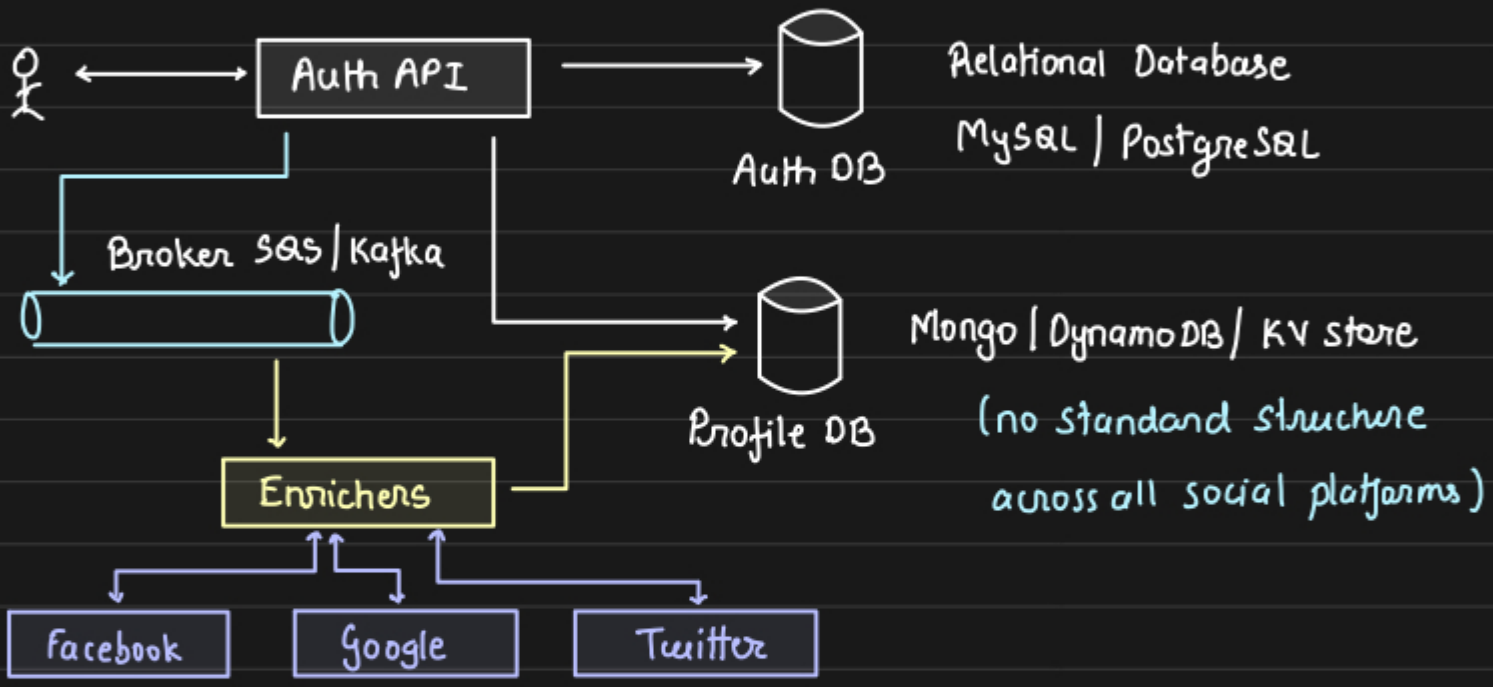
ARPIT BHAYANI

Capturing Common Interest

Profile informatio

1. Ask user to provide those details
2. Capture the details with Social login
   Google / Facebook / Twitter

Authorizing for profile / interest / connection scrapping.



Now that we have the interest information and current location we can generate feed for users...

Behaviour: Generate feed when user is about to exhaust the current one

↳ Frontend (opp) will make an API call to trigger population of the feed ← Better

↳ maintain a feed counter in backend and check every time user swipes ← Slow UX

Users swipes fast

ARPIT BHAYANI

Feed Database

user x user

Feed database can potentially explode (nxn) and will require decent storage

For each user, the DB holds Feed Items

Fetch feed item - Profile Information that user has not seen before

Approach 1

hash key — range key

Store: < user_id, candidate_user_id, created_at >

- user_id: we can partition by this
- created_at: ordered by

But whenever we are returning, we have to fetch profile info in real time and send it to user (additional n/w call)

Approach 2:

Store: < user_id, candidate_profile, created_at >

- user_id: we can partition by this
- created_at: ordered by

we require a significant storage + risk of serving stale data

but we avoid making call to profile service on runtime

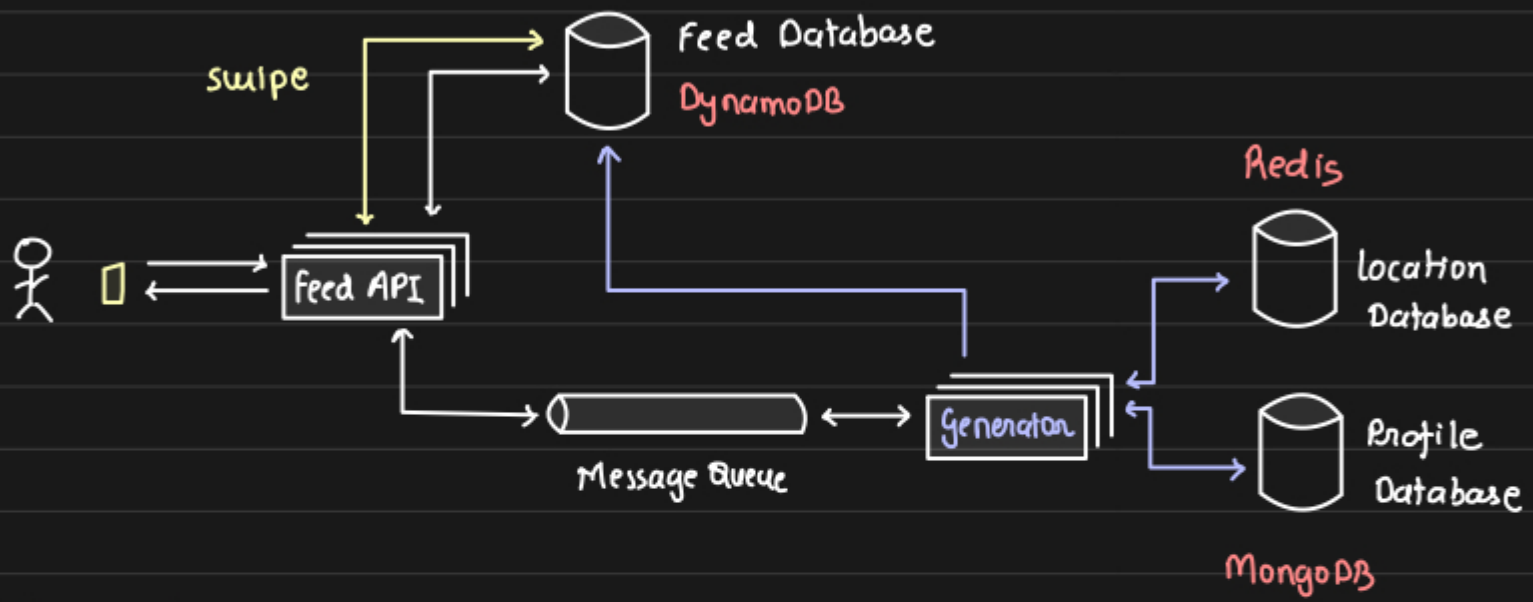
In either case, you should never store feed of a user is "list" (say MongoDB)

- ↳ it will bloat up document size on mongodb
- ↳ expensive serialization and deserialization
- ↳ iteration is difficult

ARPIT BHAYANI

Feed Generator

1. should be async, because it is time consuming
2. should be triggered by frontend



Storing Swipes

People swipe left or right to indicate interest.

* we do not need a separate DB to hold this info

and we leverage Feed Database and just add "is_interested" in each ite

Note: You do not have to create a new DB for everything

When A swipes B,

- mark is_interested in feed item
- check <B, A> in feed DB
- if entry does not exist or is_interested = False : do nothing
- else: create a match (in match DB)

Ensuring no-repetition

- When A registers a "swipe" for B
- A should never see B again in the feed
- While generating the feed, we have to check for past swipe & add only if new

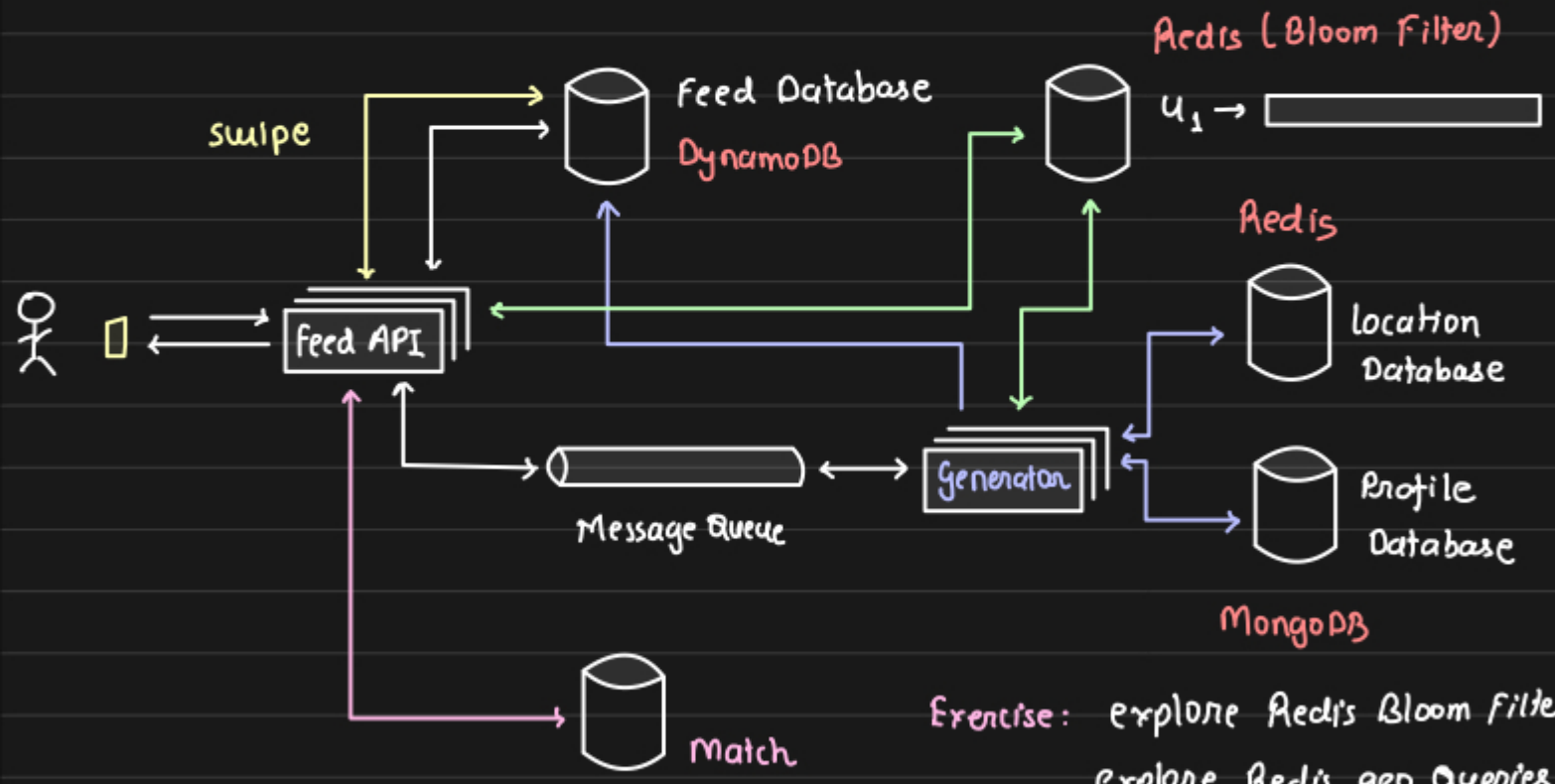
We need a definite no
approximate yes is fine

↓

classic case of Bloom Filter

We use Redis + Periodic persist to store swipe information

Bloom filter is consulted before adding any item to feed



Exercise: explore Redis Bloom Filter
explore Redis geo queries

<user_a, user_b, match_id>

↳ This match_id will be used by messaging service and all messages are for a "match"

ARPIT BHAYANI

#ASLI ENGINEERING

Designing Twitter Trends

SWIPE

BY

ARPIT BHAYANI

Motive: To understand how "many" components are needed to build at scale

Designing Twitter Trends

Twitter Trends is a page/screen where people find what's happening

← Image representing the top trend

1. # sun
   Article Top
   720 tweets ← Trend #1
2. # EPLwin
   Top article ← Trend #2

} Breaking news, entertainment, Social movement

representing the event

One article about the trend (news article)

Input to the system

almost realtime

In order to build Twitter Trends, we need all the recent tweets

Hence, we push tweets (as soon as they are published) in a persistent, high-throughput message stream - Kafka

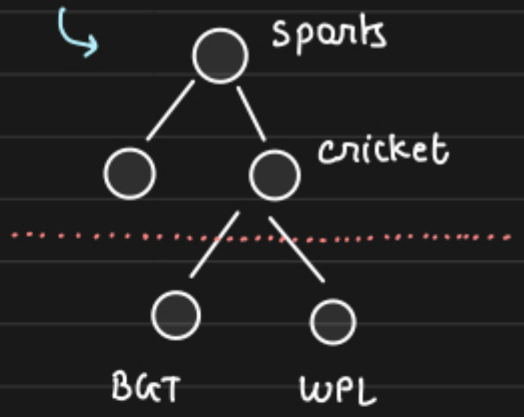
↑ Durability ↑ Volume high (350,000 Tweets per minute)

(something like Cassandra)

ARPIT BHAYANI

News Clustering

Pre-defined Hierarchy



Group similar news stories into clusters

There are various clustering algorithms that can be applied like k-means + tfidf

We have to prepare data for clustering

Process of clustering

and stores them in ElasticSearch mapped to our taxonomy

The clustering algorithm uses the crawled data and builds feature vectors to cluster the articles (supervised or semi-supervised)

Clusters we got are ranked as per recency & size

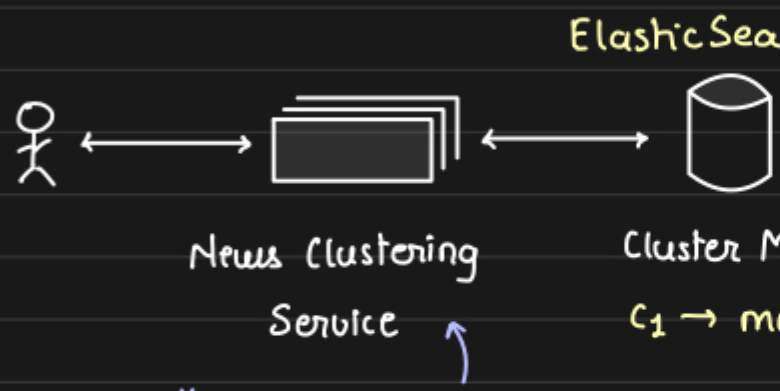
Each cluster then picks (used while rendering)

1. Top articles (media house, popularity)
2. Reference image (image from top article)
3. Keywords (common keywords from articles)

ARPIT BHAYANI

News Clustering Service

The clusters, along with metadata (top articles, keywords, top images) are stored in a database and is used to serve the trends page through News Clustering Service



For any 'trend' the News Clustering service is contacted to fetch the meta and top articles to render.

Given a "query" find the matching clusters

[used by trends, search, and discover feature of the application]

Find what's trending

not just hashtags

Trending entities are computed for a domain, location, and topic

eg: we see WPL, IND vs AUS trending

and not just hashtags

In order to find what's trending, we have to

1. prepare the data
2. Extract domain, entity and aggregate
3. score and rank

ARPIT BHAYANI

if we consider every tweet to find, what's trending

then it becomes easy to game the system

Hence, it is important to filter out "spam"

- replies
- low quality tweets
- tweet with sensitive content
- multiple tweet by same person

(clean data)

Using Named Entity Recognition and wordnet, we tag entities to every "clean" tweet

< Entity, Counter >

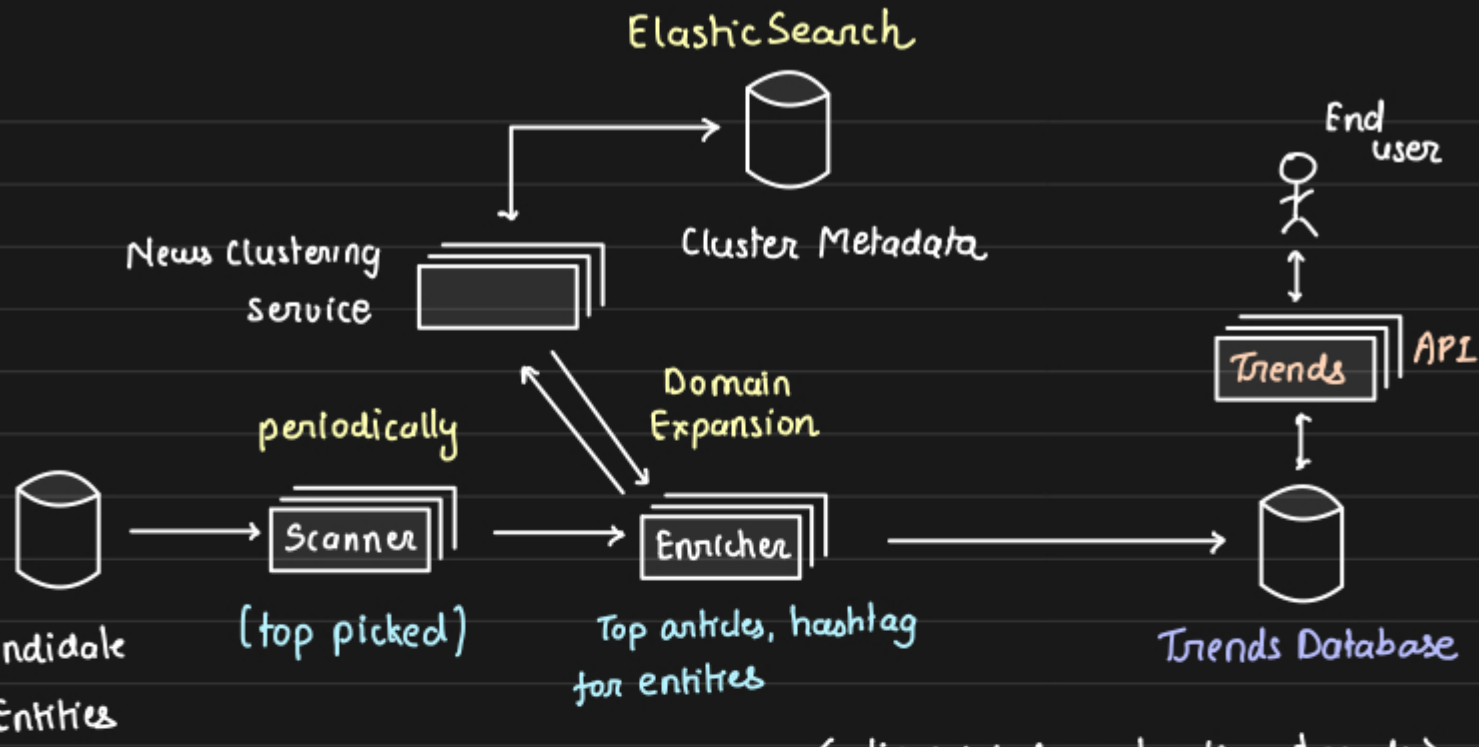
(for a given time range)

scores the entities as per "popularity", frequency and widespreadness and updates back in the DB

Thus, the candidate database now holds in a given time range, the entities that are popular along with their scores

Serving the trending

ARPIT BHAYANI



< time window, location, trends >

Each trend contains

- entity
- domain
- metadata → top tweets, top articles, statistics

Instead of looking at one huge diagram, it is better to look at one small subsystem at a time and explore it in depth.

Exercise

- Creak a comple diagram of this system
- explore clustering algorithms
- pick a text/news clustering problem on kaggle and solve it

ARPIT BHAYANI

#ASLI ENGINEERING

Designing URL Shortners

SWIPE

BY

ARPIT BHAYANI

Motive: Understand partitioning, sharding & out of the box solution

Designing URL shortners

Design a URL shortners that generates pseudo-random human readable short URLs. Upon visiting takes you to original website.

why? easy to share in sms

x no support for custom alias

~ 100M URL per month

How to shorten a URL?

Approach 1: Hash the URL

SHA 256 are 256 bits long = 16B

url.sml/abcdefghijklmnop

16 character

16B short URL are not short

Two URLs can have the same hash

* But different users can shorten the same url

Approach 2: Integer ID as short

if URL stored in DB is at ID: 1729

url.sml/1729

very short but predictable

someone can easily scrape all shortened URL

Approach 3: Custom Encoding

given we cannot expose integer directly

what if we encode (convert) them to strings

ARPIT BHAYANI

Total unique characters in short url = (a-z) + (A-Z) + (0-9)

= 26 + 26 + 10

= 52 + 10

= 62 ≈ 2^6

We define the map that maps each of 6 bit to a character

000000 → a
000001 → b
000010 → c
000011 → d

Our encoding scheme

Say a user is storing some URL (auto inc id / self assigned) and when we stored it in the DB, ID that was assigned was 79

79 → url but what is sent to user is "sml.url/bp"

79 is converted to binary → 1001111

Left pad the value, until it becomes multiple of 6 → 000001 001111

000001 → b
001111 → p

short url ⇒ url.sml/bp

But this is still predictable, so what can we do?

We define a random map of assignment (that only we know)

000000 → q
000001 → d
000010 → e
000011 → A

This makes our small url very difficult to guess.

ARPIT BHAYANI

But if one fires so back to back requests, one would know the pattern and reverse engineer the encoding (our custom encoding)

So, instead of picking sequential ID, we pick random ones?

This will form the crux of our approach !!

up until now, we do have ID → human readable code (short)

Storage

We store mapping b/w short URL and actual URL in database

Schema:

urls

short_code	url	... some other meta data
bpqA29	https://.....	
a2Am6	https://.....	

Note: short code are not derived from URL

Given our access pattern is just KV based,
we can store these in any KV store
that is partitioned by short_code

We can pick any KV store or even relational DB to hold this

code ↓ url ↓

amount of data = 100 M x (8 + 120)

per month = 128 x 100 M = 12.8 GB

Shard!

Storage is not the concern, we shard to handle load

ARPIT BHAYANI

What happens when someone visits url.sml/bp?

When someone visits the URL,

the request comes to our API server

we get the actual url from the shard

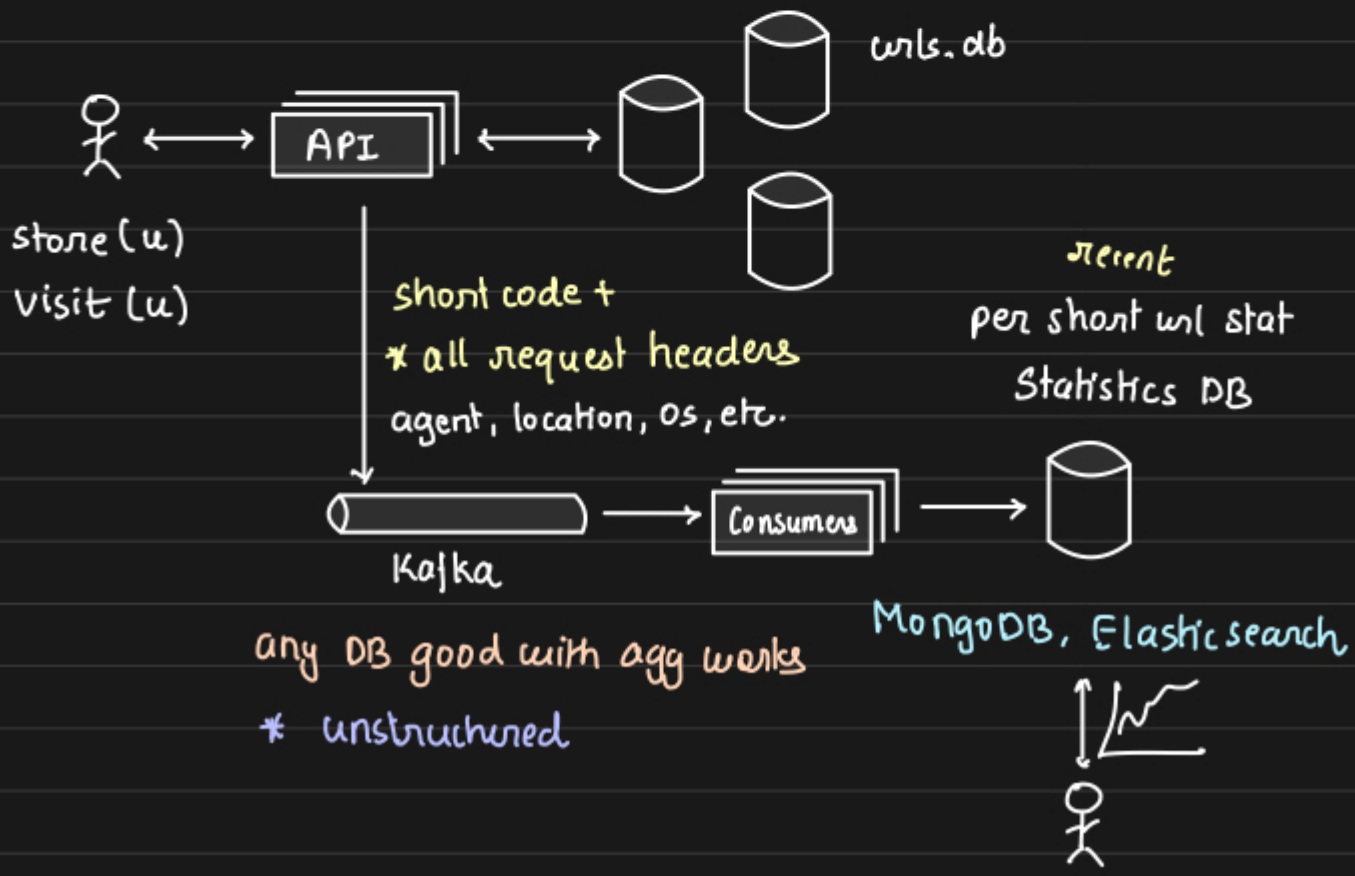
we return 301 redirect with this URL

↳ this would ensure user is redirected to original url

select * from urls
where short_code = ?

But before returning the 301 status code,

we emit an event to register the analytics



ARPIT BHAYANI

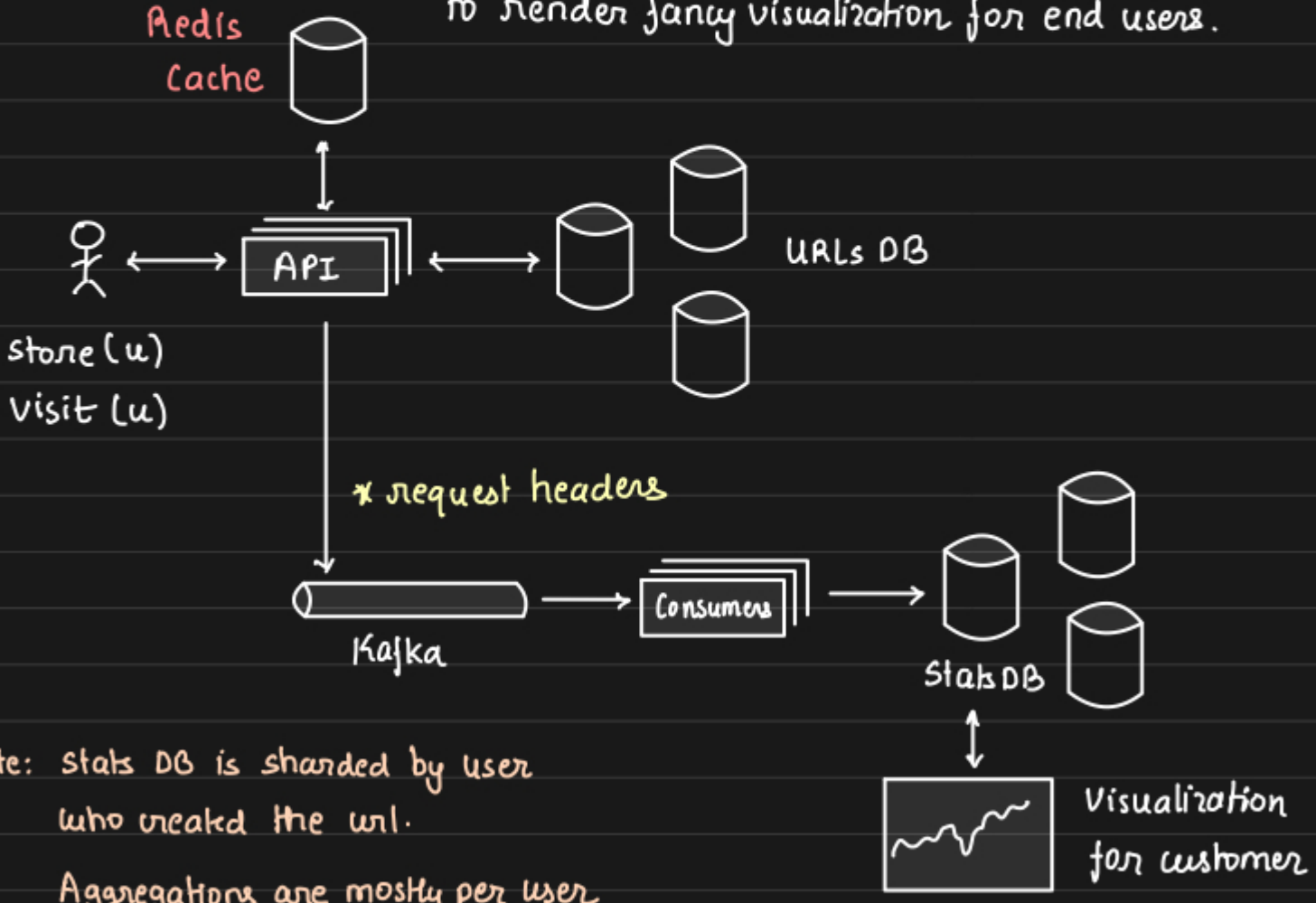
Newly published URLs are more likely to see hits

We cache the short urls that are "new" or hit "popular"

← Good usecase to fit the cache in arch. to
- reduce load on DB
- improve latency

Visualization for end user

We leverage aggregation feature of stats DB to render fancy visualization for end users.



Note: Stats DB is sharded by user who created the url.

Aggregations are mostly per user and we leverage Grafana to render them

ARPIT BHAYANI

How is short code/url generated?

- if we go one by one .... easy to predict the next one
- generating random integer is prone to collision
  - ↳ hence we have to do it uniquely pseudo random
- only one machine should not be handling this
  - ↳ too much load on one machine

100M / 30 = 3M per day
= 125K per hour
= 2K per minute (average)
peak will be way worse

We leverage partitioning, sharding and transactions to achieve this!!

We set up a few database servers having a job to issue one random integer (non-repeating) (unique) atomically

- one random integer ↑ Making it difficult to guess and reverse engineer the logic.
- atomically ↑ No matter how many requests hit the database at once there will be no collisions.

Ticket Servers

one DB handles this

On a smaller scale (0 to 1000) say we partition it in 4 ranges

start	end	current
0	250	0
250	500	250
500	750	500
750	1000	750

← the value that will be returned if read from the partition.

ARPIT BHAYANI

Getting a random integer

{ 500, 01 750, 501, 250 }

Ticket Server

ranges			
id	start	end	current
1	0	250	~~0~~ 1
2	250	500	~~250~~ 251
3	500	750	~~500~~ ~~501~~ 502
4	750	1000	~~750~~ 751

1 user wants short url

2. API server picks ticket server
   ↳ server knows available range ids ← configuration or fixed

3. API server selects one range at random

4. API server
   - selects the 'current' of the range
   - increments current by 1 if end != current

} transaction

TXN

```
SELECT * FROM ranges
  WHERE id = ?
UPDATE ranges SET current = current + 1
  WHERE id = ?
COMMIT
```

if range is exhausted, server removes the range (id) from its in-mem store (table)

Because we are picking one random range, we see a pseudorandom order
* within range the order is sequential

Now once we have the "random id", we encode and store mapping of short url & original in urls db.

ARPIT BHAYANI

We take max ids we would support, say 4 Billion (2^{32})

at 100M per month consumption, it would take 40 months ≈ 3 years to exhaust the range.

if we want to support more, we increase int to 64 bits

giving us a range till $2^{64} = 1.84 \times 10^{19}$

and it would take $\frac{2^{64}}{100 \times 10^6} = \frac{1.84 \times 10^{19}}{100 \times 10^6} = 1.84 \times 10^{11}$ months $\approx 0.1533 \times 10^{11}$ years

$= 15.33 \times 10^9$ years

$= 15$ Billion years

64 bit number ≈ 66 bit ≈ 11 char (encoded)

But if we want to restrict the short code length, just reduce the bit.

Also, instead of starting from 0, lets do it from 100,000 to have short code of length at least 3.

Say, we want short code min length = 3 max length = 8 (min)

min len = 3 = 3 char = 3×6 = 18 bits = 2^{18} ≈ 100,000 with pad zero

max len = 8 = 8 char = 8×6 = 48 bits = 2^{48} ≈ 5.62×10^{14} (max value)

Say, we want each range to exhaust in 1 year

so, it generates 100M × 12 = 1200M urls = 1.2×10^9 integers

Total ranges = $\frac{5.62 \times 10^{14}}{1.2 \times 10^9} \approx 4.5 \times 10^5 \sim 500{,}000$ ranges

(each range 1.2×10^9 int)

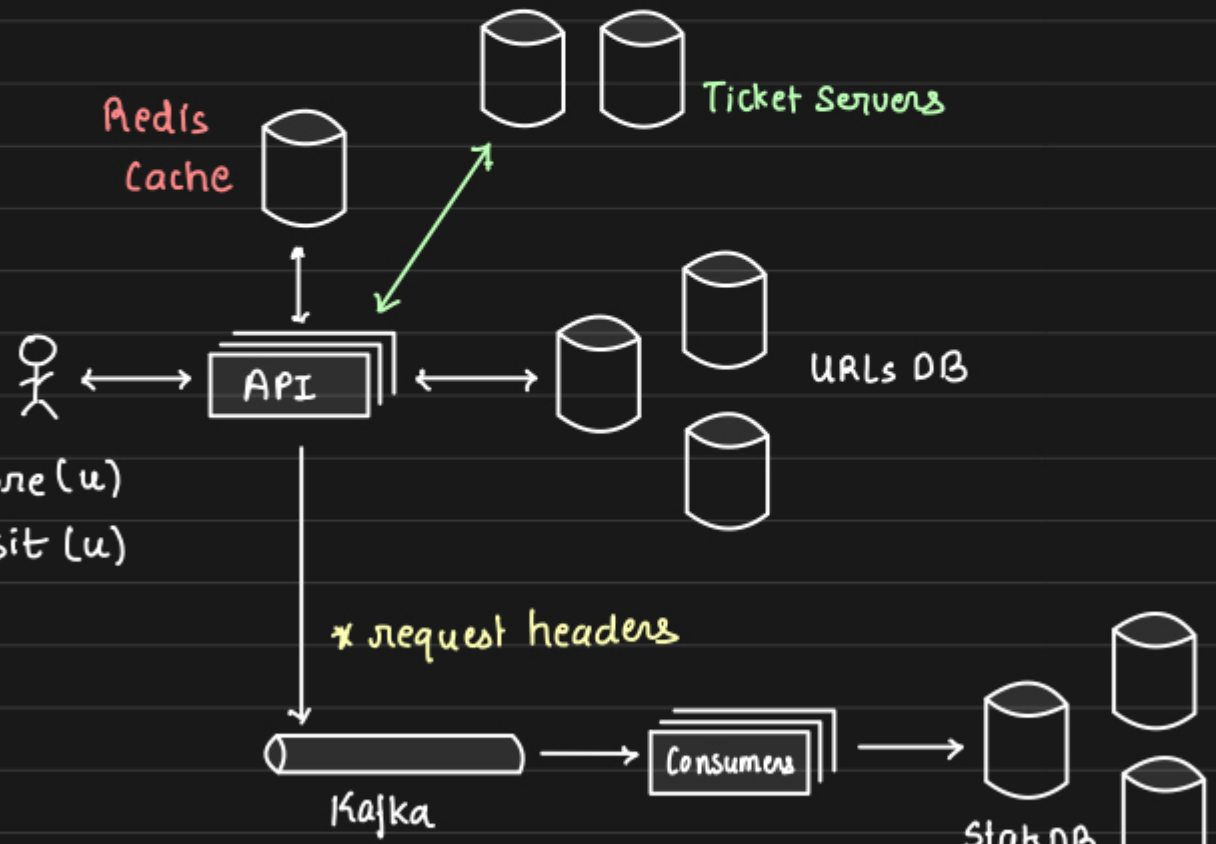
ARPIT BHAYANI

500k

Now we have a ~~million~~ ranges

100k to 1200 M	To start with, we create these 1 M entries in DB
1200M to 2400 M	and upon getting request, pick one range
2400M to 3600M	at random & get one int out of it.

if one DB becomes a bottle neck, we spin up a new DB and move some ranges there. Now API server will pick one ticket server at random.



* We can play with range length to make short urls look random
3 char first, once about to exhaust then 4, then 5, and so on.

ARPIT BHAYANI

Exercise

- implement the encoding scheme we discussed
- explore analytics we can capture and how during short url hit
- explore how exactly we share analytics data
- implement ticket server for short range (0,1000)

ARPIT BHAYANI

#ASLI ENGINEERING

Designing PasteBin

SWIPE

BY

ARPIT BHAYANI

Motive: Not storing derivables, cost effective design

Designing PasteBin (GitHub Gist)

PasteBin is an online platform that allows you to

- store any text file (content)
- share it with anyone (publicly or secretly) ↳ public to anyone / ↳ anyone with the link
- set an expiration for auto deletion
- URL generated for a paste is unique
- file stored can be long (but not huge) (10MB max)
- one who created the paste can edit it as well

Where should we store the file?

We have two options: Database or Blob store like S3

To decide which one, let's crunch some numbers

Max size per file = 10MB

Say # writes = 10M per month

Total storage req = 10M x 10MB = 10,000,000 x 10,000,000 B
= 100TB (per month)

Typical write to read = 1:50

Total data read = 5PB (Bandwidth requirement)
(per month)

Since data is huge & per file size is long enough,
there is no rationale to store this on a database.

ARPIT BHAYANI

Storage we store the files on S3 and metodata on Relational DB

schema : store

uid	name	creakdAt	visibility	owner_id
(36)	(120)	(4)	(4)	(4)
uuid			PUBLIC	
			SECRET	

10M x (36 + 120 + 12)
= 10M x 168
= 1680MB
= 1.6 GB (per month)

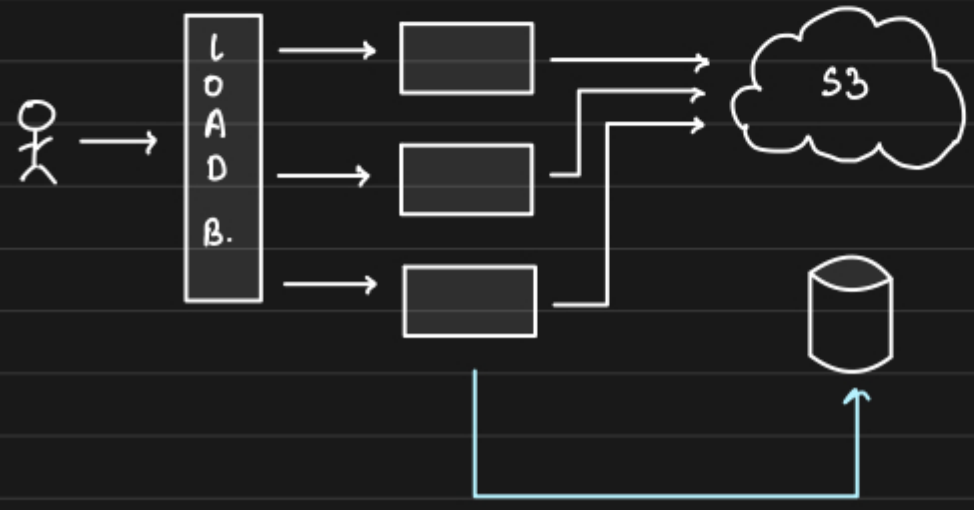
Handle-able by simple relational database ⟶

* we could also go with KV store like DDB

API and Compute

For uploading and accessing the files we need API servers.

One will not be enough, hence we add many behind load Balance



ARPIT BHAYANI

Note: we did not store S3 path in relational DB (metadb)

On S3, we configure our bucket : gist-paste

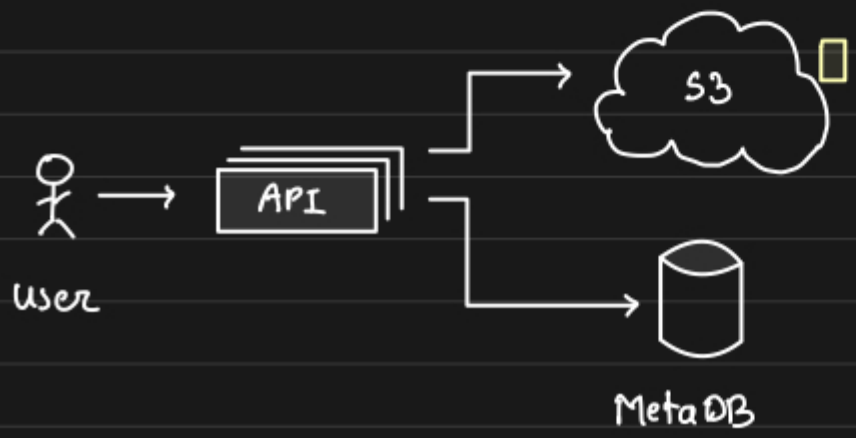
When user 1729 uploads the file with uid : 71293564

1. File is sent by the user to API server on HTTP post
2. API server gets the file
3. API server generates a random UUID
4. API server uploads file to S3 at S3://gist-paste/1729/71293564
5. API server makes an entry in meta db
6. API server returns response

uid	name	creakdAt	visibility	owner_id
71293564	abc.txt	—	PUBLIC	1729

Name is just display file on UI.

Note: avoid storing something that can be derived in the Database



ARPIT BHAYANI

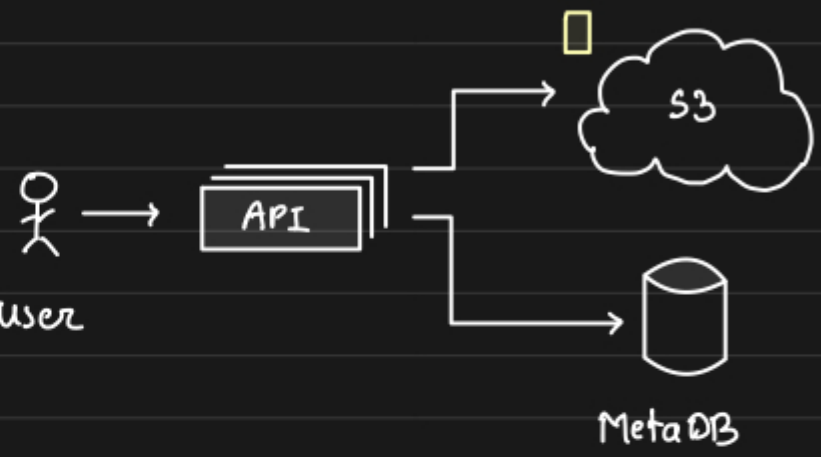
URL of the file

The URL through which we would access the file would be

https://gist.github.com/71293564

Read request flow is straight forward

* S3 path created from entry



Cache

There might be some files that are frequently accessed, so should we cache them to improve user perceived latency?

We are for sure tempted to do so? but is it worth it? ← 10MB is huge for a cache

"frequently" → how much?

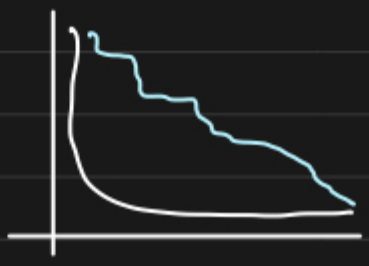
↳ Cache stores in Ram

↳ pretty expensive

If the frequency is not "huge", it does not make sense to cache the file. As a one word answer: No!!

I would rather prefer to serve 10 request from S3 than add an expensive cache and maintain it.

But be open to understanding user behaviour and access pattern



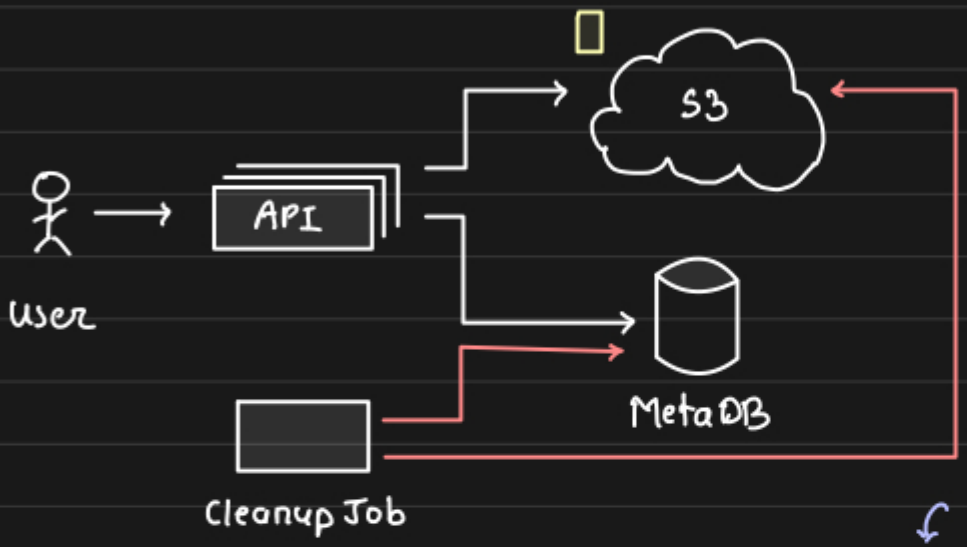
ARPIT BHAYANI

Expiration

Owner of the file can set expiration to a file, post which it becomes inaccessible and eventually deleted.

we add "expiration" column.

when file is accessed, we first check expiration time.
if beyond · return 404
else: fetch and return



Run a simple cleanup job that cleans up expired files from meta db and S3

↳ Maybe it runs once a day...

Fault Tolerance

Take periodic backup of MetaDB and store it on S3

This would help us with disaster recovery & recovering from data loss

Analytics

Whenever a file is accessed, the API server captures the request metadata and stores it on ElasticSearch for end-user analytics

↳ good for recent analytics

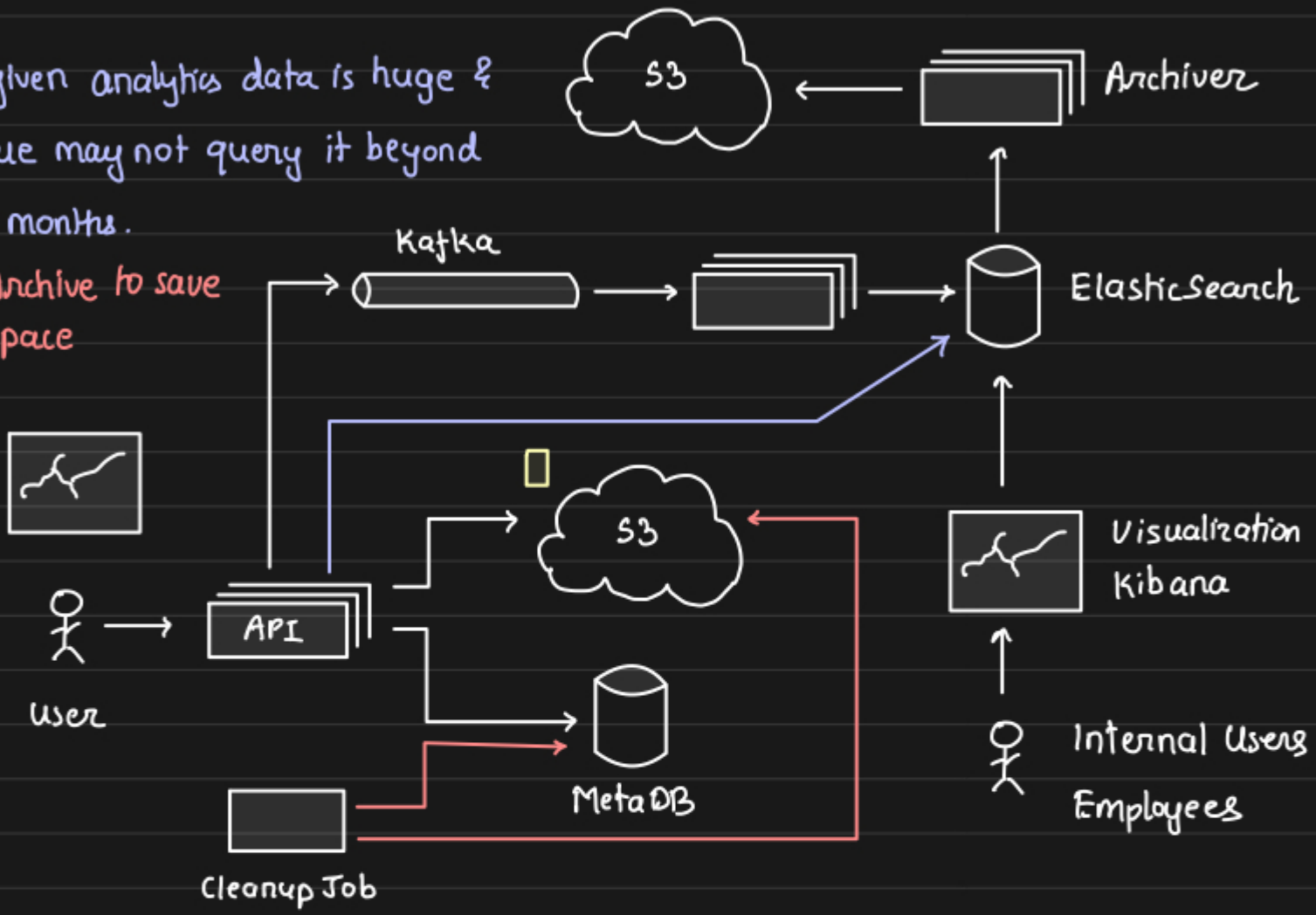
Sharded out of the box

aggregation and visualization

ARPIT BHAYANI

Given analytics data is huge & we may not query it beyond 6 months.

Archive to save space



Exercise

- capture request data (ip, region, user agent, etc) and dump it in ElasticSearch
- build visualization on top of it with Kibana
- write a small cleanup job that periodically batch deletes the data from Relational DB

compare

ARPIT BHAYANI

#ASLI ENGINEERING

Designing Fraud Detection

SWIPE

BY

ARPIT BHAYANI

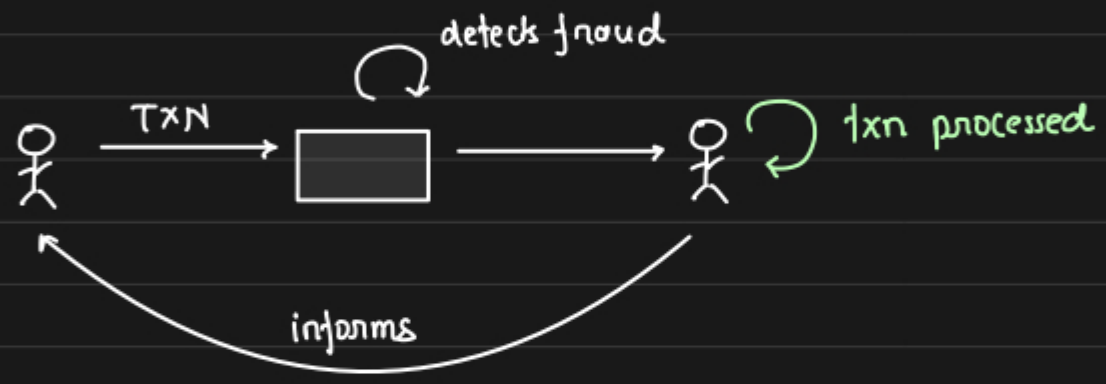
Motive: Designing systems using Big Data Technologies

Designing Fraud Detection

Financial Transactions are risky and prone to frauds. Banks have to deploy fraud detection system, that can

1. detect fraud in realtime
2. block the transaction
3. inform a customer support executive

to take consent



When a user initiates the transaction, the bank makes an entry, but before proceeding further, it checks for a potential fraud,

if not a fraud: allow the transaction

otherwise:

hold the transaction

involve the executive

call the customer and confirm

if customer says yes: allow the transaction

otherwise: abort

ARPIT BHAYANI

Storage:

we need an ability to register a transaction

ip, region, location, target bank, etc.

source_acc	target_acc	status	.....

INITIATED | BLOCKED | FRAUD | ALLOWED | DONE | FAILED

Given the schema is simple enough, choice of DB is not that complex. We start with relational.

Data is also "shardable"

* We can drop foreign key constraints here!

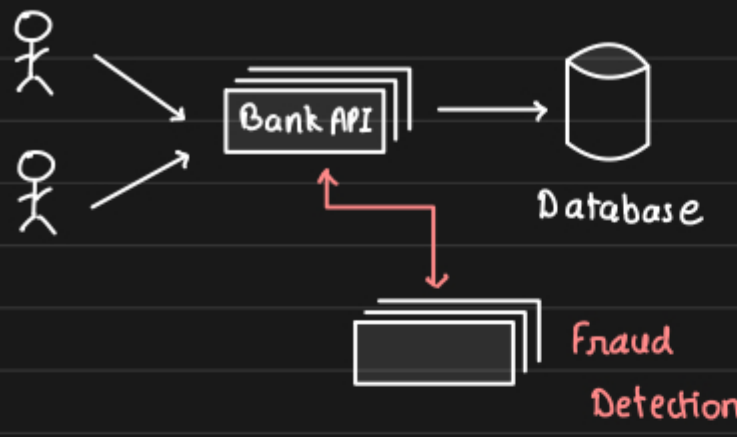
Bank API (Txn API)

Note: Bank API is a set of servers behind a load balancer.

Transaction handler (http req)

1. get source & target entry
2. extract details like (ip, location, user agent) from http request
3. send these details (synchronously) to fraud detection

ARPIT BHAYANI



Fraud Detection is a simple HTTP based service that

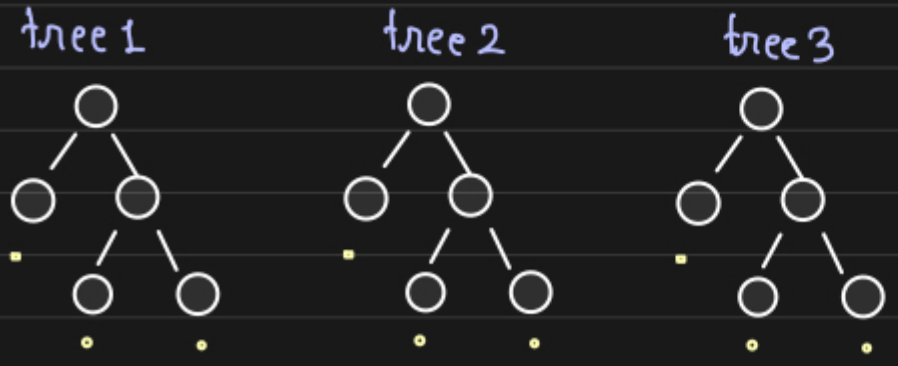
1. gets transaction and request details (over HTTP)
2. it returns if it is fraudulent or not

Note: Fraud detection service should respond within 200ms

Fraud Detection Algorithm (Decision trees | Random Forest)

Fraud Detection Service uses multiple parameters to judge if current transaction is fraud or not

We generate a lot of decision trees (random forests) from the training data and train the model. Every transaction is classified into either Fraudulent or Non Fraudulent



(loc + time) (loc + bank) (time + user + bank)

New txn is classified across all & majority is considered.

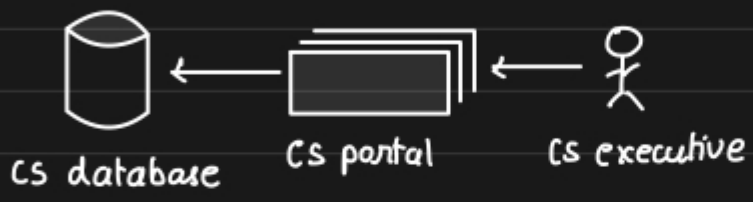
ARPIT BHAYANI

Preparing the data for training

Who knows which transactions were fraud?

Customer support and complaint portal

We need to fetch transactions that had some fraud dispute

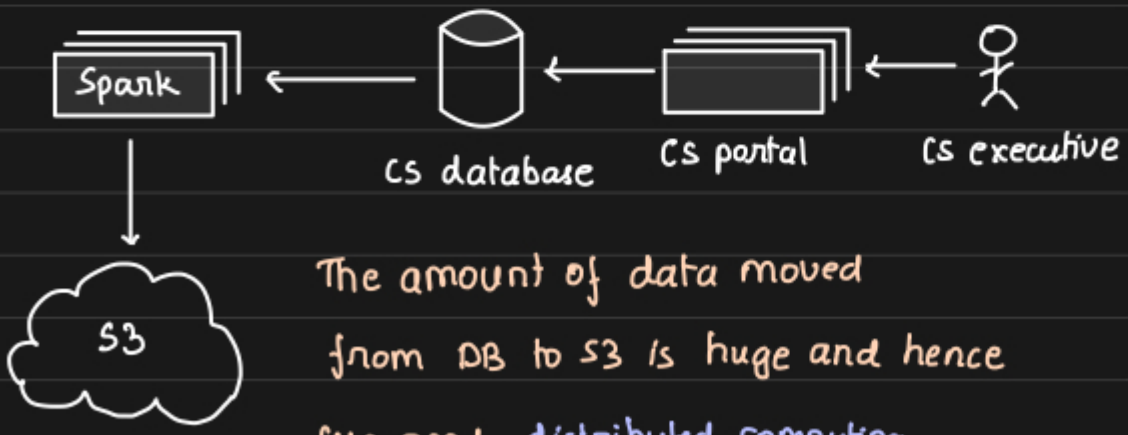


We write a job that periodically processes the data from the CS Database

extracts the fraud transactions,
communication summary,
details shared on ticket

} puts them in a blob storage like S3

We use big data processing tools like Spark to move data to S3



The amount of data moved from DB to S3 is huge and hence We need distributed computing.

One machine reading from S3 and transforming and writing to S3 will slow down the system.

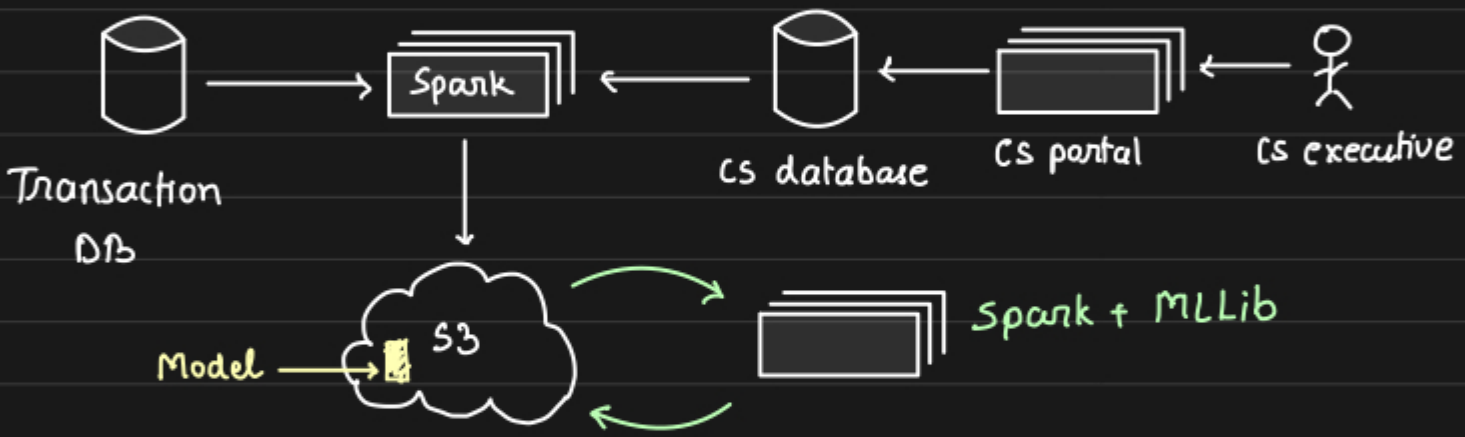
ARPIT BHAYANI

To train our model, we need data, the details about the transaction

- who sent to whom
- from which ip, location, region, currency, etc

} all this is present in transaction DB

Similar to moving data from CS DB to S3 we used spark, we will move data from



Now that all the data we want to train the model, is in S3 we use any distributed ML processing libraries to build the model

eg: using Spark + MLlib to train our "Decision Tree" and put the model back in S3

↳ just a file holding the serialized version of decision tree.

ARPIT BHAYANI

Fraud Detection using the model

Fraud detection service is simple HTTP service

that 1. loads the model from S3

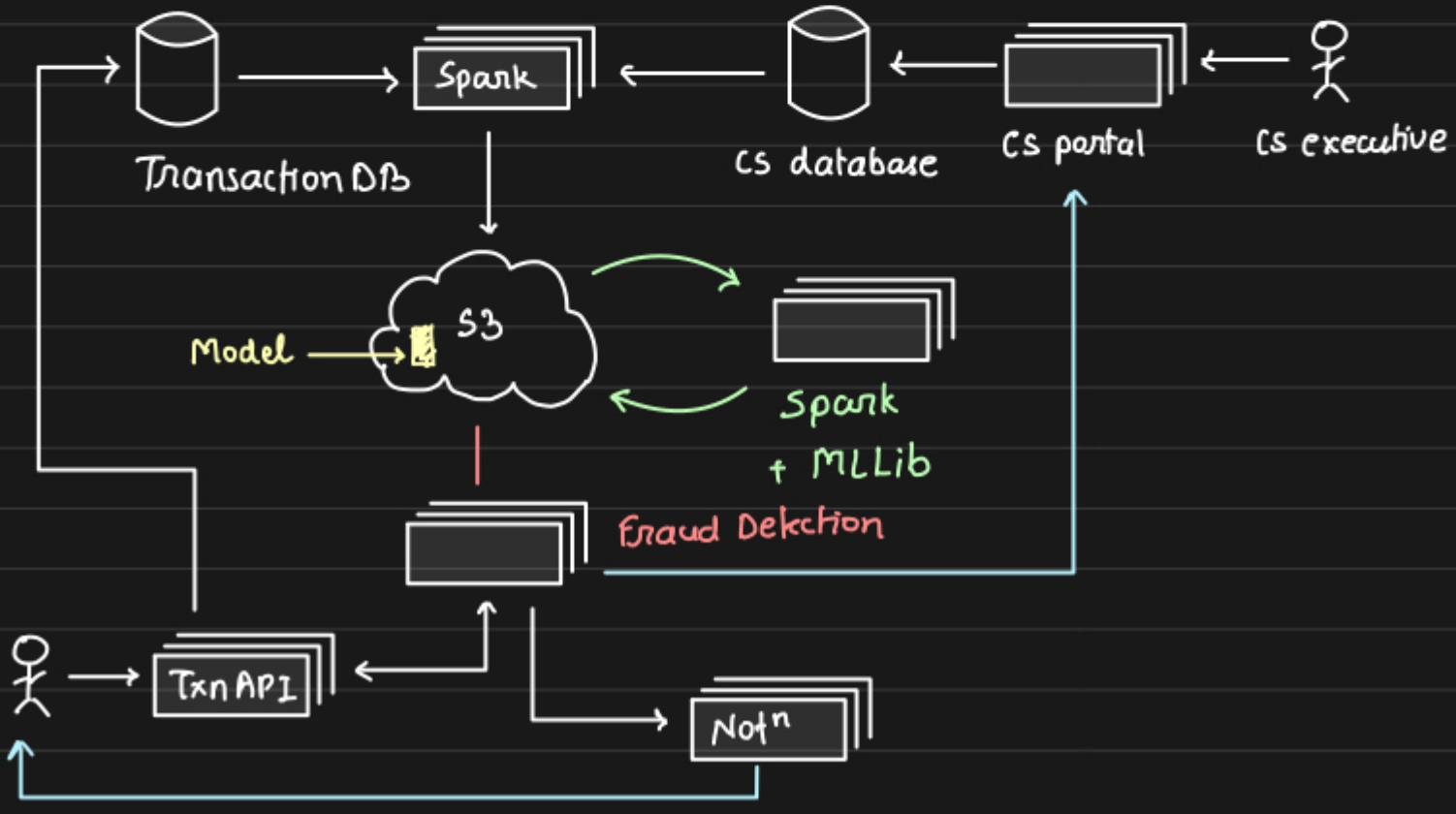
2. creates in-memory structure to classify the transactions

3. exposes API to do this synchronously

The end-user facing transaction API invokes Fraud Detection to understand if it is fraudulent or not

if not then transaction proceeds

if yes, customer service is involved & notification is sent to user



ARPIT BHAYANI

Exercise

- write a small spark job that extracts data from DB and puts them on local disk in multiple json files
- understand and write a random forest classifier
- explore MLlib of Spark to plug random forest

#ASLI ENGINEERING

Designing a recommendation engine

SWIPE

BY

ARPIT BHAYANI

Motive: Understand ML systems, plug and play, Graph DBs in ML systems

Designing a Recommendation Engine

Recommendation engines are heart and soul of consumer platforms

- YouTube
- Amazon
- Spotify

} Remember how many times you just click on a recommended video/song or purchase a book recommended by Amazon

Depending on what the platform is the "feature" changes but the overall arch remains similar.

Let's take a concrete example...

Say we are designing Recommendation Engine for Amazon

↳ Recommend products that people would buy

Naive approach → Simple, quick and dirty

Recommend the most popular items

- most popular items are same for all
- there is no personalization
- would recommend item you already bought or is out-of budget or irrelevant

eg: you already purchased iPhone, but amazon still showing it

Hence, need of personalization

you have never made a purchase > 10,000 Rs but amazon recommending 1,50,000 Rs laptop

ARPIT BHAYANI

Naive approach screams the need for personalization.

ML Approach 1: Content Filtering [Exploitation]

Show me products similar to the ones I shopped

↳ can be extended to "videos", "songs", "searches"

* works well for some niche usecase

eg: recomending similar books
recommending similar songs
recommending similar videos
recommending similar "creators"

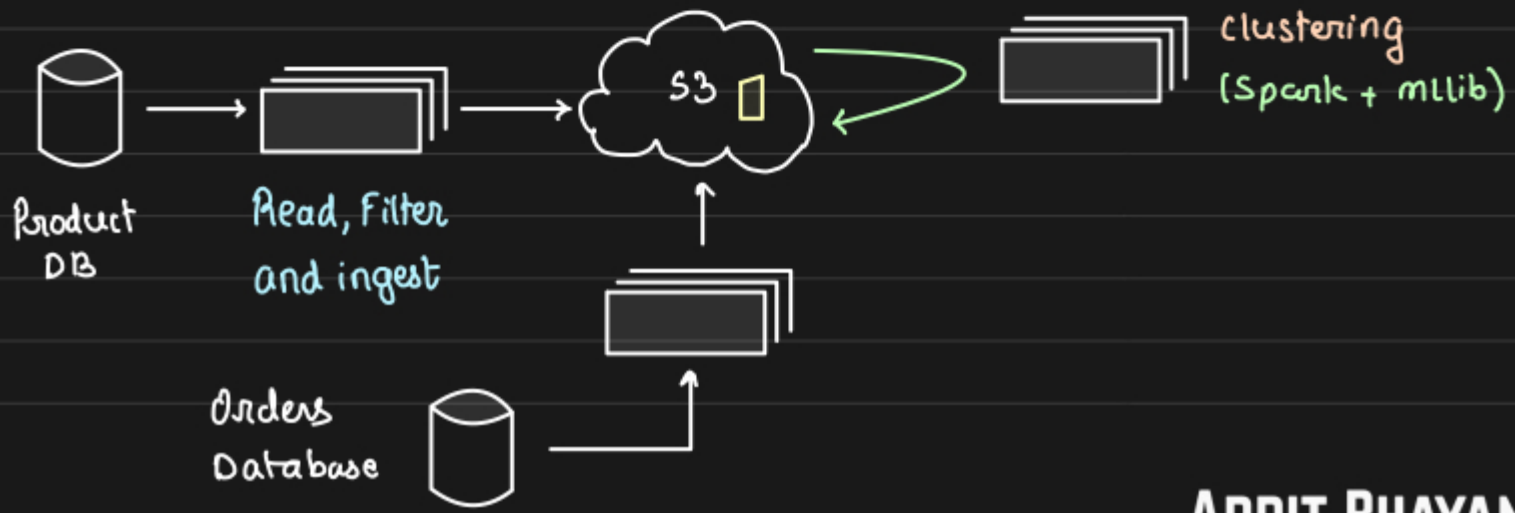
} Give people more of what they consume

How to implement?

title, description, type, tags

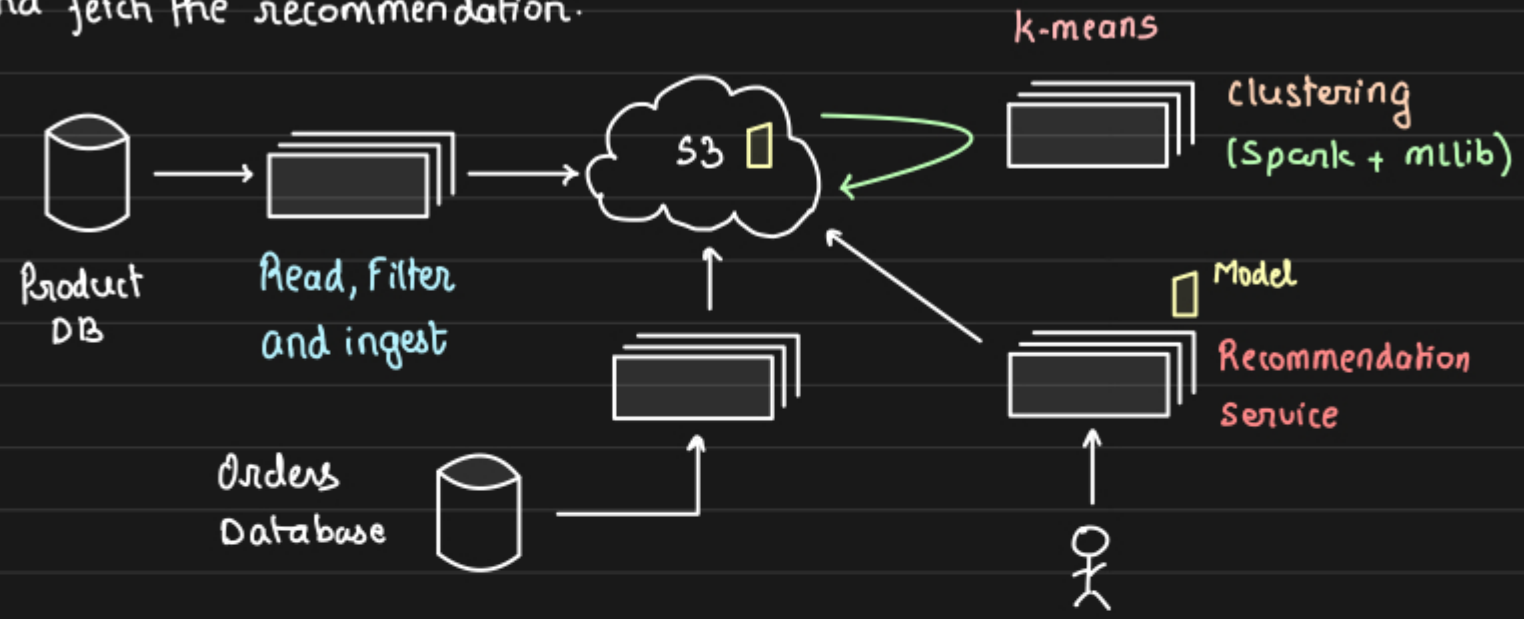
Cluster the "content" such that similar ones falls closer

When user sees/watches/reads/consumes one, we pick few more from this cluster and add to recommendation section



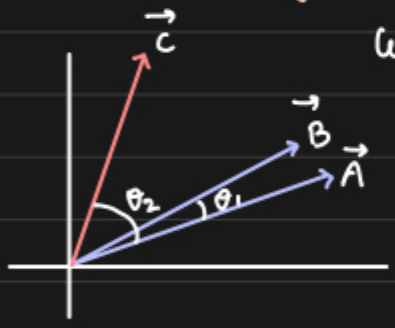
ARPIT BHAYANI

When user transacts or is on a product page we hit the recommendation svc and fetch the recommendation.



Cosine Similarity : convert product into vectors in 'n' dimensional space

Where 'n' = features like tokens, price, category, etc.



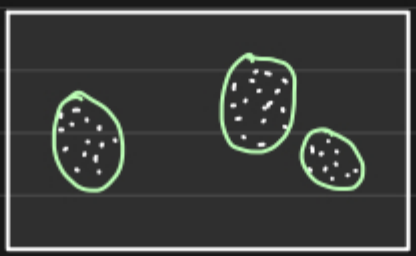
Product A and B are more similar than A & C

we know $\cos(0) = 1$ and $\cos(90) = 0$

hence, similarity = $\cos(\theta)$

Two products are similar if $\theta \to 0$ i.e. $\cos(\theta) \to 1$ (approaches)

We can also use other distances (Eucledian) to quantify similarity.



ARPIT BHAYANI

ML Approach 2 : Collaborative Filtering [Exploration]

Collaborative filtering clusters users and recommends things that other similar user bought.

eg: A <bought> iPhone

A <similar to> B

B <could buy> iPhone ← recommendation

Core idea: of all the missing edges, which one has the maximum probabilities of occuring.

Instead of recommending similar items, collaborative opts exploration

This approaches helps to spice up recommendations

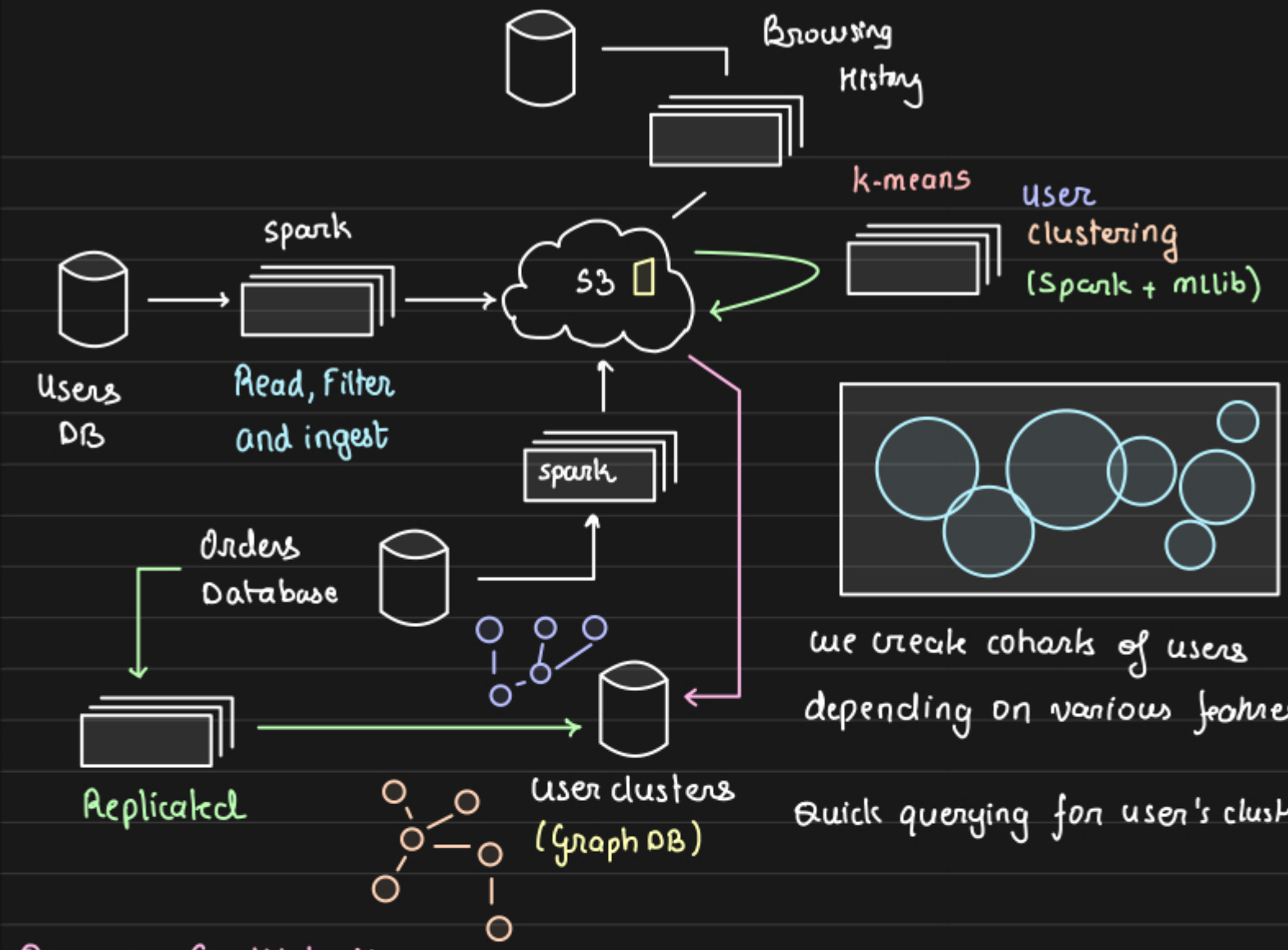
- suggest new books
- suggest new shows
- suggest new articles
- suggest new product

} they don't know they want it, yet "what's new"

How to implement ?

1. cluster the users ← on product purchase history, location, etc
2. replicate purchase history into another storage
3. train and predict

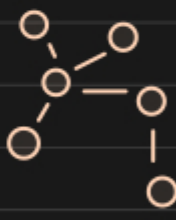
ARPIT BHAYANI



We create cohorts of users depending on various features

Quick querying for user's cluster

Querying Candidate Users



similarity score b/w users

1. pick a few users from the cluster at random
2. pick top n similar users to a particular user
3. if cluster is small enough pick all

Query Candidate Items

For each of candidate user, find items that they purchased but our user A didn't.

Simple query for a Graph database

Predicting

For each of the candidate item

predict how much would user "find it interesting" (rate the product)

as a factor of how similar they are

if u and v are very similar, they will give similar rating.

Now arrange the prediction in descending order and get top n.

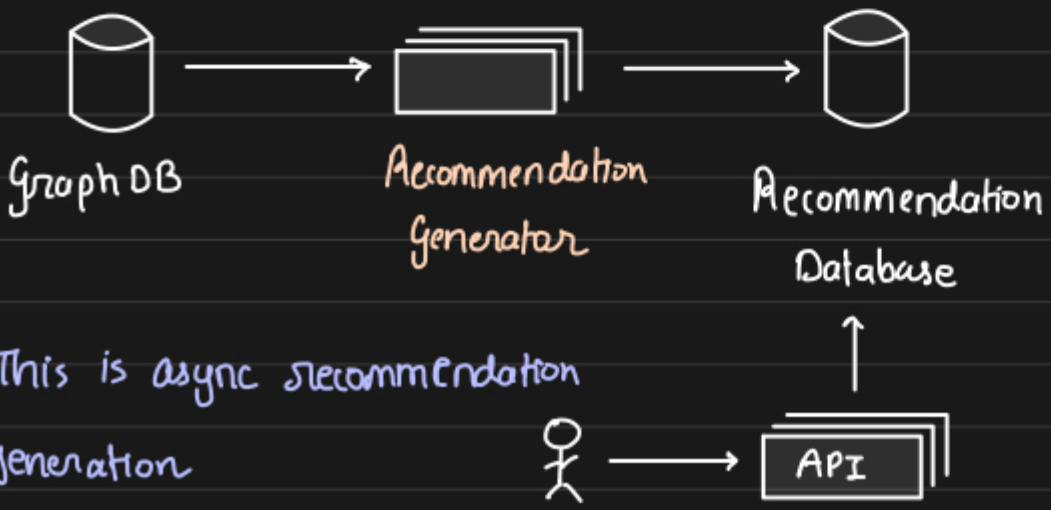
Serving Recommendations

Recommendation Database is a simple KV store that holds

{ user_id → [product 1, product 2, ····] }

Simple scoring and ranking job runs on top of Graph DB

generating recommendations for each user (as mentioned above)



ARPIT BHAYANI

Serving Recommendation & Triggering proactively

We want recommendation to be re-generated when they "exhaust"

This can be a simple event from frontend, pushed to Kafka

Similar Systems

Movie recommendation, article recommendation, ...

Recommendation in feed (twitter, instagram, tiktok)

Exercise (draw the complete system once)

1. understand clustering
2. understand collaborative filtering
3. solve one clustering question on Kaggle
4. solve one collaborative filtering question on Kaggle
5. setup Neo4j and populate with random data

   to fire query like

   give me items that are not bought by me

   but are bought by all of my friends

ARPIT BHAYANI

#ASLI ENGINEERING

Designing A Web Crawler

SWIPE

BY

ARPIT BHAYANI

Motive: Tracking, Capacity Estimation, Extensibility, Sheer Scale, working backwards

Designing a Web Crawler

Starting with the seed URLs, crawl all the links that come our way.
Process the text and create an inverted index

word → list of documents

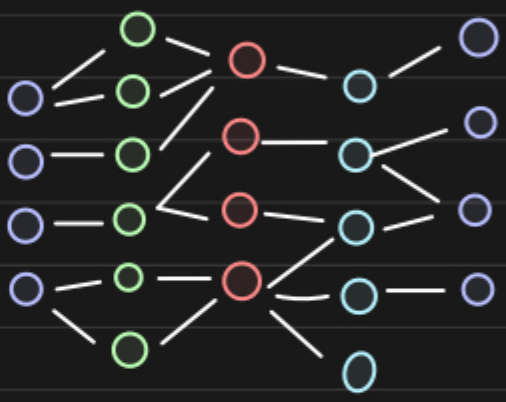
Relevance is out of scope

* HTTP pages only

Ensure efficiency, scale and no duplication of work

Crawling basics

"spiders" are used to crawl website.



Spider start with seed URL,
download the pages,
extract the links within them,
and iterate in Breadth-first-search order

eg: scrapy we can also quickly code our own

Instead of starting from top, let's work backwards

Storage of Reverse Index

For each word we need to store list of documents (URLs) containing them.

Huge number of words, huge list of document
↳ screams massive scale of data

Access is key based (given word give me list of doc)
KV database like DynamoDB

ARPIT BHAYANI

apple: [doc1, doc10, doc13, doc42 ···]

banana: [·····]

Estimating size of Reverse Index

Assume: every webpage is assigned a 32 byte identifier

There are 1,000,000,000 webpages

There are 1,000,000 unique words

On an average each word is present in 1% of pages

Size of inverted index = $1{,}000{,}000 \times (8 + 10{,}000{,}000 \times 32)$

(8 ↑ avg word length)

$= 8\text{MB} + 32 \times 10{,}000{,}000{,}000{,}000\ \text{B}$

$= 8\text{MB} + 32 \times 10\ \text{TB}$

$= 320\ \text{TB}$

Possible optimizations: Compression, Champion list

↓

instead of storing all pages where word 'w' is present we only store the 'popular' or 'significant' ones only.

Search Engine using Reverse Index

Search engine will use this reverse index to provide relevant results

How search engine determines relevance is out of scope for this system. TF-IDF

ARPIT BHAYANI

Extracting words from webpages

Assume we have webpages stored "somewhere"

we have to continuously read them, parse them, strip off style/script and tokenize and update the inverted index.

Given the amount of data to be processed is HUGE, one machine will not be able to do this

Distributed Computing

But where would we store the web pages?

Given web pages are just blob files (not actively queried)

we store them on S3 (blob store)

But how do we know which one are new ones?

In S3 the folders will be arranged/partitioned by time

```
s3://the-internet/2023/01
                    /02
                    /03/15/1200/ <batch of webpages>1.zip
                                 <batch of webpages>2.zip
                                 ⋮
                       /1205
                       /1210
                       ⋮
```

This makes it easy to read folders that interests us and also keep track of them.

ARPIT BHAYANI

How webpages get into S3?

Crawlers crawl the pages, but can they directly put them on S3?

doing a lot of microwrites (one per page) on S3 is slow and time consuming hence we batch.

Crawlers hence write the crawled page on local disk and a small daemon packs them up, zip it and upload to S3

Crawler crawls and stores data on local disk

Daemon packs and uploads to S3 (at specific location - partitioned by time)

Deletes file from HDD

How crawler crawls efficiently?

Yes, we know crawler starts with the seed URLs and all, but given there are distributed servers, they should have a DB

ARPIT BHAYANI

URL DB

All servers running crawlers co-ordinate through a common DB

The URL DB is responsible for assigning a unique id to each URL

Is responsible to hold stats about recent crawling

Amount of data is huge,

hence we need it to be sharded and KV based access is fine

DynamoDB with domain as hash key seems fine

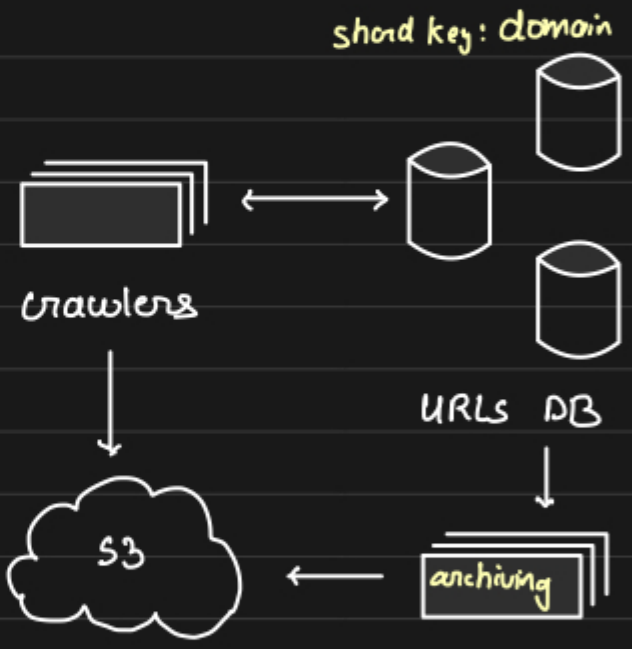
Schema

uid → id of webpage

url → url of webpage

last_crawled_at

recent_crawls: [-, -, -, -, -]



Periodically crawling stats are archived to a blob storage to keep the database optimal.

What if we stumble upon same URL multiple times in a short time window, say 1 week...

should we crawl them again?

ARPIT BHAYANI

Per domain configuration

We need a per-domain cool down period for crawling.

Whenever the crawler stumbles upon a link (URL),

1. it extracts the domain
2. loads the domain configuration
3. checks the last crawled at
4. decides to crawl or not

Separate collection

No need to have another DB. We can store this info in URL DB

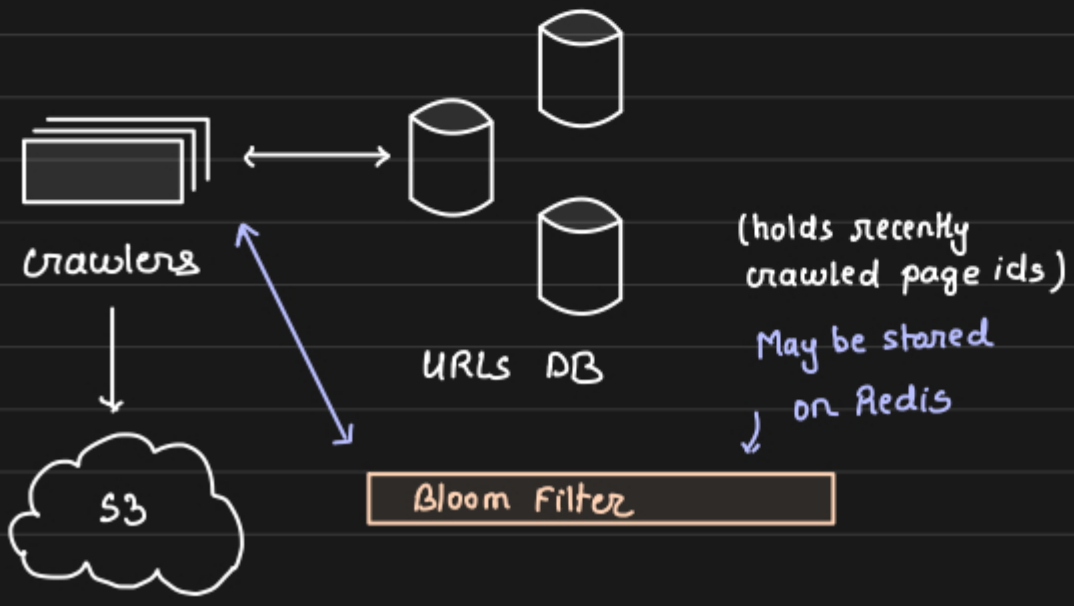
per domain config, status, reputation, rank, meta details like name, icon, etc.

Performance Optimization

We can leverage Bloom Filter to check for pages 'not recently crawled' and this would help us save a ton of DB calls.

Bloom Filter

We save recently crawled pages in Bloom Filter and it is periodically reconstructed

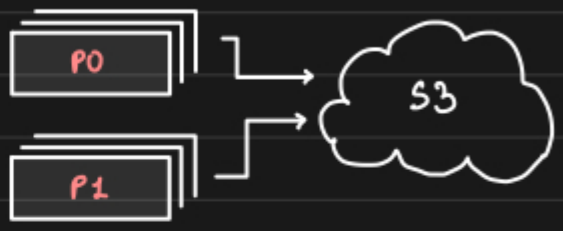


ARPIT BHAYANI

Priority Crawling

News → faster.

There are some websites that are frequently crawled & should index faster. Handling these requires us to have parallel setup of crawlers segregated by Priority

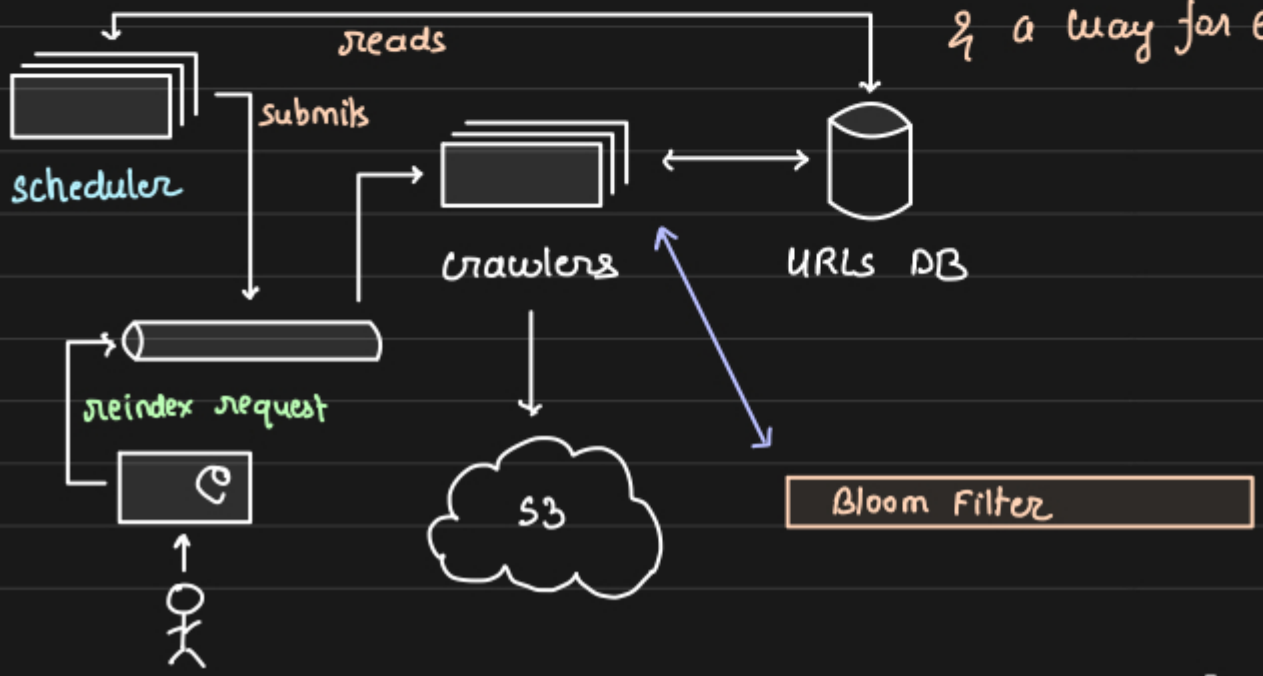


Periodic Crawling

Crawling is not a one-time activity, the pages / websites needs to be frequently crawled for 1. newer pages 2. updated information

1. define a generic crawl frequency
2. let user request for reindexing through sitemap submission

} We need a "job" that finds domains eligible for a re-index & a way for explicit reindex



ARPIT BHAYANI

Storage improvement

Entire internet is filled with duplicate content (≈ 28%) so we can reduce our storage (not processing) by the same fraction by not storing duplicate data.

Approach 1: Match web pages (super expensive, huge pages)

Approach 2: Match hash $p \rightarrow f \rightarrow h_i$ (constant for a content)

Two page with same content will have same hash

+ hash are very small (256 / 512 bits)

Risk of collision but mostly insignificant

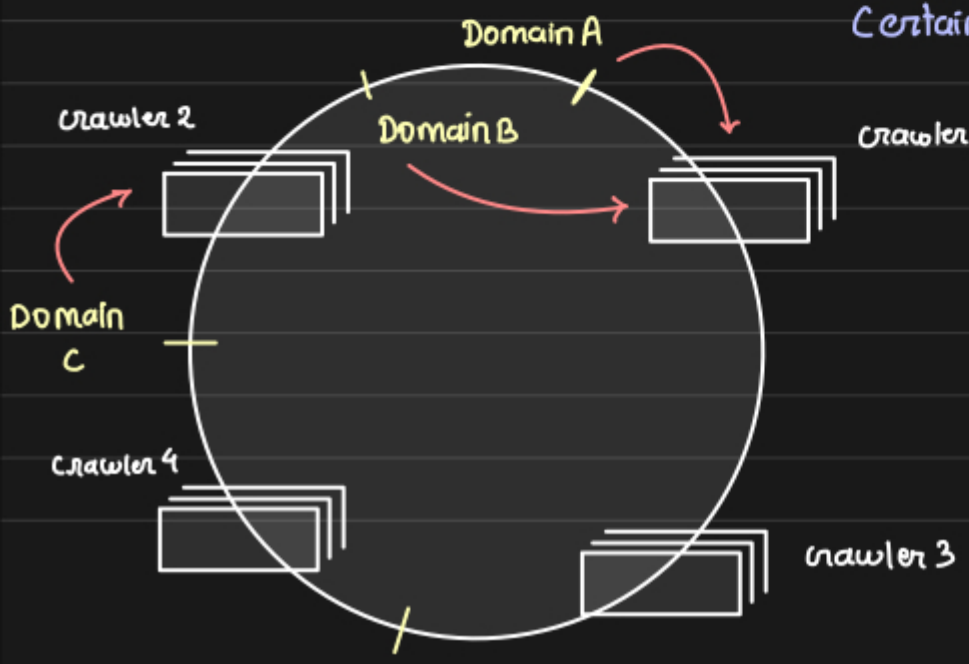
How crawlers guarantee no job overlap?

In order to guarantee uniform work distribution we can make a set of servers responsible for a set of domains

This looks like a problem of "data ownership" → Consistent Hashing

Certainly not the only way though

Fault Tolerance!



ARPIT BHAYANI

Exercise

- write a small scrapper/crawler using scrapy
- ingest data in ElasticSearch
- Query the data & play around with it
- try to not process the same webpage twice within '5' mins.
- implement consistent hashing
- Run 5 instances of crawler and ensure they do not overlap